# Daftar Isi

**Daftar Isi** ..... **1**
**Introduction to Power BI and Its Ecosystem** ..... **3**
Overview of the Power BI Platform ..... 3
Power BI Desktop Ecosystem ..... 4
Understanding Application Terminology ..... 7
Visual Cues for Fields ..... 8
Introduction to Power Query Editor ..... 11
Data Understanding ..... 12
**Understanding DAX Fundamentals** ..... **14**
Defining DAX and Its Importance ..... 14
Exploring the Internal Working of DAX ..... 15
Comparing DAX with MDX and SQL ..... 17
Creating Measures vs. Calculated Columns ..... 19
In-depth Look at Evaluation Context ..... 21
DAX Convention ..... 23
DAX Operator ..... 24
**Introduction to Index DAX Functions** ..... **25**
The Importance of Date Table in Multi Table Analysis ..... 25
Exploring Index DAX Functions ..... 29
Practical Applications of Index DAX ..... 31
**Intermediate Index DAX Techniques** ..... **34**
Advanced use of DAX Functions for Indexing Table Visual ..... 34
Dynamic Visual Table Indexing ..... 37
Combining Index DAX with other DAX function ..... 41
**How `CALCULATE()` and `FILTER()` Work in DAX** ..... **44**
Introduction to `CALCULATE()` and `CALCULATETABLE()` ..... 44
Creating Filter Context with CALCULATE() ..... 47
Using FILTER() in CALCULATE() ..... 56
Optimizing DAX with Variables for Readability & Efficiency ..... 59
Advanced Scenario for `CALCULATE()` with Filter Modifier ..... 61

algoritma

**Time Intelligence Calculations in DAX ..... 65**
The Importance of Time Intelligence in DAX ..... 65
Calculating Previous Period in DAX Time Intelligence ..... 67
Year-to-Date, Quarter-to-Date, and Month-to-Date ..... 69
Dynamic Shifting Date Calculations ..... 71
**Quick Measure and Data Analysis ..... 78**
Overview of Quick Measure ..... 78
Using Quick Measure ..... 78
**What-If Analytics in Power BI ..... 81**
Introduction to Parameterization for What-If Analysis ..... 81
Creating and Managing Parameters ..... 82
Using Parameters for Analysis ..... 84
Visualizing Different Scenarios and Outcomes ..... 87
**Predictive Analytics in Power BI ..... 90**
Finding Influence Factor with Key Influencers Visualization ..... 90
Machine Learning with Python in Power BI ..... 94

algoritma

# Introduction to Power BI and Its Ecosystem

## Overview of the Power BI Platform

Bisnis membutuhkan guide dalam mengambil keputusan. Keputusan tersebut harus tepat dan tidak boleh hanya berdasarkan insting atau perasaan. Keputusan yang dibuat berdasarkan data akan membantu bisnis melangkah lebih maju karena strategi yang diambil berdasarkan fakta (data) masa lalu yang sudah dianalisis dan disimpulkan. Power BI adalah sebuah platform yang memungkinkan kita untuk terhubung ke sumber data, menganalisis dan memvisualisasikan temuan penting dari data, dan membagikan hasil temuan tersebut ke siapapun yang membutuhkan. Temuan dari data atau yang biasa disebut sebagai insight akan membantu stakeholders menentukan langkah dan strategi bisnis kedepannya.

Berdasarkan perangkat penggunaannya, Power BI terdiri dari tiga jenis:

- **Power BI Desktop**, aplikasi berbasis OS Windows.
- **Power BI Service**, layanan berbasis web browser.
- **Power BI Mobile**, aplikasi berbasis mobile apps.

Ketiga aplikasi tersebut mempunyai tujuan yang berbeda-beda namun saling terhubung satu sama lain sehingga memungkinkan berbagai pengguna dapat mengakses dashboard yang sama. Di coursebook ini kita akan membahas penggunaan Power BI Desktop untuk memenuhi kebutuhan Business Intelligence.

**Power BI Desktop** digunakan untuk membuat visualisasi dan dashboard. Seluruh kebutuhan dalam membuat visualisasi kita selesaikan di sini. Selain itu, kebutuhan seperti transformasi dan penyesuaian data juga bisa dilakukan di Power BI Desktop dengan bantuan Power Query Editor. Pembentukan metrik-metrik bisnis diselesaikan dengan menggunakan Data Analysis Expressions (DAX).

**Power BI Service** digunakan untuk publikasi dan berbagi laporan atau dashboard yang sudah dibuat di Power BI Desktop. Walaupun Power BI Service juga menawarkan fitur untuk membuat visualisasi dan dashboard tetapi kemampuannya terbatas dan tidak secanggih Power BI Desktop. Power BI Service juga dioptimalkan untuk self-service analytics sehingga manajemen atau stakeholders dapat melakukan eksplorasi sederhana terhadap data perusahaan tanpa harus menggunakan aplikasi desktop yang terinstal. Selain itu, data yang sudah ditransformasi dan berbagai metrik yang sudah dibuat di Power BI Desktop juga akan tersimpan di Power BI Service dan dapat digunakan kembali di Power BI Desktop untuk membuat laporan atau dashboard lain dengan keperluan data yang sama. Pengguna tidak perlu lagi melakukan pembersihan, transformasi, ataupun membuat metrik yang sama pada data untuk membuat laporan atau dashboard pada file yang berbeda.

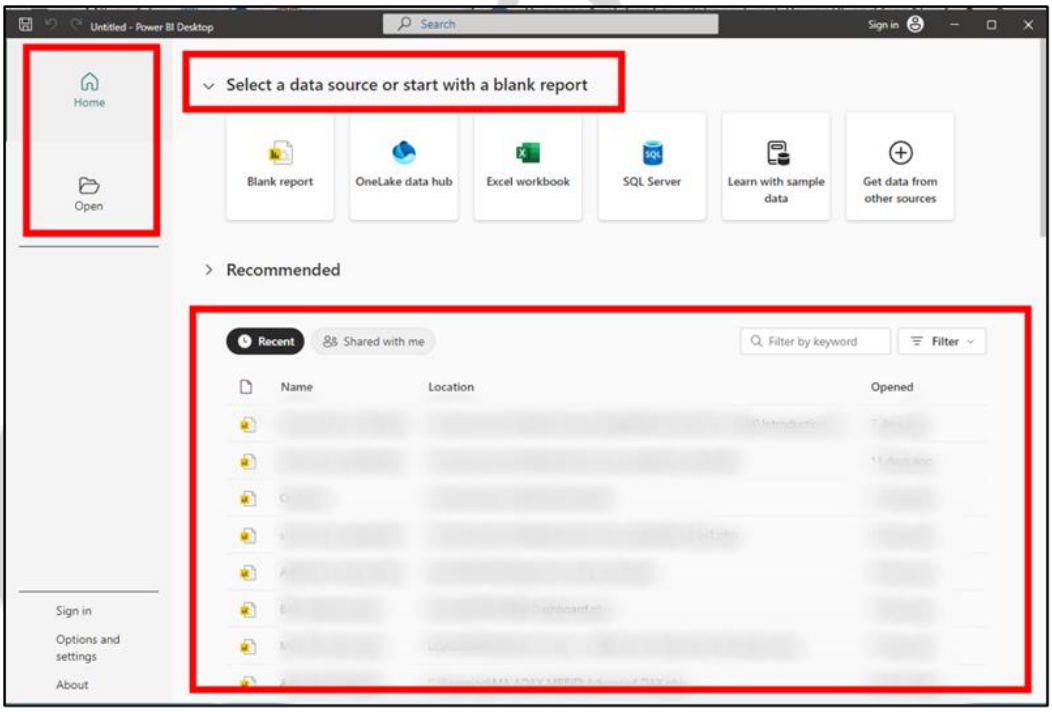
**Power BI Mobile** digunakan untuk melihat laporan dan dashboard Power BI yang ada di Power BI Service. Bayangkan Power BI Mobile ibaratkan galeri di smartphone karena fungsinya

memang hanya untuk melihat dashboard. Pengguna tidak memiliki akses untuk eksplorasi data seperti di Power BI Service. Pengguna juga tidak bisa melihat model data seperti yang dapat dilakukan di Power BI Service. Power BI Mobile sangat cocok digunakan oleh para top-level perusahaan yang hanya perlu melihat laporan dengan mudah melalui telepon genggam mereka

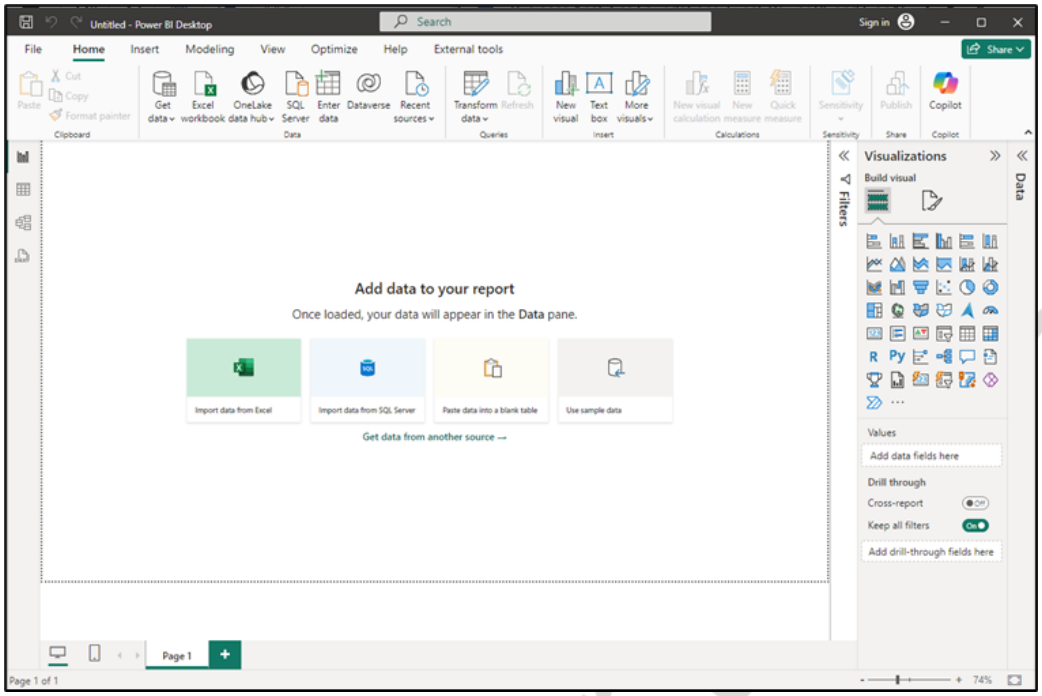
## Power BI Desktop Ecosystem

Power BI Desktop dapat di-install secara gratis melalui Microsoft Store ataupun mengunduh file instalasinya dari Website Microsoft Power BI Desktop. Kita dapat menggunakan Power BI Desktop tanpa masuk menggunakan akun tetapi hasil visualisasi ataupun dashboard hanya bisa disimpan di lokal komputer. Jika kita masuk menggunakan akun Microsoft maka hasil visualisasi atau dashboard dapat kita simpan secara online dan dibagikan. Akun yang bisa digunakan untuk login ke dalam Power BI Desktop hanya akun sekolah dan organisasi atau perusahaan, akun personal tidak bisa.

Ketika pertama kali membuka Power BI Desktop, kita akan melihat tampilan welcome screen. Dari tampilan awal ini kita bisa langsung menghubungkan ke data melalui opsi “Select a data source…”, membuka file terakhir yang dikerjakan melalui opsi “Recent”, atau membuka file kerja lain melalui opsi “Open”. Selain itu, juga ada tombol Home yang sama dengan fungsinya ketika kita memilih “Blank report”

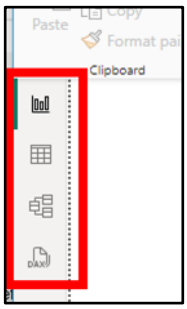



Berikut adalah tampilan lembar kerja Power BI Desktop:



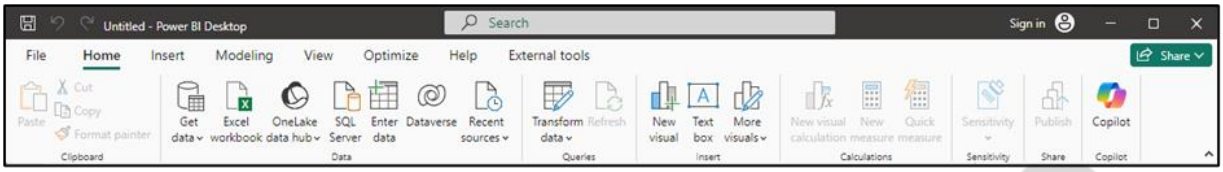

**Di bagian kiri lembar kerja** kita akan menemukan 4 opsi view yang tersedia:

1. **Report view**
   Ini adalah view default dari Power BI Desktop ketika kita membukanya. Di Report View kita membuat visualisasi, menyusun grafik-grafik menjadi satu halaman dashboard, dan mengatur tampilan halaman dashboard.
2. **Table view**
   Table view merupakan tempat kita melihat data. Di sini data disajikan dalam bentuk tabel seperti umumnya spreadsheet.
3. **Model view**
   Tempat untuk memodelkan data yang kita gunakan. Jika kita menggunakan tabel yang berelasi maka model view adalah tempat kita mendefinisikan relasi dan kardinalitas antar tabel.
4. **DAX Query View**
   Untuk melihat dan mengevaluasi DAX yang akan digunakan di dalam Power BI Desktop

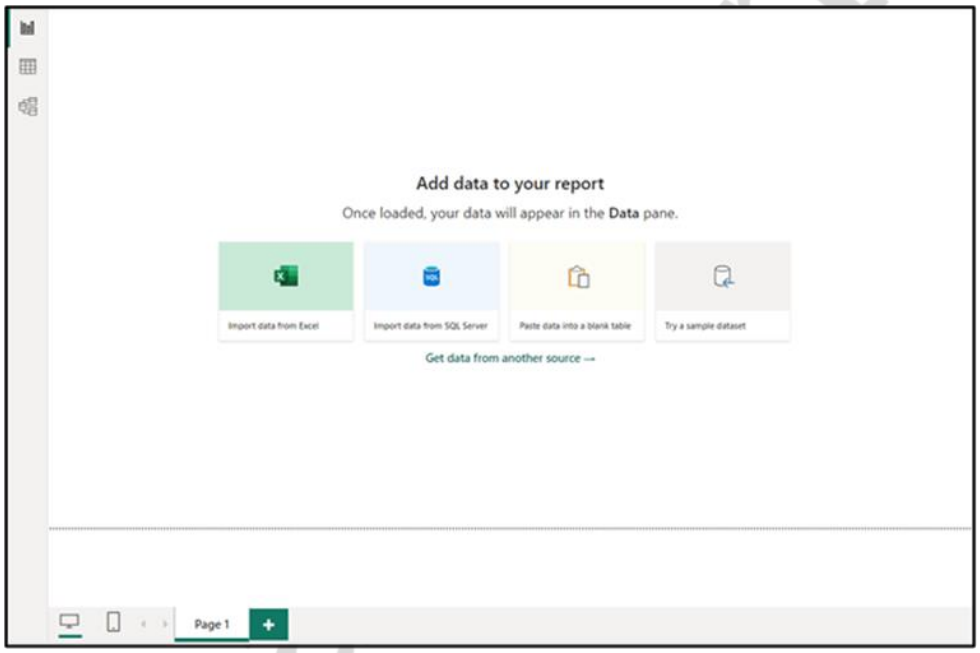



**Di bagian atas lembar kerja** kita akan menemukan ribbon selayaknya aplikasi Microsoft pada umumnya. Di sinilah berbagai menu yang tersedia diletakkan. Tampilan ribbon akan berbeda-

beda tergantung mode view yang kita gunakan. Di bawah ini adalah contoh ribbon ketika kita sedang di mode Report View.




**Di bagian tengah lembar kerja** adalah kanvas tempat kita membuat visualisasi dan menyusunnya menjadi dashboard. Halaman di Power BI desktop adalah berbasis report, artinya kita bisa membuat dan meletakkan lebih dari satu visualisasi di halaman yang sama. Kita bisa membuat halaman baru di bagian bawah kanvas.




**Di bagian kanan lembar kerja** ada tiga panel:

- **Panel Filters**
  Panel untuk menampilkan dan mengkonfigurasi filter yang diterapkan pada visualisasi. Ada tiga level filter di Power BI
  - Visualization: diterapkan hanya pada visualisasi yang bersesuaian
  - Pages: diterapkan pada seluruh visualisasi di halaman yang bersesuaian
  - All Pages: diterapkan pada seluruh visualisasi di semua halaman
- **Panel Visualizations**
  Untuk mengkonfigurasi visualisasi. Panel visualisasi memiliki 3 tab, yaitu:
  - Build Visual: mengatur kolom yang digunakan untuk membangun visualisasi
  - Format Visual: mengatur formatting dari visualisasi yang digunakan seperti warna, huruf, dan lainnya.
  - Analytics: untuk menambah kemampuan analitik dari visualisasi seperti menambah garis referensi, garis konstan, garis tren, dll.

- **Panel Data**
  Menampilkan semua tabel yang sudah kita import ke dalam Power BI. Jika kita expand tabel yang ada maka kita akan melihat kolom-kolom yang tersedia.

Tiga panel tersebut secara default akan muncul di halaman report view. Kita bisa menggunakan tombol expand-collapse di sebelah kanan atas setiap panel untuk menutup dan membuka panel-panel tersebut sesuai kebutuhan.

## Understanding Application Terminology

Ada beberapa istilah di Power BI Desktop yang perlu untuk kita ketahui sebelum memulai menggunakannya.

**Semantic Model**
Semantic Model adalah kumpulan data yang diimpor atau terhubung ke Power BI Desktop dari berbagai sumber. Kita akan mudah mengenali istilah Semantic Model dengan nama Dataset pada umumnya. Dataset atau Semantic Model menjadi dasar dari semua visualisasi dalam Power BI. Dataset dapat berasal dari file Excel, database, layanan cloud, atau bahkan data streaming langsung. Di dalam Semantic Model juga termasuk pengaturan tabel, hubungan, dan perhitungan yang menentukan bagaimana data saling berinteraksi dan digunakan dalam laporan.

**Visualization**
Visualisasi adalah representasi grafis dari data. Di Power BI Desktop, ada banyak jenis visualisasi yang dapat digunakan dan yang umum meliputi diagram batang, diagram garis, diagram lingkaran, scatter plot, tabel, peta, dan lainnya. Setiap visualisasi membantu pengguna memahami data dengan lebih mudah.

**Pages**
Power BI memberikan istilah Page selayaknya Sheet ketika kita bekerja dengan Excel. Page berisi kumpulan visualisasi (seperti grafik, diagram, dan tabel) yang didasarkan pada data yang dimuat ke dalam Power BI Desktop.

**Report**
Report dapat terdiri dari kumpulan Page, di mana setiap Page berisi berbagai elemen visual yang membantu dalam analisis data. Kita sangat mungkin menyelesaikan satu buah Dashboard dalam satu Page sehingga tidak jarang satu Report hanya berisi satu Page. Namun, pada beberapa kasus Dashboard yang kita buat bisa saja memiliki lebih dari satu topik sehingga kita memecahnya menjadi beberapa Page.

**Power Query Editor**
Power Query Editor adalah alat di Power BI Desktop yang memungkinkan pengguna untuk memuat, membersihkan, mentransformasi, dan membentuk data sebelum digunakan dalam laporan. Pengguna bisa menghapus kolom yang tidak diperlukan, menyaring data, menggabungkan tabel, dan melakukan berbagai tugas transformasi data lainnya.

**Relationships**
Relationships adalah koneksi antara tabel data yang berbeda dalam Power BI berdasarkan kolom yang sama. Menetapkan relationships memungkinkan pengguna untuk menggabungkan dan menganalisis data dari beberapa tabel dan dari berbagai sumber data..

**Fields**
Fields adalah elemen yang bisa kita gunakan untuk membangun visualisasi yang berasal dari Semantic Model. Kolom di dalam data juga termasuk ke dalam istilah Field. Semakin dalam kita menggunakan Power BI maka kita akan tahu bahwa ada banyak elemen lain selain kolom yang bisa digunakan untuk membangun sebuah visualisasi.

**Calculated Columns**
Calculated Columns adalah kolom tambahan yang ditambahkan ke data yang nilainya dihasilkan dari kolom yang ada menggunakan DAX. Kolom ini dihitung baris per baris dan disimpan dalam model data (Semantic Model). Hasil kolom baru ini akan muncul di dalam data dan dapat divalidasi melalui Table View. Calculated Column merupakan elemen yang bisa digunakan untuk membangun visualisasi sehingga ia juga termasuk ke dalam Field.

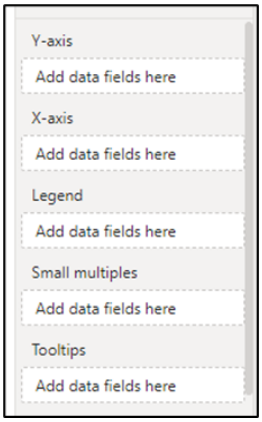
**Measure**
Measure adalah perhitungan yang dibuat menggunakan DAX (Data Analysis Expressions). Perhitungan ini bersifat dinamis dan mengagregasi data (misalnya, jumlah, rata-rata, hitungan). Measure termasuk elemen yang bisa digunakan untuk membuat visualisasi sehingga termasuk ke dalam istilah Field. Berbeda dengan Calculated Columns yang menghasilkan kolom baru pada data, Measure menghasilkan sebuah nilai skalar yang hanya bisa dilihat dengan membuatnya di dalam visualisasi.

**DAX (Data Analysis Expressions)**
DAX adalah bahasa yang digunakan di Power BI Desktop untuk membuat perhitungan dan ekspresi khusus. DAX digunakan dalam pembuatan Measure, Calculated Columns, dan Tabel untuk memanipulasi dan menganalisis data. DAX mirip dengan rumus di Excel tetapi lebih canggih.

**Parameter**
Parameter adalah suatu elemen yang nilainya bisa diganti-ganti oleh pengguna. Parameter digunakan untuk membuat visualisasi dan dashboard yang dinamis. Pengguna dapat menetapkan parameter untuk mengubah kueri, filter atau tampilan visualisasi berdasarkan berbagai skenario yang mereka butuhkan.

## Visual Cues for Fields

Ada beberapa konsep umum yang perlu dipahami dalam membangun sebuah visualisasi di Power BI.




**X-Axis**

X-Axis adalah sumbu horizontal dalam sebuah grafik. Biasanya digunakan untuk menampilkan data kategorikal atau berbasis waktu, di mana nilai atau kategori ditempatkan di sepanjang garis horizontal grafik. Misalnya, dalam diagram batang yang menunjukkan penjualan dari waktu ke waktu, bulan atau tahun akan diplot di sumbu X. Di Power BI, kita dapat menyesuaikan tampilan sumbu X, mengatur skala atau tipe sumbu (kontinu atau kategorikal), serta menerapkan format lainnya.

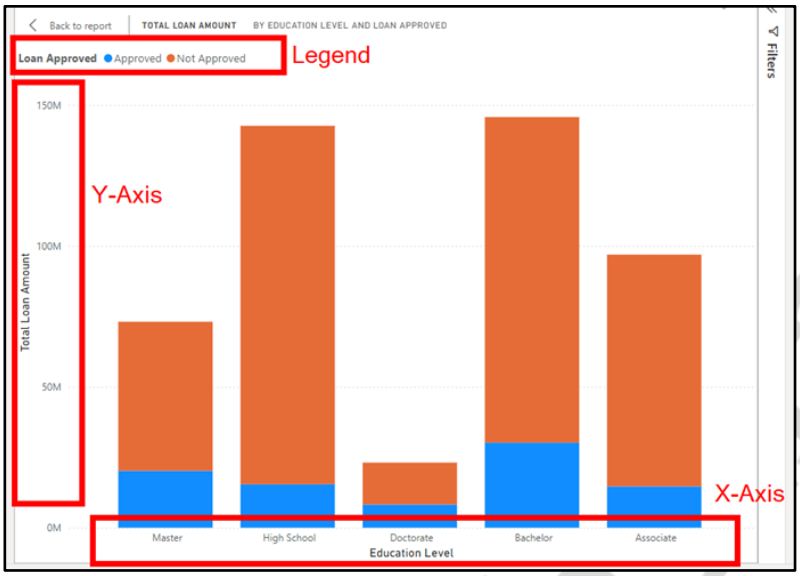
**Y-Axis**

Y-Axis adalah sumbu vertikal dalam sebuah grafik, yang biasanya digunakan untuk menampilkan nilai atau metrik numerik yang ingin diukur atau dibandingkan. Misalnya, dalam diagram batang, tinggi batang menunjukkan nilai pada sumbu Y, seperti pendapatan, penjualan, atau jumlah produk yang terjual. Sama seperti sumbu X, sumbu Y di Power BI juga dapat diformat dan disesuaikan, sehingga kita bisa mengatur skala sumbu, label, dan satuan yang ditampilkan.

**Legend**

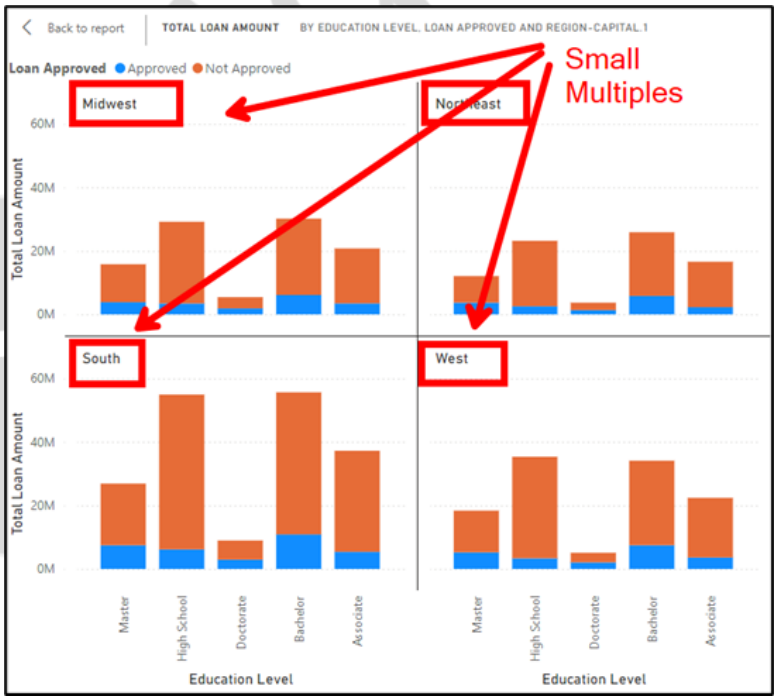
Legend adalah elemen visual yang menjelaskan apa yang diwakili oleh setiap warna atau simbol dalam grafik. Legend sangat membantu ketika kita memiliki beberapa seri atau kategori dalam satu grafik. Misalnya, dalam diagram batang, legenda akan menunjukkan kategori yang berbeda (seperti jenis produk atau wilayah) dalam warna yang berbeda, sehingga memudahkan pengguna untuk mengidentifikasi bagian visual yang sesuai dengan kategori tertentu. Di Power BI, kita dapat mengontrol posisi dan tampilan legenda untuk meningkatkan keterbacaan grafik.

algorima



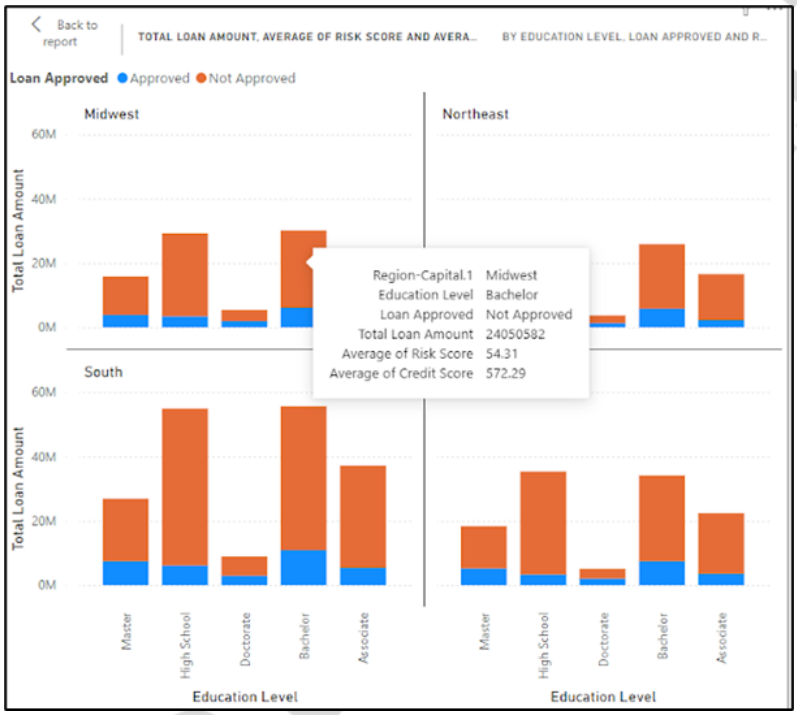

**Small Multiples**

Small Multiples memungkinkan kita membuat serangkaian grafik kecil yang identik, yang menampilkan data yang sama tetapi dibagi berdasarkan kategori. Alih-alih menampilkan semua kategori dalam satu grafik besar, Small Multiples menyajikan mereka dalam visual terpisah dan berulang di samping satu sama lain, memudahkan perbandingan antar kategori. Misalnya, jika kita menunjukkan data penjualan untuk beberapa wilayah, daripada menampilkan semua wilayah dalam satu grafik besar, small multiples memungkinkan kita membuat grafik individu untuk setiap wilayah, sehingga perbandingan menjadi lebih mudah.



**Tooltips**

Tooltips memberikan informasi detail tambahan saat kita mengarahkan kursor ke titik data dalam visual. Misalnya, jika kita mengarahkan kursor ke batang dalam diagram batang, sebuah tooltip akan muncul menampilkan nilai pasti yang diwakili oleh batang tersebut, beserta rincian lain seperti perubahan persentase, kategori, atau metrik lainnya. Power BI memungkinkan kustomisasi Tooltip di mana kita dapat menentukan Field dan data tambahan yang ingin ditampilkan sehingga memperkaya interaksi pengguna dengan data dan membantu interpretasi yang lebih mendalam.




## Introduction to Power Query Editor

Power Query Editor di Power BI Desktop adalah tools yang memungkinkan pengguna untuk membersihkan, mentransformasi, dan mempersiapkan data mereka sebelum digunakan di dalam Power BI Desktop. Power Query Editor memiliki tampilan yang intuitif dan mudah digunakan sehingga pengguna dapat melakukan berbagai tugas manipulasi data tanpa harus menulis kode yang rumit. Power Query Editor ini menjadi komponen penting di Power BI Desktop dalam proses penarikan data serta membantu mengubah data mentah menjadi format yang sesuai untuk pembuatan laporan dan dashboard.

Salah satu fitur utama dari Power Query Editor adalah kemampuan transformasi data dengan cara sekali klik. Pengguna dapat memfilter baris, mengganti nama kolom, menggabungkan atau memisahkan dataset, serta melakukan berbagai transformasi lainnya untuk menyesuaikan data sesuai dengan kebutuhan mereka. Fitur lainnya termasuk kemampuan untuk menerapkan pivot dan unpivot, mengganti nilai, dan mengelompokkan data.

Keunggulan dari penggunaan Power Query Editor adalah setiap perubahan yang kita lakukan akan tercatat pada bagian ***Applied Steps***. Fitur ini memungkinkan kita untuk memperbaiki langkah transformasi yang salah atau kembali ke beberapa langkah sebelumnya tanpa kehilangan satupun langkah.

Power Query Editor adalah bagian penting dari proses persiapan data di Power BI, memberikan fleksibilitas dan efisiensi dalam mentransformasi dan memperbaiki data. Dengan otomatisasi proses pembersihan dan transformasi data, pengguna dapat lebih fokus pada analisis dan pengambilan keputusan, memastikan bahwa data yang digunakan akurat, konsisten, dan siap untuk memberikan wawasan yang berarti.

## Data Understanding

Dataset yang akan kita gunakan di seluruh coursebook ini adalah dataset penjualan obat dari sebuah perusahaan farmasi di Indonesia. Dataset ini terbagi menjadi 4 tabel yaitu Prescript, Sales, Customer, dan Product. Dataset dapat diunduh di sini.

**Prescript**
Tabel Prescript berisi data resep yang dituliskan oleh dokter terhadap pasien. Satu baris data merepresentasikan satu resep dengan struktur kolom sebagai berikut:

| Nama Kolom | Keterangan | Tipe Data |
|---|---|---|
| NO_RESEP | Nomor resep dari dokter | Teks |
| TGL | Tanggal pembelian atau pengajuan resep | Tanggal |
| JAM_JUAL | Waktu pembelian atau pengajuan resep | Waktu |
| KD_CUST | Nomor registrasi pasien/pelanggan | Teks |
| KD_DOKTER | Nomor identitas dokter | Teks |
| RACIK | Racikan atau tidak racikan, digunakan untuk membedakan nilai tuslah | Teks |
| REG_AS | Jenis pasien; R = reguler, I = BPJS, A = Asuransi | Teks |

**Sales**
Tabel Sales berisi data obat-obat yang dituliskan di setiap resep dokter. Tabel ini merincikan tabel Prescript.

| Nama Kolom | Keterangan | Tipe Data |
|---|---|---|
| NO_RESEP | Nomor resep dokter | Teks |
| KD_OBAT | Kode obat | Teks |
| QTY | Kuantitas pembelian | Angka Integer |
| HNA | Harga netto apotek | Angka Desimal |
| HJ | Harga jual (belum termasuk tuslah dan PPN) | Angka Desimal |
| PPN_JUAL | Pajak pertambahan nilai yang dibebankan | Angka Integer |

**Product**
Tabel Product berisi detail data tentang obat yang dijual oleh perusahaan farmasi.

| Nama Kolom | Keterangan | Tipe Data |
|---|---|---|
| KD_OBAT | Kode obat | Teks |
| NAMA | Nama obat | Teks |
| SAT_JUAL | Satuan untuk penjualan | Teks |
| KD_PABRIK | Kode pabrikan obat | Teks |
| HJ_RP | Harga jual dari pabrikan | Angka Desimal |

**Customer**
Tabel Customer berisi detail informasi setiap pasien yang terdaftar.

| Nama Kolom | Keterangan | Tipe Data |
|---|---|---|
| KD_CUST | Nomor registrasi pasien/pelanggan | Teks |
| TGL_REGIS | Tanggal registrasi | Tanggal |
| GENDER | Jenis kelamin | Teks |
| NAMA_DPN | Nama depan | Teks |
| NAMA_TGH | Nama tengah inisial | Teks |
| NAMA_BLKG | Nama belakang | Teks |

| | | |
|---|---|---|
| KOTA | Kota asal | Teks |
| PROV | Provinsi asal | Teks |
| KD_POS | Kode pos | Angka Integer |
| TGL_LAHIR | Tanggal lahir | Tanggal |
| PEKERJAAN | Pekerjaan | Teks |
| PERUSAHAAN | Perusahaan | Teks |
| GEO_LAT | Geolokasi latitude | Angka Desimal |
| GEO_LON | Geolokasi longitude | Angka Desimal |

# Understanding DAX Fundamentals

## Defining DAX and Its Importance

DAX (*Data Analysis Expressions*) adalah bahasa yang digunakan dalam Microsoft Power BI, Power Pivot, dan Analysis Services. DAX memungkinkan pengguna untuk membuat perhitungan kompleks yang diperlukan untuk analisis data yang lebih mendalam.

DAX dikembangkan untuk mengatasi kebutuhan analitik yang tidak dapat dipenuhi oleh aplikasi *spreadsheet* biasa seperti Excel. Dengan menggunakan DAX, pengguna dapat membuat kolom atau tabel baru berdasarkan data yang sudah ada, membuat kalkulasi dinamis yang dapat disesuaikan dengan perubahan konteks data, hingga melakukan analisis data yang lebih cepat dan akurat.

DAX terdiri dari fungsi dan parameter yang memungkinkan kita untuk membuat rumus dan ekspresi. Berikut adalah beberapa komponen utamanya:

- **Fungsi DAX**
  Seperti di Excel, DAX memiliki berbagai fungsi yang dapat digunakan. Namun, DAX juga memiliki fungsi yang spesifik untuk model data kompleks seperti fungsi `CALCULATE()` dan `FILTER()`. Kita akan pelajari lebih dalam tentang ini nanti.
- **Parameter**
  Parameter dapat berupa nilai atau ekspresi yang kita masukkan ke dalam fungsi DAX. Parameter di dalam DAX ada yang bersifat wajib dan opsional. Parameter wajib dibutuhkan untuk menjalankan fungsi DAX. Sementara itu, parameter opsional digunakan untuk menambahkan informasi yang dibutuhkan ke dalam fungsi DAX.
- **Evaluation Context:** Salah satu konsep penting di DAX adalah bagaimana formula dievaluasi dalam konteks tertentu, baik itu *row context* atau *filter context*.

DAX memungkinkan pengguna Power BI untuk memaksimalkan potensi model data mereka. Dengan DAX, kita bisa membuat perhitungan yang menyesuaikan dengan konteks dataset yang berubah sehingga memberikan hasil dinamis untuk berbagai skenario. Ini sangat berguna saat membuat metrik bisnis seperti total penjualan per wilayah atau *month-to-month growth*.

Selain itu, formula DAX yang dirancang dengan baik dapat mengoptimalkan pemrosesan data dan meningkatkan kinerja laporan. Memahami DAX membantu kita membuat *measures* dan perhitungan yang efisien sehingga mengurangi waktu *loading* laporan Power BI kita.

## Exploring the Internal Working of DAX

Di balik kekuatan DAX, terdapat dua komponen utama yang bertanggung jawab untuk memproses semua perhitungan, yaitu Formula Engine (FE) dan Storage Engine (SE). Memahami bagaimana keduanya bekerja sangat penting untuk mengoptimalkan kinerja DAX terutama ketika berhadapan dengan skenario kompleks seperti Import dan DirectQuery dalam model data Power BI.

**Formula Engine (FE)**
Formula Engine adalah otak dari DAX yang bertanggung jawab untuk memproses perhitungan, menguraikan logika dan menerjemahkan formula DAX menjadi eksekusi perintah. Saat kita menulis DAX, Formula Engine akan menganalisis sintaks, memastikan validitas formula, dan menyusun rencana eksekusi.

Formula Engine tidak mengakses data secara langsung. FE hanya berfungsi untuk mengatur bagaimana perhitungan harus dijalankan. Saat formula membutuhkan akses data (misalnya ketika menghitung agregasi), FE akan meminta data dari Storage Engine (SE).

**Storage Engine (SE)**
Bertanggung jawab untuk mengambil data dari model, baik itu dari memori (untuk model Import) atau dari sumber data langsung (DirectQuery). SE bekerja di bawah perintah FE untuk mengambil data mentah yang kemudian diproses lebih lanjut.

Ketika kita menjalankan kueri DAX, prosesnya melibatkan langkah-langkah berikut:

1. **Kueri DAX dikirim ke FE**
   Formula Engine menganalisis rumus DAX, memecahnya menjadi instruksi logis, dan menentukan apakah kueri memerlukan data dari Storage Engine.
2. **FE Menginstruksikan SE**
   Jika data diperlukan, FE menginstruksikan Storage Engine untuk mengambil data sesuai dengan konteks filter dan parameter kueri.
3. **SE Menyediakan Data**
   Storage Engine mengakses data baik dari model Import maupun melalui DirectQuery, tergantung pada bagaimana model data dibangun. Hasil pengambilan data ini kemudian dikirim kembali ke Formula Engine.
4. **FE Memproses Hasil**

Setelah menerima data dari SE, FE melanjutkan proses perhitungan, agregasi, atau operasi lainnya sesuai dengan formula DAX. FE kemudian memberikan hasil akhir kepada pengguna.

Power BI mendukung dua mode utama untuk memproses data: Import dan DirectQuery. Kedua mode ini mempengaruhi cara kerja Storage Engine dan performa keseluruhan kueri.

**Import Mode**
Data diambil dari sumber, kemudian dimuat ke dalam memori menggunakan VertiPaq Engine. VertiPaq adalah teknologi penyimpanan data yang menggunakan kompresi tinggi untuk meningkatkan efisiensi. Dalam mode ini, data tersedia di memori dan bisa diakses dengan sangat cepat.

Kelebihan Import Mode:
- Performa kueri yang lebih cepat karena data tersedia di memori.
- Tidak memerlukan koneksi internet ke sumber data selama analisis berlangsung.

Kelemahan:
- Data tidak selalu terbaru karena bergantung pada terakhir kali di-*refresh* sehingga kurang cocok untuk kebutuhan analisa *real-time*.

**DirectQuery Mode**
Dalam mode ini, data tidak disimpan di dalam memori. Sebaliknya, setiap kali ada perhitungan, Storage Engine mengirimkan query langsung ke database sumbernya. DirectQuery digunakan saat dataset terlalu besar untuk dimuat ke dalam memori atau ketika data *real-time* diperlukan. Namun, mode ini cenderung lebih lambat dibandingkan Import Mode karena harus mengakses data eksternal setiap kali ada permintaan.

Kelebihan DirectQuery Mode:
- Cocok untuk data yang sangat besar atau real-time yang sulit diimpor ke dalam memori.
- Tidak perlu menyinkronkan data karena selalu terhubung ke sumber asli.

Kelemahan:
- Performa lebih lambat dibandingkan dengan Import karena setiap kueri harus mengakses data dari sumber aslinya.

**VertiPaq Engine**
VertiPaq Engine adalah bagian dari Storage Engine yang memproses data dalam Import Mode. VertiPaq menggunakan teknik kompresi data yang efisien sehingga memungkinkan data dalam jumlah besar untuk dimuat ke dalam memori Power BI dengan kinerja yang sangat cepat. Teknik kompresi ini adalah alasan mengapa model data Power BI bisa begitu besar tetapi tetap bisa diproses dengan cepat. VertiPaq menyimpan data dalam format *columnar*, yang memungkinkan kompresi yang lebih tinggi dan pengambilan data yang lebih efisien.

Pemahaman mendalam tentang Formula Engine dan Storage Engine membantu kita merancang dan menjalankan kueri DAX yang lebih efisien. Perbedaan antara Import Mode dan DirectQuery juga memberikan fleksibilitas dalam cara kita menangani data, baik untuk kecepatan (melalui VertiPaq) atau untuk menangani data dalam jumlah besar yang tidak dapat diimpor ke dalam memori. Dengan pemahaman ini kita bisa lebih efektif dalam memaksimalkan kemampuan DAX di Power BI.

## Comparing DAX with MDX and SQL

DAX, MDX (Multidimensional Expressions), dan SQL (Structured Query Language) adalah bahasa yang digunakan untuk bekerja dengan data, namun mereka berbeda dalam tujuan, struktur, dan cara penggunaannya. Bagi kita yang sudah terbiasa bekerja dengan data multidimensi tentunya sudah tidak asing dengan MDX. Begitupun kita yang sudah terbiasa bekerja dengan database tentunya sudah tidak asing dengan SQL.

DAX (Data Analysis Expressions) adalah bahasa yang dirancang khusus untuk model data tabular (*columnar*) di Power BI, Power Pivot, dan Analysis Services Tabular. DAX fokus pada perhitungan dan manipulasi data yang sudah terstruktur di dalam model tabular.

**MDX (Multidimensional Expressions)** adalah bahasa kueri yang digunakan di SQL Server Analysis Services (SSAS) untuk bekerja dengan model data multidimensi (OLAP cubes). MDX biasanya digunakan dalam lingkungan analisis yang melibatkan dimensi dan hierarki yang lebih rumit.

MDX bekerja dengan model multidimensi, sedangkan DAX dirancang untuk model tabular (kolomar). MDX mendukung hierarki dimensi yang lebih dalam dan lebih kompleks, membuatnya ideal untuk model data yang sangat besar dan terstruktur. Selain itu, MDX lebih sering digunakan di lingkungan OLAP dan bekerja dengan *cube*, sedangkan DAX lebih fleksibel untuk penggunaan dalam Power BI dan Excel.

**SQL (Structured Query Language)** adalah bahasa standar untuk mengelola dan memanipulasi database. SQL digunakan untuk mengambil, memasukkan, memperbarui, dan menghapus data dari tabel database. SQL lebih umum dan bekerja langsung dengan database relasional seperti SQL Server, MySQL, dan lainnya.

SQL berfokus pada pengambilan dan manipulasi data dalam database relasional, sementara DAX lebih berfokus pada analisis data dan menghasilkan perhitungan dinamis dalam model data tabular. SQL melakukan operasi langsung pada data di database sementara DAX bekerja pada model data yang diimpor ke dalam memori. DAX mampu melakukan agregasi dinamis berdasarkan konteks visualisasi di Power BI sehingga memungkinkan analisis yang lebih interaktif dan mendalam dibandingkan SQL yang cenderung bekerja pada query statis.

DAX adalah bahasa *functional* dimana setiap ekspresi dinyatakan dalam fungsi atau formula. Sementara itu, SQL adalah bahasa *declarative* dimana kita mendeklarasikan setiap data yang ingin ditarik melalui sebuah *statement* SELECT.

Sebagai contoh, jika kita ingin melihat nama-nama pelanggan yang tinggal di Eropa maka kita perlu menuliskan kueri berikut di SQL:

```
SELECT
   Customers.CustomerName,
   SUM(Sales.SalesAmount) AS TotalSales
FROM
   Sales
   INNER JOIN Customers
   ON Sales.CustomerKey = Customers.CustomerKey
WHERE
   Customers.Continent = 'Europe'
GROUP BY
   Customers.CustomerName
```

Sementara itu, dengan DAX kita menggunakan fungsi yang disematkan ke dalam fungsi untuk melakukan filter terhadap kueri data:

```
EVALUATE
SUMMARIZECOLUMNS (
   Customers[CustomerName],
   FILTER (
      Customers,
      Customers[Continent] = "Europe"
   ),
   "Total Sales", SUM ( Sales[SalesAmount] )
)
```

Sebuah fungsi `FILTER()` digunakan di dalam fungsi `SUMMARIZECOLUMNS()` untuk memfilter hasil perhitungan hanya pada kondisi yang kita inginkan.

Baik itu DAX, MDX, ataupun SQL masing-masing memiliki kemampuan dan tujuan yang berbeda-beda. Mempelajari DAX adalah suatu hal yang baru meskipun kita sudah terbiasa dengan MDX ataupun SQL. DAX adalah bahasa yang sederhana tetapi akan sulit untuk dipahami semakin dalam kita menggunakannya.

# Creating Measures vs. Calculated Columns

Dalam Power BI yang menggunakan model data tabular (*columnar*), kita sering dihadapkan pada pilihan antara membuat Measures atau Calculated Columns untuk menghitung nilai tertentu. Keduanya memiliki peran dan kegunaan yang berbeda dan memilih dengan tepat antara keduanya dapat memengaruhi performa model data dan hasil analisis.

**Calculated Columns**

Merupakan kolom yang ditambahkan ke dalam tabel pada model data. Kolom ini menghitung nilai secara *row-by-row* (berdasarkan setiap baris) menggunakan formula DAX dan hasilnya disimpan dalam model data. *Calculated columns* dihitung selama proses *refresh* data, dan nilainya tidak berubah kecuali data di-*refresh* kembali.

*Calculated columns* digunakan saat kita perlu menambahkan kolom tambahan ke dalam tabel yang menghitung nilai untuk setiap baris data. Ini cocok ketika hasil perhitungan perlu disimpan dalam tabel dan digunakan sebagai elemen data di visualisasi seperti bagian dari hierarki atau filter.

Misal, menghitung total penjualan:

```
Total Sales = Sales[HJ] * Sales[QTY]
```

Kapan kita perlu menggunakan *Calculated columns*:

- Saat kita membutuhkan hasil perhitungan yang digunakan sebagai kategori atau filter di dalam laporan.
- Saat kita ingin hasil perhitungan tersebut selalu tersedia untuk setiap baris data, tanpa perlu melakukan agregasi.
- Saat kita perlu menghitung nilai *row-by-row*, misalnya menghitung profit margin untuk setiap transaksi.

Namun, karena calculated columns dihitung selama *refresh* data, mereka dapat memengaruhi ukuran model data jika digunakan secara berlebihan.

**Measures**

*Measures* Adalah formula dinamis yang dihitung saat *runtime* (ketika data divisualisasikan). *Measures* digunakan untuk menghitung agregat seperti total, rata-rata, atau perhitungan lainnya yang dilakukan dalam konteks tertentu yang biasanya berdasarkan filter atau slicer dalam laporan.

*Measures* digunakan untuk perhitungan yang sifatnya dinamis dan kontekstual di mana hasil perhitungan bergantung pada filter atau konteks agregasi yang ada di visualisasi. *Measures* tidak menyimpan nilai di tabel melainkan dihitung saat laporan atau dashboard dijalankan. *Measures* sangat efisien dalam melakukan perhitungan agregat pada data besar karena hanya dieksekusi pada waktu kueri.

Misal, menghitung rasio antara harga jual dengan harga modal:

```
Profit Ratio = SUM(Sales[HJ]) / SUM(Sales[HNA])
```

Kapan kita perlu menggunakan *measures*:

- Saat kita perlu menghitung agregat seperti total, rata-rata, maksimum, atau perhitungan lain yang bergantung pada visualisasi.
- Saat perhitungan perlu diubah sesuai dengan konteks filter seperti tahun, wilayah, atau produk.
- Saat kita ingin mempertahankan model data tetap efisien dan tidak memperbesar ukuran data dengan menambahkan kolom yang tidak perlu.

Karena *measures* tidak menyimpan data di tabel, mereka lebih ringan dibandingkan dengan *calculated columns* dalam hal penggunaan memori dan performa.

| *Measures* | *Calculated Columns* |
|---|---|
| Dihitung secara dinamis pada waktu kueri (*runtime*) | Dihitung selama refresh data dan disimpan di model data |
| Digunakan untuk agregasi dan perhitungan yang tergantung pada konteks filter | Digunakan untuk menghitung nilai per baris yang dapat digunakan sebagai kategori atau filter |
| Lebih efisien dalam hal performa dan penggunaan memori | Menambah ukuran model data karena hasil perhitungan disimpan |

Memilih antara *measure* dan *calculated column* bergantung pada tujuan spesifik perhitungan dan bagaimana data akan digunakan dalam visualisasi. Beberapa pertimbangan tersebut diantaranya:

- Apakah perhitungan perlu dinamis? Jika ya, maka gunakan *Measures*. Misalnya, jika kita ingin menghitung total penjualan yang berubah berdasarkan filter maka *Measure* lebih cocok.
- Apakah kita perlu menggunakan hasil perhitungan sebagai filter atau kategori? Jika ya, maka *Calculated Columns* lebih tepat. Misalnya, jika kita ingin membuat kolom yang menghitung status pelanggan (aktif atau tidak aktif) dan menggunakannya sebagai filter dalam laporan.
- Performa: *Measures* lebih efisien dalam hal performa dan penggunaan memori karena mereka hanya dihitung saat dibutuhkan. Sebaliknya, *Calculated Columns* memakan lebih banyak ruang dalam model data.

# In-depth Look at Evaluation Context

Dalam DAX, *evaluation contexts* memainkan peran kunci dalam menentukan bagaimana perhitungan dilakukan. *Evaluation contexts* menentukan cara DAX mengevaluasi nilai di dalam model data, dan terdapat dua jenis utama dari *evaluation contexts*: *Row Context* dan *Filter Context*. Memahami konsep ini penting untuk membuat perhitungan dinamis dan mendapatkan hasil yang diinginkan dalam Power BI.

***Row Context***

*Row context* terjadi ketika DAX mengevaluasi perhitungan berdasarkan baris demi baris dalam tabel. *Row context* paling mudah dipahami ketika kita membuat *calculated columns*. Membuat *calculated column* atau membuat kolom baru pada model data akan membuat DAX mengevaluasi formula atau ekspresi yang dimasukkan secara baris demi baris..

Contoh: Dalam contoh *calculated column* berikut, kita menggunakan *row context* untuk menghitung harga jual setelah PPN untuk setiap transaksi:

```
Discount Amount = Sales[HJ] * Sales[PPN_JUAL]
```

Pada setiap baris, DAX akan mengevaluasi nilai Amount dan Discount berdasarkan nilai di baris tersebut.

***Filter Context***

*Filter context* mengacu pada set filter yang aktif pada waktu ketika perhitungan dilakukan. *Filter context* terdiri dari semua filter yang diterapkan melalui slicers, visual, atau fungsi DAX seperti `CALCULATE()`. Filter context memungkinkan DAX untuk menghitung nilai yang dipengaruhi oleh filter yang berlaku pada tabel atau kolom tertentu.

Contoh: Misal, kita memiliki visualisasi tabel yang terdiri dari nama-nama produk maka ketika kita menggunakan rumus DAX untuk menghitung total penjualan seperti berikut:

```
Total Sales = SUMX(Sales, Sales[HJ] * Sales[QTY])
```

DAX akan mengevaluasi fungsi `SUM()` mengikuti setiap produk yang ada di dalam visualisasi tabel. Artinya, pada baris pertama DAX hanya menghitung total penjualan untuk produk Acid Salicyl seperti berikut:

| Row Labels | Total Amount |
|---|---|
| Acid Salicyl | 11,427,446,963 |
| LCD | 4,468,864,303 |
| Lanolin 100GR | 3,250,158,334 |
| Herceptin 440 mg/20 | 1,740,193,497 |
| Brexel 80 mg/vial | 695,495,765 |
| Taxotere 80 mg | 336,258,282 |
| lanolin 150gr | 319,600,000 |
| Symbicort 160/4.5mcg | 192,637,827 |

Dalam kebanyakan skenario, kita bekerja dengan *filter context* saat membuat *measures*, sedangkan *row context* lebih sering diterapkan pada saat *calculated columns*.

***Context Transition***

*Context Transition* adalah salah satu konsep penting dalam DAX yang terjadi saat *row context* berubah menjadi *filter context*. Ini sering terjadi ketika kita menggunakan fungsi seperti `CALCULATE()` di dalam sebuah formula DAX yang sedang mengevaluasi perhitungan dalam *row context*.

Contoh:

```
Total Sales by Row = CALCULATE(SUMX(Sales, Sales[HJ] * Sales[QTY]))
```

Pada contoh di atas, kita membuat *calculated column* dengan menggunakan fungsi `SUMX()` dan `CALCULATE()`. *Calculated column* tidak mengenal *filter context* sehingga apabila kita hanya menggunakan fungsi `SUM` maka nilai yang dihasilkan setiap baris adalah sama yaitu total dari satu kolom tersebut.

| QTY | HJ | Total Sales Amount (Only SUMX) |
|---|---|---|
| 1000 | 0,007 | 28240546065,3516 |
| 800 | 0,00... | 28240546065,3516 |
| 500 | 0,012 | 28240546065,3516 |
| 390 | 0,01... | 28240546065,3516 |
| 450 | 0,01... | 28240546065,3516 |
| 400 | 0,0175 | 28240546065,3516 |
| 400 | 0,0175 | 28240546065,3516 |
| 630 | 0,02... | 28240546065,3516 |
| 4400 | 0,0315 | 28240546065,3516 |
| 400 | 0,0396 | 28240546065,3516 |
| 161 | 0,04... | 28240546065,3516 |
| 161 | 0,04... | 28240546065,3516 |
| 2000 | 0,04... | 28240546065,3516 |
| 5415 | 0,04... | 28240546065,3516 |
| 2590 | 0,05... | 28240546065,3516 |

Namun, berbeda halnya ketika kita menggabungkan fungsi `CALCULATE()` ke dalamnya maka hasil evaluasi `SUMX()` pada *calculated column* akan menghitung total untuk baris yang bersesuaian seperti pada gambar berikut:

| QTY | HJ | Total Sales Amount (CALCULATE) |
|---|---|---|
| 1000 | 0,007 | 7 |
| 800 | 0,00... | 7 |
| 500 | 0,012 | 6 |
| 390 | 0,01... | 6 |
| 450 | 0,01... | 7 |
| 400 | 0,0175 | 7 |
| 400 | 0,0175 | 7 |
| 630 | 0,02... | 13,2 |
| 4400 | 0,0315 | 138,6 |
| 400 | 0,0396 | 15,84 |
| 161 | 0,04... | 7 |
| 161 | 0,04... | 7 |
| 2000 | 0,04... | 89,76 |
| 5415 | 0,04... | 260,4 |
| 2590 | 0,05... | 130,8 |

Menggunakan `CALCULATE()` akan memicu perubahan ke *filter context* sehingga *calculated column* yang pada dasarnya tidak mengenal *filter context* menjadi memahaminya. DAX akan mengubah *row context* saat ini menjadi *filter context* dan menghitung nilai agregat berdasarkan filter (baris yang sedang aktif) tersebut.

Proses evaluasi dalam DAX dimulai dari *row context* (jika ada) kemudian berkembang ke *filter context* berdasarkan kueri atau formula yang dijalankan. Beberapa langkah penting dalam konsep *evaluation context*:

- *Row context* dihasilkan ketika DAX mengevaluasi setiap baris dalam tabel.
- Ketika kita menggunakan fungsi seperti `CALCULATE()`, DAX mengubah *row context* menjadi *filter context* dan menerapkan filter tambahan pada data.
- DAX kemudian menghitung nilai akhir berdasarkan *filter context* yang aktif.

*Filter context* juga bisa disesuaikan secara manual dengan menambahkan atau menghapus filter menggunakan fungsi seperti `REMOVEFILTERS()`, `ALLEXCEPT()`, atau `KEEPFILTERS()` di dalam `CALCULATE()` yang akan kita pelajari nanti.

## DAX Convention

Berikut adalah beberapa hal yang bisa dijadikan panduan dalam menulis DAX agar formula yang ditulis konsisten, mudah dibaca dan dievaluasi oleh kita:

- Gunakan `[measure]` dan jangan gunakan `table[measure]`
- Gunakan `table[column]` dan jangan gunakan `[column]`
- Definisi (nama *calculated column* atau *measure*) diberikan di baris pertama beserta tanda *assignment* '='

- Formula DAX dimulai pada baris setelahnya
- Pisahkan baris parameter pada fungsi yang menggunakan lebih dari satu parameter

```
Sales Regional 3 = CALCULATE([Total Sales], Customer[Region] = 3)
```

DAX di atas dapat dibuat menjadi lebih rapi dan mudah dibaca dengan mengikuti DAX Convention menjadi:

```
Sales Regional 3 =
CALCULATE(
    [Total Sales],
    Customer[Region] = 3
)
```

## DAX Operator

DAX memiliki beberapa operator dasar layaknya kita bekerja menggunakan formula Excel. Ada 3 operator utama yaitu Arithmetic, Comparison, dan Text & Logical.

**Arithmetic Operators**

| Operator | Arti |
|---|---|
| + | Tambah |
| - | Kurang |
| * | Kali |
| / | Bagi |
| ^ | Pangkat |

Operator tambah (+) dan kurang (-) dapat digunakan sebagai binary operator ataupun unary operator. Binary operator akan menerapkan operasi sesuai operatornya. Contoh:

```
3 + 3
```

Akan menghasilkan 6. Sementara itu, unary operator akan memberikan tanda ke bilangan yang diberikan. Contoh:

```
- 3
```

Akan menghasilkan bilangan negatif 3. Apabila teks diterapkan pada binary operator maka akan otomatis diubah menjadi angka. Contoh:

```
“3” + “3”
```

Akan sama saja dengan:

```
3 + 3
```

**Comparison Operators**

| Operator | Arti |
|---|---|
| = | Sama dengan |
| == | Benar-benar sama dengan |
| <> | Tidak sama dengan |
| > | Lebih besar dari |
| >= | Lebih besar dari atau sama dengan |
| < | Lebih kecil dari |
| <= | Lebih kecil dari atau sama dengan |

Operator “sama dengan” (=) akan menganggap nilai BLANK sama dengan nilai 0, teks kosong “”, Tanggal 30-12-1899, atau FALSE. Namun, operator “benar-benar sama dengan” (==) tetap akan menganggap nilai BLANK sebagai BLANK.

**Text & Logical Operators**

| Operator | Arti | Contoh |
|---|---|---|
| & | Concatenate Teks | [Region] & “ “ & [Province] |
| && | AND | [Gender]=”F” && [Gender]=”M” |
| \|\| | OR | [Gender]=”F” || [Gender]=”M” |
| IN | OR di dalam List | [Gender] IN {“F”, “M”, “Not Specified”} |

# Introduction to Index DAX Functions

## The Importance of Date Table in Multi Table Analysis

**Date Table** atau Tabel Tanggal adalah tabel khusus dalam model data yang berisi deretan tanggal yang mencakup periode waktu tertentu (misalnya, tahun, bulan, hari, dan lain-lain) yang digunakan untuk analisis berbasis waktu.

Date Table tidak hanya menyimpan tanggal, tetapi juga berbagai atribut terkait tanggal seperti tahun, kuartal, bulan, minggu, atau bahkan hari dalam seminggu. Dalam DAX, Tabel Tanggal sangat penting karena banyak fungsi DAX seperti fungsi Time Intelligence (`TOTALYTD()`, `DATEADD()`, atau `SAMEPERIODLASTYEAR()` membutuhkan Date Table yang valid untuk beroperasi dengan benar.

**Mengapa kita membutuhkan Date Table dalam model data?**
Banyak analisis bisnis yang melibatkan perbandingan waktu seperti menghitung total penjualan berdasarkan tahun, mengukur pertumbuhan dari bulan ke bulan, atau membandingkan penjualan tahunan. Fungsi-fungsi Time Intelligence seperti

- `TOTALYTD()`: Menghitung total kumulatif hingga tahun berjalan.
- `DATEADD()`: Memindahkan waktu maju atau mundur.
- `SAMEPERIODLASTYEAR()`: Membandingkan nilai dari periode yang sama dari tahun sebelumnya.

Fungsi-fungsi ini memerlukan Date Table agar dapat berfungsi dengan baik, karena mereka bekerja dengan tanggal untuk menghitung selisih waktu, tren, atau agregasi berdasarkan periode tertentu.

Selain itu, kegunaan dari Date Table adalah kita bisa mendapatkan wawasan mendalam terkait data kita yang dibagi berdasarkan tahun, bulan, atau kuartal. Misalnya, kita bisa mengetahui kinerja bisnis dari tahun ke tahun (YoY), kuartal ke kuartal (QoQ), atau bulan ke bulan (MoM). Tabel ini memungkinkan kita membuat visualisasi seperti tren penjualan bulanan atau perbandingan performa kuartal antar tahun.

Date Table juga berperan sebagai dimensi utama dalam model data *Star Schema* ataupun *Snowflake Schema* yang digunakan di Power BI. Setiap tabel faktual (misalnya, tabel penjualan) biasanya bisa memiliki lebih dari satu kolom tanggal seperti kolom tanggal transaksi, kolom tanggal invoice, hingga kolom tanggal pengiriman. Kita memerlukan tabel tanggal untuk menciptakan relasi antara tabel penjualan dengan kalender agar bisa menganalisis penjualan tersebut berdasarkan tanggal kasus yang kita butuhkan. Tanpa relasi ini, analisis waktu tidak akan bisa dilakukan dengan benar.

Perlu diketahui juga bahwa banyak perusahaan menggunakan kalender fiskal atau periode kustom yang berbeda dari kalender biasa (misalnya, tahun fiskal tidak dimulai dari Januari melainkan dari April). Dengan Date Table, kita bisa menambahkan kolom-kolom khusus yang merepresentasikan periode waktu yang spesifik bagi perusahaan tersebut seperti Tahun fiskal, Kuartal kustom, dan Bulan kustom. Hal Ini memungkinkan penghitungan dinamis seperti total penjualan dalam tahun fiskal tertentu atau perbandingan performa berdasarkan kuartal kustom.

Selain itu, kita berisiko melakukan analisis waktu yang tidak konsisten terutama jika ada gap dalam data tanggal atau jika kita bekerja dengan data dari berbagai sumber yang mungkin menggunakan format tanggal yang berbeda. Dengan menggunakan Date Table kita bisa memastikan bahwa analisis berbasis waktu selalu konsisten di seluruh model data.

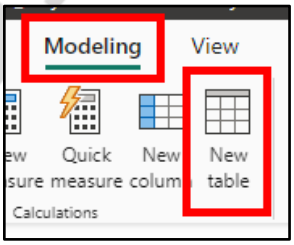
**Apa yang biasanya ada di Date Table?**
Tabel tanggal yang ideal biasanya memiliki kolom-kolom berikut:

- Date: Kolom yang berisi semua tanggal unik tanpa duplikasi.
- Tahun: Berisi tahun untuk setiap tanggal.
- Bulan: Nama atau angka bulan.
- Hari dalam Minggu: Representasi numerik (misalnya, 1 = Senin) atau nama hari (misalnya, Senin).
- Kuartal: Angka kuartal (Q1, Q2, dll.).
- Flag untuk Weekend: Misalnya, kolom yang menunjukkan apakah hari tersebut adalah akhir pekan atau tidak.

Oleh karena itu, kita akan membuat Date Table untuk model data yang akan kita gunakan di Power BI. Date Table bisa dibuat secara manual di Power BI dengan DAX menggunakan fungsi seperti `CALENDAR()` atau `CALENDARAUTO()`.

Penting untuk dipahami bahwa fungsi `CALENDARAUTO()` akan membuat Date Tabel berisi seluruh tanggal yang merentang dari tanggal terawal hingga tanggal terakhir dari seluruh kolom bertipe tanggal yang ada di model data. Terkadang cara ini kurang efektif pada beberapa keadaan seperti adanya kolom “TGL_LAHIR” pada tabel Customer. Tahun lahir pasien yang sangat bisa dimulai dari 1900-an akan membuat ukuran Date Table semakin besar sedangkan tidak digunakan secara maksimal.

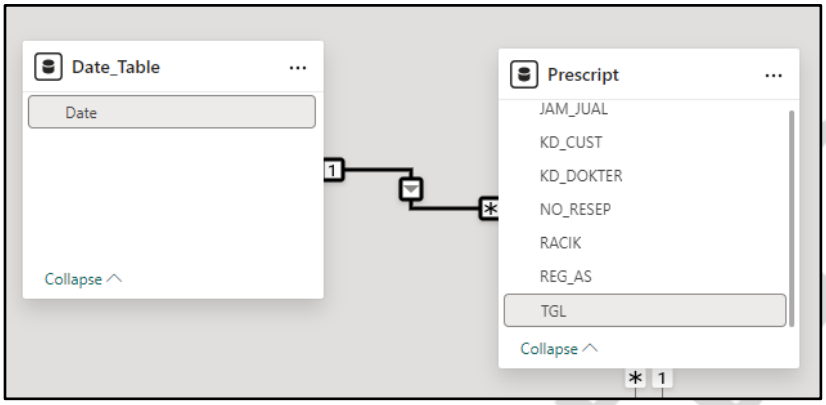
Pada kondisi seperti ini, kita perlu menggunakan fungsi `CALENDAR()` dimana kita harus memasukkan secara manual rentang minimum dan maksimum dari tanggal yang ada di Date Table. Kita akan buat tabel baru pada model data dengan cara pergi ke Ribbon **Modeling** lalu pilih menu **New Table**




Kita akan gunakan sintaks berikut untuk membuat Date Table:

```
Date_Table =
CALENDAR(
    MIN(Prescript[TGL]),
    MAX(Prescript[TGL])
)
```

Sebuah tabel baru akan terbentuk dengan satu kolom bernama “Date”. Date Table akan terhubung dengan tabel Prescript melalui kolom “TGL”. Oleh karena itu, kita perlu sesuaikan relasinya pada Model View menjadi seperti berikut:




Selanjutnya, kita bisa menambahkan kolom Tahun, Kuartal, dan Bulan untuk mempermudah kita menganalisis data berdasarkan tanggal dalam tingkat granularitas tertentu. Untuk membuat kolom tersebut kita bisa menggunakan *calculated column* pada tabel Date_Table dengan menggunakan DAX seperti berikut:

```
Tahun = YEAR(Date_Table[Date])

Bulan = UPPER(FORMAT(Date_Table[Date], "MMM"))

Kuartal = "Q" & QUARTER(Date_Table[Date])
```

Selain itu, kita perlu juga untuk menambahkan kolom bulan dalam bentuk angka untuk menjadikannya sebagai dasar pengurutan kolom bulan yang dalam bentuk nama. Hal ini perlu untuk dilakukan karena nantinya Power BI akan mengurutkan kolom nama-nama bulan berdasarkan abjadnya sehingga kita perlu mendefinisikan urutan angkanya sebagai kolom baru untuk acuan pengurutan nama-nama bulan berdasarkan urutan angkanya.

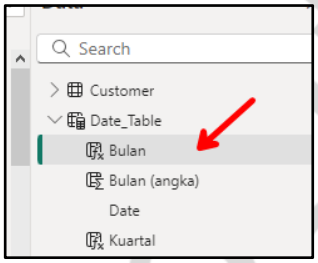
```
Bulan (angka) = MONTH(Date_Table[Date])
```

Sehingga hasil akhir dari tabel Date_Table adalah sebagai berikut:

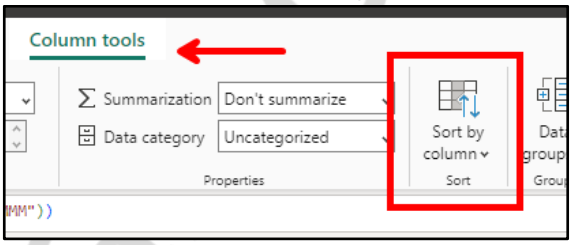
| Date | Tahun | Bulan | Bulan (angka) | Kuartal |
|---|---|---|---|---|
| 01/01/2015 | 2015 | JAN | 1 | Q1 |
| 01/02/2015 | 2015 | JAN | 1 | Q1 |
| 01/03/2015 | 2015 | JAN | 1 | Q1 |
| 01/04/2015 | 2015 | JAN | 1 | Q1 |
| 01/05/2015 | 2015 | JAN | 1 | Q1 |
| 01/06/2015 | 2015 | JAN | 1 | Q1 |
| 01/07/2015 | 2015 | JAN | 1 | Q1 |
| 01/08/2015 | 2015 | JAN | 1 | Q1 |
| 01/09/2015 | 2015 | JAN | 1 | Q1 |
| 01/10/2015 | 2015 | JAN | 1 | Q1 |

Selanjutnya, kita perlu mendefinisikan bahwa kolom “Bulan” diurutkan berdasarkan kolom “Bulan (angka)” agar setiap kita membuat visualisasi, nama-nama bulan otomatis berurutan dari bulan Januari hingga Desember. Untuk mengurutkannya kita bisa lakukan:

1. Pilih kolom “Bulan”




2. Pada Ribbon ***Column tools*** pilih menu ***Sort by column***




3. Lalu pilih kolom “Bulan (angka)”

Date Table sangat penting dalam model data Power BI untuk memungkinkan kita menggunakan fungsi Time Intelligence dengan efektif, melakukan analisis berbasis waktu, dan mengelola data tanggal secara konsisten. Analisis yang berhubungan dengan waktu akan terbatas atau tidak akurat tanpa adanya Date Table. Oleh karena itu, memastikan bahwa model data kita memiliki Date Table adalah langkah penting untuk meningkatkan kualitas dan kemampuan analisis data.

## Exploring Index DAX Functions

Dalam DAX (Data Analysis Expressions), fungsi Index adalah fungsi yang memungkinkan kita untuk mengambil atau mengakses nilai tertentu dari data yang tersusun secara urut. Fungsi-fungsi ini berguna ketika kita perlu mengakses data spesifik berdasarkan posisi atau urutan di dalam set data seperti menemukan baris pertama, kedua, atau terakhir dalam tabel.

Beberapa fungsi Index utama yang digunakan di DAX antara lain `FIRSTNONBLANK()`, `LASTNONBLANK()`, `FIRSTNONBLANKVALUE()`, `LASTNONBLANKVALUE()`, `INDEX()`. Meskipun istilah "index" mengacu pada posisi atau urutan data tetapi DAX tidak secara langsung menggunakan konsep nomor indeks dalam tabel seperti pada bahasa pemrograman lain. Sebaliknya, DAX mengelola data berdasarkan urutan atau kondisi yang telah difilter.

Dua fungsi dasar untuk bekerja dengan indeks di DAX adalah `FIRSTNONBLANK()` dan `LASTNONBLANK`. Fungsi-fungsi ini memungkinkan kita untuk mengakses nilai pertama atau terakhir dari sebuah kolom, dengan syarat nilai tersebut bukan `BLANK`.

**`FIRSTNONBLANK(column, expression)`**
Mengembalikan nilai pertama dalam sebuah kolom yang bukan kosong, berdasarkan *filter context* yang ada.

**`LASTNONBLANK(column, expression)`**
Mengembalikan nilai terakhir dalam sebuah kolom yang bukan kosong, berdasarkan *filter context* yang ada.

**`FIRSTNONBLANKVALUE(column, expression)`**
Mengembalikan nilai ekspresi pertama dalam sebuah kolom yang bukan kosong, berdasarkan *filter context* yang ada.

**`LASTNONBLANKVALUE(column, expression)`**
Mengembalikan nilai ekspresi pertama dalam sebuah kolom yang bukan kosong, berdasarkan filter context yang ada.

Untuk lebih memahami perbedaan keempat fungsi tersebut mari perhatikan contoh berikut:

| NAMA | JAN | FEB | MAR | APR |
|---|---|---|---|---|
| ZYPRAZ 0.5 MG TAB | | | | |
| ZYLORIC TAB 300 MG | | 5,368.44 | 5,598.00 | |
| Zyloric 100 tab | | | | 2,651.00 |
| Zometa inj | 4,221,360.00 | | 1,080,000.24 | |
| Zometa 4 mg/5 ml | 3,187,500.25 | | 3,187,500.25 | 3,187,500.25 |
| Zolmia | | 17,325.00 | 6,188.00 | 6,187.50 |
| ZISTIC 500 | | 32,656.25 | | 32,656.00 |
| ZINKID SYR | | 27,060.00 | 34,375.00 | 22,550.00 |
| Zinkid 10 mg/5 ml | 23,406.63 | 46,814.36 | 24,344.38 | 20,427.60 |
| ZINKID | | 3,437.50 | | |

Misal, kita memiliki tabel seperti di atas yang terdiri dari kolom NAMA, JAN, FEB, MAR, dan APR. Ketika kita masukkan DAX berikut:

```
FIRSTNONBLANK(NAMA, JAN)
```

Maka DAX akan mencari nilai dari kolom NAMA dengan syarat kolom JAN tidak kosong dan ditemukan pertama kali dari urutan baris sehingga hasilnya adalah `Zometa Inj`. Namun, apabila kita menggunakan DAX berikut:

```
FIRSTNONBLANKVALUE(NAMA, JAN)
```

Maka DAX akan mencari nilai dari kolom JAN yang pertama kali tidak kosong ketika dievaluasi berdasarkan kolom NAMA sehingga hasilnya adalah 4,221,360.00. Proses evaluasi serupa juga terjadi untuk fungsi `LASTNONBLANK()` dan `LASTNONBLANKVALUE()` tetapi urutannya dari yang paling terakhir.

## Practical Applications of Index DAX

Dalam aplikasi bisnis, seringkali kita perlu mengetahui kapan transaksi pertama atau terakhir terjadi untuk menganalisis tren penjualan atau kebiasaan pelanggan. Menggunakan `FIRSTNONBLANK()` dan `LASTNONBLANK()`, kita bisa dengan mudah menentukan tanggal transaksi pertama dan terakhir untuk sebuah produk atau pelanggan.

Misal, kita ingin menemukan tanggal transaksi pertama untuk setiap produk dalam tabel penjualan. Kita bisa membuat sebuah *measure* dan menggunakan DAX seperti berikut:

```
Transaksi Pertama =
FIRSTNONBLANK(
    Date_Table[Date],
    [Total Sales]
)
```

DAX tersebut akan mencari tanggal terjadinya pertama kali transaksi berdasarkan `[Total Sales]` yang tidak BLANK. Evaluasi tersebut dilakukan mengikuti *filter context* yang diterapkan di mana pada kasus kita kali ini kita menggunakan kolom “NAMA” dari tabel Product melalui visual Matrix sehingga akan memberikan informasi seperti berikut:

| NAMA | Transaksi Pertama |
|---|---|
| | 15/01/2015 |
| 3 Way Stop Cock (BD) | 10/01/2015 |
| 3 Way Stop Cock buntut Polymed | 14/01/2015 |
| 3 Way stop cock polymed | 01/01/2015 |
| 3.5 Cortex Screw L 10-30 mm | 15/08/2015 |
| 4.5 Cortex Screw L 14-50 mm | 23/10/2015 |
| abbocath 14G | 05/11/2015 |
| ABBOCATH 18 (x) | 25/02/2015 |
| Abbocath 18G | 04/06/2015 |

Misal, pada kasus lain dimana kita juga perlu untuk mengetahui besarnya nilai transaksi pada hari pertama kali produk tersebut terjual maka kita perlu menggunakan fungsi `FIRSTNONBLANKVALUE()` dengan DAX sebagai berikut:

```
Transaksi Pertama (Value) =
```

```
FIRSTNONBLANKVALUE(
    Date_Table[Date],
    [Total Sales]
)
```

DAX tersebut akan mencari tanggal terjadinya pertama kali transaksi dan menampilkan besarnya nilai transaksi `[Total Sales]` sebagai hasilnya. Dapat kita lihat hasilnya pada gambar di bawah dan membandingkan hasilnya dengan keseluruhan total nilai transaksi untuk setiap produk.

| NAMA | Transaksi Pertama | Transaksi Pertama (Value) | Total Sales |
|---|---|---|---|
| | 15/01/2015 | 7,768.75 | 1,021,970.21 |
| 3 Way Stop Cock (BD) | 10/01/2015 | 13,921.88 | 116,841.63 |
| 3 Way Stop Cock buntut Polymed | 14/01/2015 | 15,114.00 | 15,114.00 |
| 3 Way stop cock polymed | 01/01/2015 | 12,575.75 | 1,224,316.79 |
| 3.5 Cortex Screw L 10-30 mm | 15/08/2015 | 226,875.00 | 426,525.00 |
| 4.5 Cortex Screw L 14-50 mm | 23/10/2015 | 226,875.00 | 1,252,350.00 |
| abbocath 14G | 05/11/2015 | 15,000.00 | 15,000.00 |
| ABBOCATH 18 (x) | 25/02/2015 | 16,500.00 | 16,500.00 |
| Abbocath 18G | 04/06/2015 | 16,500.00 | 48,840.00 |
| Abbocath 20G | 25/03/2015 | 16,500.00 | 142,680.00 |
| ABBOCATH 22 (x) | 12/04/2015 | 16,500.00 | 31,500.00 |
| Abbocath 22G | 06/03/2015 | 33,000.00 | 257,520.00 |

Selanjutnya, misalkan kita ingin melihat kapan terakhir kali pelanggan melakukan transaksi atau pembelian. Dalam kasus ini, kita bisa memanfaatkan fungsi `LASTNONBLANK()` menggunakan *measure* dengan DAX sebagai berikut:

```
Transaksi Terakhir Cust =
LASTNONBLANK(
    Date_Table[Date],
    [Total Sales]
)
```

DAX di atas akan mencari tanggal terakhir kali terjadinya transaksi berdasarkan `[Total Sales]` tergantung dari *context filter* yang kita berikan. Pada visual Matrix di bawah ini, kita menggunakan kolom “KD_CUST” sehingga *filter context* yang diterapkan adalah mengikuti kode pelanggan. *Measure* yang kita buat untuk mencari tanggal transaksi terakhir kali berdasarkan nilai penjualan akan dilihat berdasarkan kode pelanggan.

| KD_CUST | Transaksi Terakhir Cust |
|---|---|
| | 02/01/2015 |
| 001 | 31/12/2015 |
| 201501010001 | 31/12/2015 |
| 201501010003 | 31/12/2015 |
| 201504010004 | 19/06/2015 |
| 201504010014 | 17/06/2015 |
| 201504010017 | 25/06/2015 |
| 201504010018 | 17/06/2015 |
| 201504010023 | 08/04/2015 |

Begitupun sama halnya ketika kita ingin mengetahui nilai transaksi terakhir kali yang dilakukan oleh pelanggan maka fungsi yang kita gunakan kali ini adalah `LASTNONBLANKVALUE` melalui *measure* dengan DAX seperti berikut:

```
Transaksi Terakhir Cust (Value) =
LASTNONBLANKVALUE(
    Date_Table[Date],
    [Total Sales]
)
```

Dapat kita lihat pada hasil di bawah ini bahwa walaupun terdapat tanggal transaksi terakhir untuk setiap pelanggan, tanggal ini tidak bekerja sebagai *filter context* sehingga apabila kita memasukkan *measure* `[Total Sales]` saja ia akan menghitung seluruh transaksi dari setiap pelanggan. Oleh karena itu kita perlu membuat *measure* lain yang untuk menghitung nilai transaksi terakhir dari setiap pelanggan.

| KD_CUST | Transaksi Terakhir Cust | Transaksi Terakhir Cust (Value) | Total Sales |
|---|---|---|---|
| | 02/01/2015 | 1,482.25 | 1,482.25 |
| 001 | 31/12/2015 | 937,350.59 | 3,097,727,611.71 |
| 201501010001 | 31/12/2015 | 3,178.00 | 3,178.00 |
| 201501010003 | 31/12/2015 | 1,095.00 | 1,095.00 |
| 201504010004 | 19/06/2015 | 15,000.00 | 123,355.00 |
| 201504010014 | 17/06/2015 | 13,733.00 | 67,148.00 |
| 201504010017 | 25/06/2015 | 5,166.00 | 206,579.00 |
| 201504010018 | 17/06/2015 | 6,492.00 | 31,613.00 |
| 201504010023 | 08/04/2015 | 1,091.00 | 1,091.00 |
| 201504010027 | 23/06/2015 | 138.00 | 25,159.00 |
| 201504010032 | 08/04/2015 | 10,600.00 | 10,600.00 |
| 201504010038 | 09/05/2015 | 298.00 | 44,346.00 |
| 201504010051 | 17/04/2015 | 25,186.00 | 25,186.00 |

# Intermediate Index DAX Techniques

## Advanced use of DAX Functions for Indexing Table Visual

Dalam DAX, kemampuan untuk melakukan kalkulasi dinamis pada kumpulan data yang spesifik sangatlah penting. Dua fungsi utama yang mendukung kemampuan ini adalah `INDEX()` dan `CALCULATE()`. `INDEX()` memungkinkan kita untuk mengekstraksi baris atau nilai berdasarkan posisi dari sebuah tabel atau kumpulan data, sementara `CALCULATE()` memungkinkan kita untuk memodifikasi *filter context* dalam perhitungan.

`CALCULATE()` dikenal sebagai fungsi paling penting dalam DAX karena dapat memengaruhi *context filter* sehingga memungkinkan kita melakukan perhitungan berdasarkan kondisi yang dinamis. Sementara itu, `INDEX()` memberikan fleksibilitas dalam memilih elemen tertentu dari tabel, misalnya, memilih nilai pada posisi ke-n dalam tabel yang diurutkan.

Struktur dari fungsi `INDEX()` adalah sebagai berikut:
**`INDEX(`*`<position>[, <relation>][, <orderBy>][, <blanks>][, <partitionBy>]`*`)`**

Struktur dari fungsi CALCULATE() adalah sebagai berikut:
**`CALCULATE(`*`<expression>[, <filter1> [, <filter2> [, …]]]`*`)`**

Misal, kita ingin menghitung selisih antara penjualan di bulan ini terhadap penjualan di awal kuartal untuk mengevaluasi performa penjualan. Pertama, kita bisa membuat sebuah visual Matrix yang menampilkan kuartal dan bulan beserta dengan total penjualan seperti berikut:

| Kuartal | Total Sales |
|---|---|
| **Q1** | **1,226,802,241.78** |
| JAN | 292,557,196.54 |
| FEB | 351,279,082.45 |
| MAR | 582,965,962.79 |
| **Q2** | **1,705,594,831.47** |
| APR | 563,298,539.11 |
| MAY | 580,786,567.65 |
| JUN | 561,509,724.70 |
| **Q3** | **1,905,637,821.89** |
| JUL | 503,712,423.70 |
| AUG | 579,562,183.71 |
| SEP | 822,363,214.48 |
| **Q4** | **1,879,253,592.98** |
| OCT | 770,733,420.11 |
| NOV | 594,493,406.92 |
| DEC | 514,026,765.95 |

Kemudian, kita bisa menggunakan fungsi `INDEX()` dan `CALCULATE()` pada *measure* dengan DAX sebagai berikut:

```
Total Sales Opening Quartal =
CALCULATE(
    [Total Sales],
    INDEX(
        1,
        ALL(
            Date_Table[Kuartal],
            Date_Table[Bulan (angka)],
            Date_Table[Bulan]
        ),
        ORDERBY(
            Date_Table[Kuartal], ASC,
            Date_Table[Bulan (angka)], ASC,
            Date_Table[Bulan], ASC
        ),
        PARTITIONBY(Date_Table[Kuartal])
    )
)
```

DAX di atas akan menghitung nilai `[Total Sales]` mengikuti ***<filter1>*** yang diterapkan pada fungsi `CALCULATE()`, yaitu menampilkan nilai `[Total Sales]` pada baris pertama dari visual Matrix yang kita buat sebelumnya.

Karena kita menggunakan kolom "Kuartal" dan "Bulan" maka kita perlu memanggilnya menggunakan fungsi `ALL()` sebagai parameter ***<relation>***. Parameter ini akan digunakan oleh fungsi `INDEX()` sebagai tabel acuan untuk parameter ***<position>***. Fungsi `ALL()` akan menampilkan seluruh baris yang ada pada data sesuai dengan kolom yang kita masukkan, fungsi ini mengabaikan *filter context* yang ada.

Kemudian, kita menggunakan fungsi `ORDERBY()` untuk parameter ***<orderBy>***. Parameter ini digunakan sebagai acuan urutan untuk parameter ***<position>***. Di sini kita mengurutkannya berdasarkan "Kuartal" terlebih dahulu baru kemudian diikuti oleh "Bulan".

Namun, jika kita perhatikan bahwa di dalam DAX di atas kita juga menggunakan kolom "Bulan (angka)" yang tidak kita gunakan untuk membuat visual Matrix. Kolom ini kita masukkan untuk membantu mengurutkan nama-nama bulan agar tidak mengikuti abjad. Kolom "Bulan (angka)" perlu dipanggil juga pada parameter ***<relation>*** agar bisa dipanggil di parameter ***<orderBy>***.

Terakhir, kita menggunakan fungsi `PARTITIONBY()` untuk parameter ***<partition>*** menggunakan kolom "Kuartal" karena kita ingin proses pencarian ***<position>*** dipartisi oleh "Kuartal". Jika kita rangkum, fungsi `INDEX()` tersebut akan mencari nilai `[Total Sales]` untuk posisi bulan pertama (baris ke-1) dari setiap "Kuartal". Jika kita masukkan *measure* tersebut ke dalam visual Matrix akan menghasilkan nilai seperti berikut:

| Kuartal | Total Sales | Total Sales Opening Quartal |
|---|---|---|
| **Q1** | **1,226,802,241.78** | **292,557,196.54** |
| JAN | 292,557,196.54 | 292,557,196.54 |
| FEB | 351,279,082.45 | 292,557,196.54 |
| MAR | 582,965,962.79 | 292,557,196.54 |
| **Q2** | **1,705,594,831.47** | **563,298,539.11** |
| APR | 563,298,539.11 | 563,298,539.11 |
| MAY | 580,786,567.65 | 563,298,539.11 |
| JUN | 561,509,724.70 | 563,298,539.11 |
| **Q3** | **1,905,637,821.89** | **503,712,423.70** |
| JUL | 503,712,423.70 | 503,712,423.70 |
| AUG | 579,562,183.71 | 503,712,423.70 |
| SEP | 822,363,214.48 | 503,712,423.70 |
| **Q4** | **1,879,253,592.98** | **770,733,420.11** |
| OCT | 770,733,420.11 | 770,733,420.11 |
| NOV | 594,493,406.92 | 770,733,420.11 |
| DEC | 514,026,765.95 | 770,733,420.11 |

Sehingga untuk menghitung selisih antara penjualan bulan ini terhadap bulan di awal kuartal kita bisa membuat *measure* dengan DAX sebagai berikut:

```
Total Sales to Opening Quartal =
[Total Sales] - [Total Sales Opening Quartal]
```

| Kuartal | Total Sales | Total Sales Opening Quartal | Total Sales to Opening Quartal |
|---|---|---|---|
| **Q1** | **1,226,802,241.78** | **292,557,196.54** | **934,245,045.24** |
| JAN | 292,557,196.54 | 292,557,196.54 | 0.00 |
| FEB | 351,279,082.45 | 292,557,196.54 | 58,721,885.91 |
| MAR | 582,965,962.79 | 292,557,196.54 | 290,408,766.24 |
| **Q2** | **1,705,594,831.47** | **563,298,539.11** | **1,142,296,292.36** |
| APR | 563,298,539.11 | 563,298,539.11 | 0.00 |
| MAY | 580,786,567.65 | 563,298,539.11 | 17,488,028.54 |
| JUN | 561,509,724.70 | 563,298,539.11 | -1,788,814.40 |
| **Q3** | **1,905,637,821.89** | **503,712,423.70** | **1,401,925,398.19** |
| JUL | 503,712,423.70 | 503,712,423.70 | 0.00 |
| AUG | 579,562,183.71 | 503,712,423.70 | 75,849,760.01 |
| SEP | 822,363,214.48 | 503,712,423.70 | 318,650,790.77 |
| **Q4** | **1,879,253,592.98** | **770,733,420.11** | **1,108,520,172.87** |
| OCT | 770,733,420.11 | 770,733,420.11 | 0.00 |
| NOV | 594,493,406.92 | 770,733,420.11 | -176,240,013.19 |
| DEC | 514,026,765.95 | 770,733,420.11 | -256,706,654.16 |

Selanjutnya, kita bisa lihat bahwa `[Total Sales to Opening Quartal]` masih menampilkan nilai untuk Subtotal setiap "Kuartal" yang sebetulnya tidak kita butuhkan dan tidak memberikan informasi bermakna. Oleh karena itu, kita bisa menambahkan fungsi `ISINSCOPE()` pada DAX yang kita buat. Fungsi ini bertujuan untuk memastikan bahwa proses perhitungan atau evaluasi DAX hanya terjadi pada kolom yang didefinisikan di dalam fungsi `ISINSCOPE()`. *Measure* `[Total Sales to Opening Quartal]` yang kita tulis sebelumnya bisa kita modifikasi menjadi seperti berikut:

```
Total Sales to Opening Quartal =
IF(
    ISINSCOPE(Date_Table[Bulan]),
    [Total Sales] - [Total Sales Opening Quartal]
)
```

DAX hanya akan mengevaluasi perhitungan apabila *context filter* saat ini sedang berada di dalam cakupan kolom "Bulan" dan mengabaikan perhitungan apabila tidak berada dalam cakupan "Bulan". Dengan membuang *measure* [Total Sales Opening Quartal] dari visual Matrix kita bisa dapatkan hasil akhir sebagai berikut:

| Kuartal | Total Sales | Total Sales to Opening Quartal |
|---|---|---|
| **Q1** | **1,226,802,241.78** | |
| JAN | 292,557,196.54 | 0.00 |
| FEB | 351,279,082.45 | 58,721,885.91 |
| MAR | 582,965,962.79 | 290,408,766.24 |
| **Q2** | **1,705,594,831.47** | |
| APR | 563,298,539.11 | 0.00 |
| MAY | 580,786,567.65 | 17,488,028.54 |
| JUN | 561,509,724.70 | -1,788,814.40 |
| **Q3** | **1,905,637,821.89** | |
| JUL | 503,712,423.70 | 0.00 |
| AUG | 579,562,183.71 | 75,849,760.01 |
| SEP | 822,363,214.48 | 318,650,790.77 |
| **Q4** | **1,879,253,592.98** | |
| OCT | 770,733,420.11 | 0.00 |
| NOV | 594,493,406.92 | -176,240,013.19 |
| DEC | 514,026,765.95 | -256,706,654.16 |

Kita tetap mendapatkan Subtotal penjualan untuk setiap kuartal tetapi tidak menampilkan informasi selisih penjualan pada setiap Subtotal kuartal yang jelas tidak memberikan informasi relevan. Cara ini juga bisa kita terapkan apabila ingin menghitung rasio penjualan bulan ini terhadap penjualan awal kuartal.

## Dynamic Visual Table Indexing

Pada kasus sebelumnya, kita memiliki kebutuhan untuk mengakses nilai pada posisi yang absolut yaitu posisi awal. Pada beberapa kasus, kita juga butuh untuk mengakses nilai pada posisi baris yang sifatnya relatif seperti mengakses nilai pada baris sebelumnya atau setelahnya dari baris yang ada saat ini.

DAX menyediakan fungsi `OFFSET()` yang berfungsi untuk mengakses nilai pada baris lain yang sifatnya relatif terhadap baris saat ini. Misal, kita ingin melihat perkembangan dari bulan ke bulan (*MoM Growth*) dari penjualan maka kita bisa mulai dengan membuat visual Table menggunakan kolom "Kuartal" dan "Bulan" dari tabel Date_Table, dan `[Total Sales]` seperti berikut:

| Kuartal | Bulan | Total Sales |
|---|---|---|
| | | 1,803,562,698.63 |
| Q1 | JAN | 292,557,196.54 |
| Q1 | FEB | 351,279,082.45 |
| Q1 | MAR | 582,965,962.79 |
| Q2 | APR | 563,298,539.11 |
| Q2 | MAY | 580,786,567.65 |
| Q2 | JUN | 561,509,724.70 |
| Q3 | JUL | 503,712,423.70 |
| Q3 | AUG | 579,562,183.71 |
| Q3 | SEP | 822,363,214.48 |
| Q4 | OCT | 770,733,420.11 |
| Q4 | NOV | 594,493,406.92 |
| Q4 | DEC | 514,026,765.95 |
| **Total** | | **8,520,851,186.75** |

Struktur dari fungsi `OFFSET()` adalah sebagai berikut:
**`OFFSET ( <delta>[, <relation>][, <orderBy>][, <blanks>][, <partitionBy>][, <matchBy>][, <reset>] )`**

Untuk mencari nilai `[Total Sales]` dari baris sebelumnya kita bisa membuat *measure* menggunakan DAX seperti berikut:

```
Tota Sales Prev Month =
CALCULATE(
    [Total Sales],
    OFFSET(
        -1,
        ALL(
            Date_Table[Kuartal],
            Date_Table[Bulan (angka)],
            Date_Table[Bulan]
        ),
        ORDERBY(
            Date_Table[Kuartal], ASC,
            Date_Table[Bulan (angka)], ASC,
            Date_Table[Bulan]
        )
    )
)
```

DAX diatas memiliki bentuk yang mirip dengan kasus sebelumnya ketika kita menggunakan `INDEX()`, hanya saja parameter **`<delta>`** dari fungsi `OFFSET()` akan mengambil nilai pada posisi relatif bukan absolut seperti parameter ***`<position>`*** dari fungsi `INDEX()`. Masukkan *measure*

tersebut ke dalam visual Table yang kita buat sebelumnya sehingga memberikan hasil seperti berikut:

| Kuartal | Bulan | Total Sales | Tota Sales Prev Month |
|---|---|---|---|
| | | 1,803,562,698.63 | |
| Q1 | JAN | 292,557,196.54 | 1,803,562,698.63 |
| Q1 | FEB | 351,279,082.45 | 292,557,196.54 |
| Q1 | MAR | 582,965,962.79 | 351,279,082.45 |
| Q2 | APR | 563,298,539.11 | 582,965,962.79 |
| Q2 | MAY | 580,786,567.65 | 563,298,539.11 |
| Q2 | JUN | 561,509,724.70 | 580,786,567.65 |
| Q3 | JUL | 503,712,423.70 | 561,509,724.70 |
| Q3 | AUG | 579,562,183.71 | 503,712,423.70 |
| Q3 | SEP | 822,363,214.48 | 579,562,183.71 |
| Q4 | OCT | 770,733,420.11 | 822,363,214.48 |
| Q4 | NOV | 594,493,406.92 | 770,733,420.11 |
| Q4 | DEC | 514,026,765.95 | 594,493,406.92 |
| **Total** | | **8,520,851,186.75** | **8,006,824,420.79** |

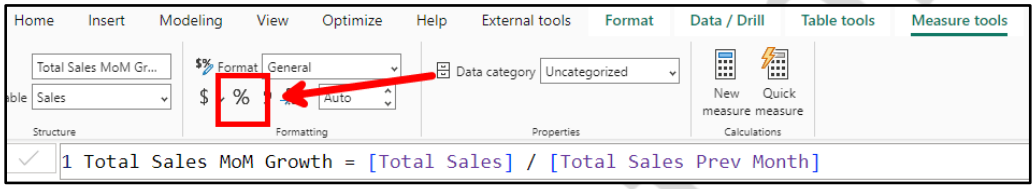
Karena ada kolom “Bulan” yang BLANK maka kita bisa modifikasi DAX agar mengabaikan nilai pada “Bulan” yang BLANK tersebut menjadi seperti berikut:

```
Tota Sales Prev Month =
CALCULATE(
    [Total Sales],
    OFFSET(
        -1,
        ALL(
            Date_Table[Kuartal],
            Date_Table[Bulan (angka)],
            Date_Table[Bulan]
        ),
        ORDERBY(
            Date_Table[Kuartal], ASC,
            Date_Table[Bulan (angka)], ASC,
            Date_Table[Bulan]
        )
    ),
    NOT ISBLANK(Date_Table[Bulan])
)
```

| Kuartal | Bulan | Total Sales | Total Sales Prev Month |
|---|---|---|---|
| | | 1,803,562,698.63 | |
| Q1 | JAN | 292,557,196.54 | |
| Q1 | FEB | 351,279,082.45 | 292,557,196.54 |
| Q1 | MAR | 582,965,962.79 | 351,279,082.45 |
| Q2 | APR | 563,298,539.11 | 582,965,962.79 |

Selanjutnya, untuk menghitung *MoM Growth* dari penjualan, kita bisa membuat *measure* menggunakan DAX berikut dan memformatnya menjadi persentase:

```
Total Sales MoM Growth = [Total Sales] / [Total Sales Prev Month]
```




| Kuartal | Bulan | Total Sales | Total Sales Prev Month | Total Sales MoM Growth |
|---|---|---|---|---|
| | | 1,803,562,698.63 | | Infinity |
| Q1 | JAN | 292,557,196.54 | | Infinity |
| Q1 | FEB | 351,279,082.45 | 292,557,196.54 | 120.07% |
| Q1 | MAR | 582,965,962.79 | 351,279,082.45 | 165.96% |
| Q2 | APR | 563,298,539.11 | 582,965,962.79 | 96.63% |
| Q2 | MAY | 580,786,567.65 | 563,298,539.11 | 103.10% |
| Q2 | JUN | 561,509,724.70 | 580,786,567.65 | 96.68% |
| Q3 | JUL | 503,712,423.70 | 561,509,724.70 | 89.71% |
| Q3 | AUG | 579,562,183.71 | 503,712,423.70 | 115.06% |
| Q3 | SEP | 822,363,214.48 | 579,562,183.71 | 141.89% |
| Q4 | OCT | 770,733,420.11 | 822,363,214.48 | 93.72% |
| Q4 | NOV | 594,493,406.92 | 770,733,420.11 | 77.13% |
| Q4 | DEC | 514,026,765.95 | 594,493,406.92 | 86.46% |

Namun, hasil dari `[Total Sales MoM Growth]` menunjukkan infinity pada nilai yang kosong. Oleh karena itu, kita bisa memanfaatkan fungsi `DIVIDE()` untuk menangani kasus pembagi nol atau kosong.

Struktur fungsi `DIVIDE()` adalah sebagai berikut:
**`DIVIDE(<numerator>, <denominator> [,<alternateresult>])`**

Sehingga kita bisa modifikasi DAX sebelumnya menjadi seperti berikut:

```
DIVIDE([Total Sales],[Total Sales Prev Month])
```

| Kuartal | Bulan | Total Sales | Total Sales Prev Month | Total Sales MoM Growth |
|---|---|---|---|---|
| | | 1,803,562,698.63 | | |
| Q1 | JAN | 292,557,196.54 | | |
| Q1 | FEB | 351,279,082.45 | 292,557,196.54 | 120.07% |
| Q1 | MAR | 582,965,962.79 | 351,279,082.45 | 165.96% |
| Q2 | APR | 563,298,539.11 | 582,965,962.79 | 96.63% |
| Q2 | MAY | 580,786,567.65 | 563,298,539.11 | 103.10% |
| Q2 | JUN | 561,509,724.70 | 580,786,567.65 | 96.68% |
| Q3 | JUL | 503,712,423.70 | 561,509,724.70 | 89.71% |
| Q3 | AUG | 579,562,183.71 | 503,712,423.70 | 115.06% |
| Q3 | SEP | 822,363,214.48 | 579,562,183.71 | 141.89% |
| Q4 | OCT | 770,733,420.11 | 822,363,214.48 | 93.72% |
| Q4 | NOV | 594,493,406.92 | 770,733,420.11 | 77.13% |
| Q4 | DEC | 514,026,765.95 | 594,493,406.92 | 86.46% |
| **Total** | | **8,520,851,186.75** | **6,203,261,722.17** | **137.36%** |

Hasil dari perhitungan fungsi `DIVIDE()` secara default mengabaikan pembagi bernilai nol atau BLANK sehingga hasilnya fokus pada informasi MoM yang seharusnya.

## Combining Index DAX with other DAX function

Sebelumnya, kita sudah menggunakan fungsi `INDEX()` dan `OFFSET()` untuk mengambil nilai dari baris tertentu pada data berdasarkan posisi absolut atau relatif. Kasus selanjutnya yang akan sering kita temui adalah ketika kita ingin mengambil kumpulan nilai dari rentang tertentu pada data. DAX menyediakan fungsi `WINDOW()` untuk memenuhi kebutuhan tersebut.

Misal, kita ingin menghitung persentase penjualan untuk setiap bulan terhadap total per kuartalnya. Hal ini dilakukan untuk mengukur performa penjualan setiap bulan dan melihat bulan yang berkontribusi paling besar di setiap kuartalnya.

Struktur dari fungsi `WINDOW()` adalah sebagai berikut:
**`WINDOW ( from[, from_type], to[, to_type][, <relation> or <axis>][, <orderBy>][, <blanks>][, <partitionBy>][, <matchBy>][, <reset>] )`**

Pertama-tama kita akan membuat *measure* untuk menghitung total penjualan per kuartal menggunakan DAX berikut:

```
Total Sales by Quartal =
CALCULATE(
    [Total Sales],
    WINDOW(
        1, ABS,
        3, ABS,
        ALL(
            Date_Table[Kuartal],
```

```
            Date_Table[Bulan (angka)],
            Date_Table[Bulan]
        ),
        ORDERBY(
            Date_Table[Kuartal], ASC,
            Date_Table[Bulan (angka)], ASC,
            Date_Table[Bulan]
        ),
        PARTITIONBY(
            Date_Table[Kuartal]
        )
    )
)
```

DAX di atas memiliki struktur yang serupa dengan cara kita menggunakan fungsi `INDEX()` dan `OFFSET()` sebelumnya. Namun, hal yang baru adalah parameter ***from [, from_type], to[, to_type]***. Parameter-parameter tersebut berperan sebagai penentu "jendela" (*window*) atau rentang dari baris yang akan diambil nilainya. Nilai `ABS` artinya adalah kita ingin mengambil nilai pada posisi absolutnya. Nilai `1` artinya adalah kita ingin mengambil posisi absolut baris pertama dari rentang yang kita tentukan. Kita menggunakan angka 1 untuk parameter ***from*** dan angka 3 untuk parameter ***to*** karena dalam satu kuartal hanya ada 3 bulan. Sehingga hasilnya akan seperti berikut:

| Kuartal | Bulan | Total Sales | Total Sales by Quartal |
|---|---|---|---|
| | | 1,803,562,698.63 | 1,803,562,698.63 |
| Q1 | JAN | 292,557,196.54 | 1,226,802,241.78 |
| Q1 | FEB | 351,279,082.45 | 1,226,802,241.78 |
| Q1 | MAR | 582,965,962.79 | 1,226,802,241.78 |
| Q2 | APR | 563,298,539.11 | 1,705,594,831.47 |
| Q2 | MAY | 580,786,567.65 | 1,705,594,831.47 |
| Q2 | JUN | 561,509,724.70 | 1,705,594,831.47 |
| Q3 | JUL | 503,712,423.70 | 1,905,637,821.89 |
| Q3 | AUG | 579,562,183.71 | 1,905,637,821.89 |
| Q3 | SEP | 822,363,214.48 | 1,905,637,821.89 |
| Q4 | OCT | 770,733,420.11 | 1,879,253,592.98 |
| Q4 | NOV | 594,493,406.92 | 1,879,253,592.98 |
| Q4 | DEC | 514,026,765.95 | 1,879,253,592.98 |
| **Total** | | **8,520,851,186.75** | **8,520,851,186.75** |

Dapat kita lihat dari hasil di atas bahwa DAX menghitung total penjualan untuk setiap kuartal. Fungsi `PARTITIONBY()` yang kita gunakan di dalam fungsi `WINDOW()` di atas berperan membatasi perhitungan pada cakupan kuartal.

Selanjutnya, kita tinggal menghitung persentase setiap bulan terhadap total penjualan kuartalnya menggunakan DAX berikut:

```
Total Sales Perc to Quartal =
DIVIDE(
    [Total Sales],
    [Total Sales by Quartal]
)
```

| Kuartal | Bulan | Total Sales | Total Sales by Quartal | Total Sales Perc to Quartal |
|---|---|---|---|---|
| | | 1,803,562,698.63 | 1,803,562,698.63 | 100.00% |
| Q1 | JAN | 292,557,196.54 | 1,226,802,241.78 | 23.85% |
| Q1 | FEB | 351,279,082.45 | 1,226,802,241.78 | 28.63% |
| Q1 | MAR | 582,965,962.79 | 1,226,802,241.78 | 47.52% |
| Q2 | APR | 563,298,539.11 | 1,705,594,831.47 | 33.03% |
| Q2 | MAY | 580,786,567.65 | 1,705,594,831.47 | 34.05% |
| Q2 | JUN | 561,509,724.70 | 1,705,594,831.47 | 32.92% |
| Q3 | JUL | 503,712,423.70 | 1,905,637,821.89 | 26.43% |
| Q3 | AUG | 579,562,183.71 | 1,905,637,821.89 | 30.41% |
| Q3 | SEP | 822,363,214.48 | 1,905,637,821.89 | 43.15% |
| Q4 | OCT | 770,733,420.11 | 1,879,253,592.98 | 41.01% |
| Q4 | NOV | 594,493,406.92 | 1,879,253,592.98 | 31.63% |
| Q4 | DEC | 514,026,765.95 | 1,879,253,592.98 | 27.35% |
| **Total** | | **8,520,851,186.75** | **8,520,851,186.75** | **100.00%** |

Kasus selanjutnya yang biasa ditemui dalam kebutuhan penggunaan indeks adalah total kumulatif atau *running total*. Konsepnya adalah sama, dimana kita mencari total nilai dari rentang baris tertentu hanya saja kali ini kita akan memodifikasi cara menentukan posisi absolut dan relatifnya. Misal, kita ingin melihat total kumulatif penjualan dari bulan ke bulan. Hal ini berguna untuk melihat bulan mana yang memengaruhi sebagian besar penjualan.

Untuk menghitung total kumulatif menggunakan fungsi `WINDOW()`, kita dapat membuat *measure* menggunakan DAX berikut:

```
Total Sales Cumulative MtM =
CALCULATE(
    [Total Sales],
    WINDOW(
        1, ABS,
        0, REL,
        ALL(
            Date_Table[Kuartal],
            Date_Table[Bulan (angka)],
            Date_Table[Bulan]
        ),
        ORDERBY(
            Date_Table[Kuartal], ASC,
            Date_Table[Bulan (angka)], ASC,
```

```
            Date_Table[Bulan]
        )
    )
)
```

| Kuartal | Bulan | Total Sales | Total Sales Cumulative MtM |
|---|---|---|---|
| | | 1,803,562,698.63 | 1,803,562,698.63 |
| Q1 | JAN | 292,557,196.54 | 2,096,119,895.17 |
| Q1 | FEB | 351,279,082.45 | 2,447,398,977.63 |
| Q1 | MAR | 582,965,962.79 | 3,030,364,940.41 |
| Q2 | APR | 563,298,539.11 | 3,593,663,479.52 |
| Q2 | MAY | 580,786,567.65 | 4,174,450,047.17 |
| Q2 | JUN | 561,509,724.70 | 4,735,959,771.88 |
| Q3 | JUL | 503,712,423.70 | 5,239,672,195.58 |
| Q3 | AUG | 579,562,183.71 | 5,819,234,379.29 |
| Q3 | SEP | 822,363,214.48 | 6,641,597,593.76 |
| Q4 | OCT | 770,733,420.11 | 7,412,331,013.88 |
| Q4 | NOV | 594,493,406.92 | 8,006,824,420.79 |
| Q4 | DEC | 514,026,765.95 | 8,520,851,186.75 |
| **Total** | | **8,520,851,186.75** | **8,520,851,186.75** |

Karena kita menggunakan posisi `ABS` untuk parameter ***`from`*** dan `REL` untuk parameter ***`to`*** maka proses perhitungan total penjualan akan selalu bertambah semakin ke bawah dari baris visual Table yang kita buat tetapi posisi 1 akan terus bertahan sehingga perhitung akan selalu dimulai dari baris paling atas dari visual Table.

# How `CALCULATE()` and `FILTER()` Work in DAX

## Introduction to `CALCULATE()` and `CALCULATETABLE()`

Dalam DAX, `CALCULATE()` dan `CALCULATETABLE()` adalah dua fungsi penting yang memungkinkan kita untuk memanipulasi *filter context* pada sebuah ekspresi atau tabel data. Keduanya digunakan untuk membuat kalkulasi yang lebih kompleks dengan mengubah atau mengatur ulang *filter context* yang ada, sehingga memungkinkan perhitungan dengan logika yang lebih dinamis.

`CALCULATE()` bekerja pada level ekspresi dan digunakan untuk menghasilkan satu nilai scalar. Sementara itu, `CALCULATETABLE()` bekerja pada level tabel dan mengembalikan seluruh tabel yang bisa digunakan sebagai bagian dari perhitungan yang lebih lanjut.

**`CALCULATE()`**

`CALCULATE()` adalah salah satu fungsi DAX yang paling kuat dan esensial karena memungkinkan kita untuk mengubah *filter context* secara dinamis dan kemudian menghitung ulang ekspresi berdasarkan filter yang diterapkan. Secara teknis, `CALCULATE()` mengubah *filter context* yang berlaku saat ini dengan menambahkan, mengganti, atau menghapus filter.

Misalkan kita ingin menghitung total penjualan hanya untuk produk tertentu, katakanlah untuk "Acid Salicyl". Maka kita bisa untuk membatasi perhitungan `[Total Sales]` hanya pada produk tersebut kita bisa masukkan DAX seperti berikut:

```
Total Sales for Acid Salicyl =
CALCULATE(
    [Total Sales],
    'Product'[NAMA] = "Acid Salicyl"
)
```

| Kuartal | Bulan | Total Sales | Total Sales for Acid Salicyl |
|---|---|---|---|
| | | 1,803,562,698.63 | 0.00 |
| Q1 | JAN | 292,557,196.54 | 0.00 |
| Q1 | FEB | 351,279,082.45 | 6,933.91 |
| Q1 | MAR | 582,965,962.79 | 4,749.25 |
| Q2 | APR | 563,298,539.11 | 59,874.28 |
| Q2 | MAY | 580,786,567.65 | 112,168.88 |
| Q2 | JUN | 561,509,724.70 | 80,326.00 |
| Q3 | JUL | 503,712,423.70 | 176,622.88 |
| Q3 | AUG | 579,562,183.71 | 159,501.00 |
| Q3 | SEP | 822,363,214.48 | 260,990.88 |
| Q4 | OCT | 770,733,420.11 | 361,906.48 |
| Q4 | NOV | 594,493,406.92 | 687.60 |
| Q4 | DEC | 514,026,765.95 | 0.00 |
| **Total** | | **8,520,851,186.75** | **1,223,761.14** |

| NAMA | Total Sales | Total Sales for Acid Salicyl |
|---|---|---|
| Acid Citric | | 1,223,761.14 |
| Acid Salicyl | 1,223,761.14 | 1,223,761.14 |
| Aclar 200 mg | | 1,223,761.14 |
| Aclam 500 | | 1,223,761.14 |
| ACLAM FORTE SYR | | 1,223,761.14 |
| ACLAM SYR 60ML | | 1,223,761.14 |
| ACNA CARE CAP | 23,122.00 | 1,223,761.14 |
| ACNE AID SOAP | | 1,223,761.14 |
| Acne Feldin Lotion | 69,959.64 | 1,223,761.14 |
| ACPULSIF 5MG | | 1,223,761.14 |
| Acran 150mg tab | 203,953.18 | 1,223,761.14 |
| Acran Inj | 893,691.92 | 1,223,761.14 |
| Acrios 100 mg | | 1,223,761.14 |
| **Total** | **8,520,851,186.75** | **1,223,761.14** |

Dapat kita lihat pada hasil di atas perbedaan antara [Total Sales] tanpa filter dan `[Total Sales for Acid Salicyl]` yang sudah difilter hanya untuk produk “Acid Salicyl”. Begitupun pada visual Table yang di gambar sebelah kanan, *measure* `[Total Sales for Acid Salicyl]` mengabaikan *filter context* yang ada saat itu, dan hanya menganggap bahwa setiap perhitungan adalah untuk produk “Acid Salicyl”.

Ini adalah salah satu kelebihan dari fungsi `CALCULATE()`, kemampuannya yang dapat merubah *filter context* dari suatu perhitungan menjadikannya fungsi yang sangat canggih untuk keperluan perhitungan yang kompleks.

**`CALCULATETABLE()`**

`CALCULATETABLE()` bekerja mirip dengan `CALCULATE()`, tetapi fungsi ini mengembalikan seluruh tabel yang bisa digunakan dalam konteks perhitungan yang lebih lanjut. Fungsi ini sangat berguna ketika kita ingin menghasilkan subset dari data dalam tabel berdasarkan filter tertentu.

Misal, DAX berikut akan menghitung seluruh penjualan hanya pada bulan Januari (awal tahun):

```
Total Sales for January =
SUMX(
```

```
    CALCULATETABLE(
        Date_Table,
        Date_Table[Bulan] = "JAN"
    ),
    [Total Sales]
)
```

| Bulan | Total Sales | Total Sales for January |
|---|---|---|
| | 1,803,562,698.63 | |
| JAN | 292,557,196.54 | 292,557,196.54 |
| FEB | 351,279,082.45 | |
| MAR | 582,965,962.79 | |
| APR | 563,298,539.11 | |
| MAY | 580,786,567.65 | |
| JUN | 561,509,724.70 | |
| JUL | 503,712,423.70 | |
| AUG | 579,562,183.71 | |
| SEP | 822,363,214.48 | |
| OCT | 770,733,420.11 | |
| NOV | 594,493,406.92 | |
| DEC | 514,026,765.95 | |
| **Total** | **8,520,851,186.75** | **292,557,196.54** |

Hasil dari *measure* tersebut hanya menampilkan nilai pada bulan Januari, dan BLANK pada bulan lainnya.

**Mengapa `CALCULATE()` dan `CALCULATETABLE()` penting?**

`CALCULATE()` dan `CALCULATETABLE()` memungkinkan kita mengontrol *filter context* secara dinamis dan fleksibel, yang menjadi dasar dari banyak skenario analisis yang kompleks dalam DAX. Beberapa alasan penting menggunakan fungsi ini antara lain:

- Menghitung ulang metrik berdasarkan subset data: Misalnya, menghitung rata-rata penjualan hanya untuk produk tertentu atau wilayah tertentu.
- Menyediakan filter yang dinamis: Filter bisa diterapkan dan diubah sesuai kebutuhan analisis, tanpa perlu memodifikasi seluruh model data.
- Analisis temporal: Digunakan untuk menghitung metrik berdasarkan waktu atau periode tertentu (misalnya, menghitung total penjualan hanya untuk tahun lalu atau bulan tertentu).

Perbedaan utama antara kedua fungsi ini terletak pada jenis hasil yang mereka kembalikan. `CALCULATE()` mengembalikan nilai scalar (satu nilai) sebagai hasil kalkulasi, misalnya penjumlahan atau rata-rata. `CALCULATETABLE()` mengembalikan tabel yang berisi data yang telah difilter sehingga memungkinkan analisis yang lebih lanjut pada subset data tersebut.

Fungsi `CALCULATE()` dan `CALCULATETABLE()` adalah kunci untuk melakukan perhitungan yang lebih kompleks dan dinamis dalam DAX. Keduanya memungkinkan kita untuk memanipulasi *filter context* sehingga kita dapat menghitung ulang ekspresi atau menghasilkan subset data yang

relevan untuk analisis lebih lanjut. Memahami cara menggunakan kedua fungsi ini akan sangat membantu dalam membuat laporan Power BI yang lebih kaya dan lebih bermakna.

## Creating Filter Context with CALCULATE()

Ketika menggunakan CALCULATE(), kita memiliki kemampuan untuk mengontrol dan memodifikasi *filter context* secara dinamis. DAX menyediakan beberapa fungsi yang dapat digunakan untuk mengubah atau menyesuaikan filter ini yang dikenal sebagai ***filter modifier***. ***Filter modifier*** ini memungkinkan kita untuk menambah, menghapus, atau mempertahankan filter secara selektif sesuai kebutuhan dalam analisis data.

*Filter modifier* yang paling umum digunakan dalam DAX adalah `ALL()`, `ALLEXCEPT()`, `FILTER()`, `KEEPFILTERS()`, `REMOVEFILTERS()`. Mari kita bahas lebih dalam masing-masing fungsi ini dan cara kerjanya dalam `CALCULATE()`.

**`ALL()`**
Struktur dari fungsi ini adalah sebagai berikut:
**`ALL( `** ***`[<table> | <column>[, <column>[, <column>[,…]]]]`*** **`)`**

Kegunaan dari fungsi ini adalah mengembalikan seluruh baris dan nilai yang ada pada tabel atau kolom yang diberikan. Fungsi ini akan mengabaikan semua filter yang sebelumnya diterapkan, kita bisa tetap mendapatkan data utuh meskipun terdapat filter yang sudah diterapkan. Tabel yang digunakan harus tabel asli dan bukan sebuah ekspresi yang menghasilkan tabel.

Misal, kita ingin menghitung rasio penjualan setiap pabrikan obat terhadap keseluruhan penjualan maka kita bisa membuat *measure* dengan menggunakan DAX berikut:

```
Total Sales (ALL) =
CALCULATE(
    [Total Sales],
    ALL('Product'[KD_PABRIK])
)
```

DAX tersebut akan menghitung `[Total Sales]` dengan mengabaikan semua *filter context* yang datang dari kolom “KD_PABRIK” dari tabel Product sehingga dapat kita lihat hasilnya seperti di gambar berikut:

| KD_PABRIK | Total Sales | Total Sales (ALL) |
|---|---|---|
| Zn, | 4,936,111.13 | 8,520,851,186.75 |
| Yari | 10,952,797.49 | 8,520,851,186.75 |
| Wida | 105,347,106.49 | 8,520,851,186.75 |
| vial | 408,465.73 | 8,520,851,186.75 |
| tube | 32,906,720.54 | 8,520,851,186.75 |
| Tpam | | 8,520,851,186.75 |
| Tmin | 26,383,641.62 | 8,520,851,186.75 |
| Temp | 1,283,985.40 | 8,520,851,186.75 |
| Tana | 3,644,092.42 | 8,520,851,186.75 |
| Take | 78,199,489.68 | 8,520,851,186.75 |
| tab drag | | 8,520,851,186.75 |
| tab | 167,801.69 | 8,520,851,186.75 |
| suppos | 460,027.00 | 8,520,851,186.75 |
| **Total** | **8,520,851,186.75** | **8,520,851,186.75** |

Perlu diperhatikan bahwa fungsi `ALL()` hanya akan mengabaikan *filter context* dari kolom atau tabel yang kita definisikan di dalamnya. Misal, ada nama kolom yang sama dari tabel yang berbeda maka tidak akan memengaruhi cara `CALCULATE()` mengevaluasi *filter context* yang ada.

Contohnya, kita memiliki kolom “KD_OBAT” pada tabel Product dan Sales. Misal, kita buat *measure* yang mengabaikan *filter context* dari kolom “KD_OBAT” pada tabel Product seperti berikut:

```
Total Sales (ALL KD_OBAT Product) =
CALCULATE(
    [Total Sales],
    ALL('Product'[KD_OBAT])
)
```

KD_OBAT dari tabel Product

| KD_OBAT | Total Sales (ALL KD_OBAT Product) |
|---|---|
| | 8,520,851,186.75 |
| AI-0001 | 8,520,851,186.75 |
| AI-0002 | 8,520,851,186.75 |
| AI-0003 | 8,520,851,186.75 |
| AI-0004 | 8,520,851,186.75 |
| AI-0005 | 8,520,851,186.75 |
| AI-0006 | 8,520,851,186.75 |
| AI-0007 | 8,520,851,186.75 |
| AI-0008 | 8,520,851,186.75 |
| AI-0009 | 8,520,851,186.75 |
| AI-0010 | 8,520,851,186.75 |
| AI-0011 | 8,520,851,186.75 |
| AI-0012 | 8,520,851,186.75 |
| **Total** | **8,520,851,186.75** |

KD_OBAT dari tabel Sales

| KD_OBAT | Total Sales (ALL KD_OBAT Product) |
|---|---|
| AI-0001 | 460,027.00 |
| AI-0010 | 792.00 |
| AI-0011 | 12,300.00 |
| AI-0024 | 33,500.45 |
| AI-0025 | 104,938.06 |
| AI-0027 | 1,817,798.27 |
| AI-0028 | 14,852.00 |
| AI-0029 | 18,222.00 |
| AI-0031 | 144,131,189.00 |
| AI-0032 | 11,920.74 |
| AI-0033 | 2,713,743.80 |
| AI-0039 | 2,214,437.13 |
| AI-0043 | 476,012.61 |
| **Total** | **8,520,851,186.75** |

Ketika kita membuat visual Tabel menggunakan “KD_OBAT” dari tabel Product maka kita akan mendapatkan hasil sesuai yang diharapkan yaitu nilai `[Total Sales]` yang mengabaikan *filter context* dari kode obat. Namun, berbeda hasilnya ketika kita menggunakan “KD_OBAT” dari tabel Sales, nilai `[Total Sales]` tetap mengikuti *filter context* dari kode obat yang diberikan. Hal ini terjadi karena ketika kita mendefinisikan *measure* `[Total Sales (ALL KD_OBAT Product)]`, kolom yang digunakan di dalam fungsi `ALL()` adalah kolom “KD_OBAT” dari tabel Product.

Sampai sini kita sudah memahami cara kerja fungsi `ALL()`, kembali lagi ke kasus di awal yaitu kita ingin melihat rasio penjualan setiap pabrikan obat terhadap keseluruhan penjualan. Kita sudah berhasil membuat *measure* untuk menghitung keseluruhan penjualan yang mengabaikan *filter context* saat masuk ke dalam visual. Selanjutnya, kita tinggal membuat *measure* untuk mencari rasionya menggunakan DAX seperti berikut:

```
Total Sales % KD_PABRIK =
DIVIDE(
    [Total Sales],
    [Total Sales (ALL)]
)
```

| KD_PABRIK | Total Sales | Total Sales (ALL) | Total Sales % KD_PABRIK |
|---|---|---|---|
| Zn, | 4,936,111.13 | 8,520,851,186.75 | 0.06% |
| Yari | 10,952,797.49 | 8,520,851,186.75 | 0.13% |
| Wida | 105,347,106.49 | 8,520,851,186.75 | 1.24% |
| vial | 408,465.73 | 8,520,851,186.75 | 0.00% |
| tube | 32,906,720.54 | 8,520,851,186.75 | 0.39% |
| Tpam | | 8,520,851,186.75 | |
| Tmin | 26,383,641.62 | 8,520,851,186.75 | 0.31% |
| Temp | 1,283,985.40 | 8,520,851,186.75 | 0.02% |
| Tana | 3,644,092.42 | 8,520,851,186.75 | 0.04% |
| Take | 78,199,489.68 | 8,520,851,186.75 | 0.92% |
| tab drag | | 8,520,851,186.75 | |
| tab | 167,801.69 | 8,520,851,186.75 | 0.00% |
| suppos | 460,027.00 | 8,520,851,186.75 | 0.01% |
| **Total** | **8,520,851,186.75** | **8,520,851,186.75** | **100.00%** |

**ALLEXCEPT()**

Struktur dari fungsi ini adalah sebagai berikut:

**`ALLEXCEPT(<table>,<column>[,<column>[,…]])`**

Fungsi ini menghilangkan seluruh *filter context* yang sudah diterapkan pada ***<table>*** kecuali pada ***<column>*** yang diberikan. kita bisa tetap mendapatkan data utuh meskipun terdapat filter yang sudah diterapkan. Tabel atau kolom yang digunakan harus tabel/kolom asli dan bukan sebuah ekspresi yang menghasilkan tabel/kolom.

Misal, kita memiliki visual Table yang menampilkan penjualan obat untuk setiap resep yang dibuat oleh setiap dokter seperti berikut:

| KD_DOKTER | NO_RESEP | Total Sales |
|---|---|---|
| **RL-11.2014-65-0** | RL-11.2015-65-0077 | 73,977.00 |
| **RL-11.2014-63-0** | RL-11.2015-63-0052 | 360,703.00 |
| | RL-11.2015-63-0054 | 238.00 |
| **DISP_IF** | KC-07.2015-09-0035 | 2,556.00 |
| **098** | KC-11.2015-09-0046 | 5,909.00 |
| | KC-12.2015-09-0028 | 980.00 |
| | RJ-10.2015-37-0255 | 6,249.00 |
| | RJ-12.2015-38-0949 | 436.00 |
| | RL-09.2015-09-0037 | 495.00 |
| | RL-10.2015-37-0530 | 837.00 |
| | RL-11.2015-09-0274 | 495.00 |
| | RL-11.2015-09-0275 | 495.00 |
| | RL-11.2015-09-0278 | 4,232.00 |

Kemudian, kita ingin menghitung persentase penjualan obat untuk setiap resep yang ditulis oleh masing-masing dokter. Kita perlu mencari nilai total penjualan untuk setiap dokter baru kemudian membagi total penjualan setiap resep terhadap seluruh resep yang ditulis oleh dokter tersebut.

Kita perlu membuat *measure* menggunakan DAX seperti berikut:

```
Total Sales per Dokter =
CALCULATE(
    [Total Sales],
    ALLEXCEPT(
        Prescript,
        Prescript[KD_DOKTER]
    )
)
```

| KD_DOKTER | NO_RESEP | Total Sales | Total Sales per Dokter |
|---|---|---|---|
| **RL-11.2014-65-0** | RL-11.2015-65-0077 | 73,977.00 | 73,977.00 |
| **RL-11.2014-63-0** | RL-11.2015-63-0052 | 360,703.00 | 360,941.00 |
| | RL-11.2015-63-0054 | 238.00 | 360,941.00 |
| **DISP_IF** | KC-07.2015-09-0035 | 2,556.00 | 2,556.00 |
| **098** | KC-11.2015-09-0046 | 5,909.00 | 20,128.00 |
| | KC-12.2015-09-0028 | 980.00 | 20,128.00 |
| | RJ-10.2015-37-0255 | 6,249.00 | 20,128.00 |
| | RJ-12.2015-38-0949 | 436.00 | 20,128.00 |
| | RL-09.2015-09-0037 | 495.00 | 20,128.00 |
| | RL-10.2015-37-0530 | 837.00 | 20,128.00 |
| | RL-11.2015-09-0274 | 495.00 | 20,128.00 |
| | RL-11.2015-09-0275 | 495.00 | 20,128.00 |
| | RL-11.2015-09-0278 | 4,232.00 | 20,128.00 |
| **097** | KC-06.2015-37-0045 | 18,975.00 | 10,126,280.00 |
| | KC-08.2015-37-0029 | 31,324.00 | 10,126,280.00 |
| | RJ-06.2015-38-0042 | 13,294.00 | 10,126,280.00 |

Fungsi `ALLEXCEPT()` yang kita gunakan akan mengabaikan seluruh *filter context* aktif yang datang dari seluruh kolom dari tabel Prescript kecuali dari kolom “KD_DOKTER”. Ini adalah kelebihan dari penggunaan `ALLEXCEPT()` karena ketika kita menambahkan kolom lain yang berasal dari tabel Prescript ke dalam visual tersebut, maka kolom tersebut tidak akan memengaruhi *filter context* dari proses evaluasi DAX.

| KD_DOKTER | REG_AS | NO_RESEP | Total Sales | Total Sales per Dokter |
|---|---|---|---|---|
| **092** | **R** | RL-12.2015-37-0162 | 715.00 | 495,470.92 |
| | | RL-12.2015-37-0187 | 4,616.00 | 495,470.92 |
| | | RL-12.2015-65-0036 | 4,375.00 | 495,470.92 |
| ⊟ **091** | ⊟ **I** | RJ-04.2015-38-0192 | 11,552.59 | 14,196,438.59 |
| | | RJ-05.2015-38-0015 | 21,205.00 | 14,196,438.59 |
| | | RJ-05.2015-38-0360 | 748.00 | 14,196,438.59 |
| | | RJ-05.2015-38-0361 | 79,892.00 | 14,196,438.59 |
| | | RJ-06.2015-38-0145 | 3,893.00 | 14,196,438.59 |
| | | RJ-06.2015-38-0447 | 4,559.00 | 14,196,438.59 |
| | | RJ-06.2015-38-0473 | 18,130.00 | 14,196,438.59 |
| | | RJ-06.2015-38-0738 | 75,604.00 | 14,196,438.59 |
| | | RJ-08.2015-38-0377 | 6,267.00 | 14,196,438.59 |
| | | RJ-09.2015-38-0680 | 3,465.00 | 14,196,438.59 |
| | | RJ-11.2015-38-1202 | 17,160.00 | 14,196,438.59 |
| | ⊟ **R** | KC-12.2015-64-0005 | 668,525.00 | 14,196,438.59 |
| | | RI-04.2015-36-0845 | 10,627.38 | 14,196,438.59 |
| | | RI-04.2015-36-0868 | 570.63 | 14,196,438.59 |
| | | RI-04.2015-36-1645 | 8,925.00 | 14,196,438.59 |
| | | RI-04.2015-36-1653 | 7,395.00 | 14,196,438.59 |
| | | RI-04.2015-36-1654 | 7,395.00 | 14,196,438.59 |
| | | RI-04.2015-36-1656 | 4,377.00 | 14,196,438.59 |

Seperti pada contoh di atas, ketika kita menambahkan kolom “REG_AS” maka `[Total Sales per Dokter]` tidak dipengaruhi oleh *filter context* yang datang dari kolom tersebut karena fungsi `ALLEXCEPT()` yang kita gunakan di dalam *measure* tersebut. Kemudian, langkah terakhir untuk mendapatkan persentase penjualan setiap resep untuk setiap dokter kita bisa gunakan DAX berikut:

```
Total Sales % per Resep Dokter =
DIVIDE(
    [Total Sales],
    [Total Sales per Dokter]
)
```

| KD_DOKTER | REG_AS | NO_RESEP | Total Sales | Total Sales per Dokter | Total Sales % per Resep Dokter |
|---|---|---|---|---|---|
| ⊟ **RL-11.2014-65-0** | ⊟ **R** | RL-11.2015-65-0077 | 73,977.00 | 73,977.00 | 100.00% |
| ⊟ **RL-11.2014-63-0** | ⊟ **R** | RL-11.2015-63-0052 | 360,703.00 | 360,941.00 | 99.93% |
| | | RL-11.2015-63-0054 | 238.00 | 360,941.00 | 0.07% |
| ⊟ **DISP_IF** | ⊟ **R** | KC-07.2015-09-0035 | 2,556.00 | 2,556.00 | 100.00% |
| ⊟ **098** | ⊟ **I** | RJ-12.2015-38-0949 | 436.00 | 20,128.00 | 2.17% |
| | ⊟ **R** | KC-11.2015-09-0046 | 5,909.00 | 20,128.00 | 29.36% |
| | | KC-12.2015-09-0028 | 980.00 | 20,128.00 | 4.87% |
| | | RJ-10.2015-37-0255 | 6,249.00 | 20,128.00 | 31.05% |
| | | RL-09.2015-09-0037 | 495.00 | 20,128.00 | 2.46% |
| | | RL-10.2015-37-0530 | 837.00 | 20,128.00 | 4.16% |
| | | RL-11.2015-09-0274 | 495.00 | 20,128.00 | 2.46% |
| | | RL-11.2015-09-0275 | 495.00 | 20,128.00 | 2.46% |
| | | RL-11.2015-09-0278 | 4,232.00 | 20,128.00 | 21.03% |
| ⊟ **097** | ⊟ **I** | RI-09.2015-38-0094 | 12,650.00 | 10,126,280.00 | 0.12% |
| | | RI-09.2015-40-0300 | 60,857.00 | 10,126,280.00 | 0.60% |
| | | RJ-05.2015-38-0643 | 18,975.00 | 10,126,280.00 | 0.19% |

**KEEPFILTERS()**
Struktur dari fungsinya adalah sebagai berikut:
**KEEPFILTERS(*<expression>*)**

Secara default, `CALCULATE()` dapat menggantikan filter yang sudah ada. Namun, dengan `KEEPFILTERS()`, kita dapat menambahkan filter baru tanpa menghapus atau menggantikan filter yang sudah ada. Ini berguna untuk menjaga filter yang diterapkan pada visual atau laporan, sekaligus menambahkan filter tambahan pada ekspresi kita.

Fungsi ini mengembalikan sebuah tabel dengan nilai yang memenuhi syarat dari parameter ***<expression>***. Fungsi ini khusus digunakan untuk memodifikasi sifat filter (*filter modifier*) dari fungsi `CALCULATE()` dan `CALCULATETABLE()`.

Kita telah memahami bagaimana cara `CALCULATE()` dan *filter modifier* bekerja untuk mengubah *filter context* yang ada. Fungsi `KEEPFILTERS()` bekerja dengan cara yang lebih unik. Misal, kita membuat *measure* dengan DAX sebagai berikut:

```
Total Sales BPJS & Insurance (KEEPFILTERS) =
CALCULATE(
    [Total Sales],
    KEEPFILTERS(
        Prescript[REG_AS] in {"A", "I"}
    )
)
```

DAX tersebut akan menghitung `[Total Sales]` hanya untuk tipe resep “A” (BPJS) dan “I” (Insurance). Jika kita masukkan *measure* tersebut ke dalam sebuah visual misalnya Table maka kita akan dapatkan hasil seperti berikut:

| REG_AS | Total Sales | Total Sales BPJS & Insurance (KEEPFILTERS) |
|---|---|---|
| | 1,803,562,698.63 | |
| A | 4,947,474.40 | 4,947,474.40 |
| I | 853,423,500.24 | 853,423,500.24 |
| R | 5,858,917,513.48 | |
| **Total** | **8,520,851,186.75** | **858,370,974.64** |

Jika kita lihat hasilnya, `CALCULATE()` memberikan hasil yang berbeda dari biasanya. Kita ketahui bahwa sifat bawaan `CALCULATE()` adalah mengubah *filter context* yang ada sehingga jika kita memfilter kolom “REG_AS” hanya untuk BPJS dan Insurance menggunakan filter dasar dari `CALCULATE()` dengan DAX sebagai berikut:

```
Total Sales BPJS & Insurance (Plan Filter) =
CALCULATE(
```

```
    [Total Sales],
    Prescript[REG_AS] in {"A", "I"}
)
```

Maka hasil perhitungan CALCULATE() seharusnya adalah seperti berikut:

| REG_AS | Total Sales | Total Sales BPJS & Insurance (Plan Filter) |
|---|---|---|
| | 1,803,562,698.63 | 858,370,974.64 |
| A | 4,947,474.40 | 858,370,974.64 |
| I | 853,423,500.24 | 858,370,974.64 |
| R | 5,858,917,513.48 | 858,370,974.64 |
| **Total** | **8,520,851,186.75** | **858,370,974.64** |

Setiap *filter context* dari kolom "REG_AS" akan diabaikan dan hanya menghitung total penjualan untuk BPJS ("A") dan Insurance ("I") sehingga hasil dari setiap baris adalah sama yaitu `858,370,974.64`.

Selanjutnya, mari kita lihat nilai untuk `[Total Sales BPJS & Insurance (KEEPFILTERS)]`. Fungsi `KEEPFILTERS()` akan mencari "irisan" antara *outer filter* dengan *inner filter* dari kolom "REG_AS". Ketika kita bilang *outer filter*, artinya adalah seluruh filter yang berasal dari luar fungsi `CALCULATE()`. Sementara itu, *inner filter* adalah filter yang diekspresikan di dalam fungsi `CALCULATE()`. Walaupun kita sudah mendefinisikan untuk menghitung nilai `[Total Sales]` hanya pada BPJS dan Insurance. Namun, adanya kolom paling kiri yang berisi "REG_AS" berperan sebagai *outer filter* menyebabkan nilai `[Total Sales]` untuk "A" & "I" tetap sesuai dengan nilainya masing-masing. Sementara itu, nilai `[Total Sales]` untuk "REG_AS" Reguler ("R") adalah kosong karena tidak ada "irisan" antara nilai "R" dengan nilai "I" atau "A".

| REG_AS | Total Sales | Total Sales BPJS & Insurance (KEEPFILTERS) | Total Sales BPJS & Insurance (Plan Filter) |
|---|---|---|---|
| | 1,803,562,698.63 | | 858,370,974.64 |
| A | 4,947,474.40 | 4,947,474.40 | 858,370,974.64 |
| I | 853,423,500.24 | 853,423,500.24 | 858,370,974.64 |
| R | 5,858,917,513.48 | | 858,370,974.64 |
| **Total** | **8,520,851,186.75** | **858,370,974.64** | **858,370,974.64** |

**REMOVEFILTERS()**

Struktur dari fungsinya adalah sebagai berikut:

**`REMOVEFILTERS(`*`[<table> | <column>[, <column>[, <column>[,…]]]]`*`)`**

`REMOVEFILTERS()` secara eksplisit menghapus filter pada kolom atau tabel tertentu dalam sebuah perhitungan. Fungsi ini memungkinkan kita menghapus filter dari satu atau beberapa kolom/tabel, tetapi tetap mempertahankan filter pada kolom atau tabel lain yang tidak disebutkan.

Misal, kita memiliki visual Table yang dibuat dari kolom “RACIK” dan “REG_AS” dari tabel Prescript seperti berikut:

| RACIK | REG_AS | Total Sales |
|---|---|---|
| | | 1,803,562,698.63 |
| N | A | 4,947,474.40 |
| | I | 846,275,696.75 |
| | R | 5,857,864,212.85 |
| Y | I | 7,147,803.50 |
| | R | 1,053,300.63 |

Kemudian, kita ingin menghitung total penjualan untuk setiap jenis “RACIK” sehingga kita perlu menghilangkan *filter context* dari kolom “REG_AS”. Kita bisa membuat *measure* dengan DAX seperti berikut:

```
Total Sales (REMOVEFILTERS REG_AS) =
CALCULATE(
    [Total Sales],
    REMOVEFILTERS(Prescript[REG_AS])
)
```

| RACIK | REG_AS | Total Sales | Total Sales (REMOVEFILTERS REG_AS) |
|---|---|---|---|
| | | 1,803,562,698.63 | 1,803,562,698.63 |
| N | A | 4,947,474.40 | 6,709,087,384.00 |
| | I | 846,275,696.75 | 6,709,087,384.00 |
| | R | 5,857,864,212.85 | 6,709,087,384.00 |
| Y | I | 7,147,803.50 | 8,201,104.12 |
| | R | 1,053,300.63 | 8,201,104.12 |

Kita bisa gunakan informasi dari *measure* tersebut misalnya untuk mencari rasio penjualan dari masing-masing jenis resep terhadap raciknya untuk menghitung biaya seperti tuslah dan lain-lain.

Pada dasarnya, prinsip kerja `REMOVEFILTERS()` dan `ALL()` adalah sama. Namun, terdapat perbedaan yang signifikan dalam perilaku keduanya:

- `ALL()` menghapus filter dan mengembalikan sebuah tabel atau kolom, yang bisa digunakan dalam fungsi DAX lain seperti `CALCULATE()` dan `SUMX()`. Ini memungkinkan untuk membuat measure dan calculated column yang kompleks berdasarkan data tanpa filter.
- `REMOVEFILTERS()` tidak mengembalikan tabel atau kolom; ia hanya menghapus filter dari tabel atau kolom yang ditentukan. Fungsi ini dapat digunakan di dalam `CALCULATE()` untuk menghapus filter, tetapi tidak seperti `ALL()`, ia tidak bisa digunakan bersama fungsi DAX lain seperti `SUMX()`.

## Using FILTER() in CALCULATE()

Fungsi DAX `CALCULATE()` dan `CALCULATETABLE()` adalah fungsi yang penting dan berguna. Fungsi ini memungkinkan kita menulis kalkulasi yang menghapus atau menambahkan filter, atau memodifikasi jalur relasi pada tabel. Hal ini dilakukan dengan memasukkan argumen filter, yang bisa berupa ekspresi Boolean, ekspresi tabel, atau fungsi filter khusus.

DAX memiliki fungsi `FILTER()` yang berguna untuk memfilter tabel mengikuti kebutuhan kita. Namun, berbeda halnya ketika kita menggunakan fungsi `CALCULATE()` dimana fungsi ini sudah memiliki parameter filter untuk merubah *filter context*. Misal, ketika kita perlu untuk selalu menampilkan `[Total Sales]` hanya pada BPJS dan Insurance maka kita menuliskan fungsi DAX berikut:

```
Total Sales BPJS & Insurance (Plan Filter) =
CALCULATE(
    [Total Sales],
    Prescript[REG_AS] IN {"A", "I"}
)
```

| REG_AS | Total Sales | Total Sales BPJS & Insurance (Plan Filter) |
|---|---|---|
| | 1,803,562,698.63 | 858,370,974.64 |
| A | 4,947,474.40 | 858,370,974.64 |
| I | 853,423,500.24 | 858,370,974.64 |
| R | 5,858,917,513.48 | 858,370,974.64 |
| **Total** | **8,520,851,186.75** | **858,370,974.64** |

Hasilnya adalah seperti yang sudah kita ketahui sebelumnya. Namun, uniknya adalah hal yang berbeda ketika kita menggunakan fungsi `FILTER()` sebagai parameter ***<filter>*** untuk `CALCULATE()` tersebut.

```
Total Sales BPJS & Insurance (Plan Filter) =
CALCULATE(
    [Total Sales],
    FILTER(
        Prescript,
        Prescript[REG_AS] IN {"A", "I"}
    )
)
```

| REG_AS | Total Sales | Total Sales BPJS & Insurance (Plan Filter) | Total Sales BPJS & Insurance (FILTER) |
|---|---|---|---|
| | 1,803,562,698.63 | 858,370,974.64 | |
| A | 4,947,474.40 | 858,370,974.64 | 4,947,474.40 |
| I | 853,423,500.24 | 858,370,974.64 | 853,423,500.24 |
| R | 5,858,917,513.48 | 858,370,974.64 | |
| **Total** | **8,520,851,186.75** | **858,370,974.64** | **858,370,974.64** |

Hasilnya mirip dengan ketika kita menggunakan fungsi `KEEPFILTERS()` sebagai parameter ***<filter>*** untuk `CALCULATE()`.

```
Total Sales BPJS & Insurance (KEEPFILTERS) =
CALCULATE(
    [Total Sales],
    KEEPFILTERS(
        Prescript[REG_AS] in {"A", "I"}
    )
)
```

| REG_AS | Total Sales | Total Sales BPJS & Insurance (Plan Filter) | Total Sales BPJS & Insurance (FILTER) | Total Sales BPJS & Insurance (KEEPFILTERS) |
|---|---|---|---|---|
| | 1,803,562,698.63 | 858,370,974.64 | | |
| A | 4,947,474.40 | 858,370,974.64 | 4,947,474.40 | 4,947,474.40 |
| I | 853,423,500.24 | 858,370,974.64 | 853,423,500.24 | 853,423,500.24 |
| R | 5,858,917,513.48 | 858,370,974.64 | | |
| **Total** | **8,520,851,186.75** | **858,370,974.64** | **858,370,974.64** | **858,370,974.64** |

Fungsi FILTER() memiliki struktur sebagai berikut:
**`FILTER(<table>,<filter>)`**

Fungsi ini membutuhkan parameter ***<table>*** sebagai syarat menggunakannya. Selain itu, fungsi `FILTER()` akan mengembalikan sebuah tabel utuh dari ***<table>*** yang sudah di filter berdasarkan ***<filter>*** yang diberikan. Hal ini akan memakan memori lebih besar dalam proses perhitungan terutama ketika kita bekerja pada data yang besar.

Berbeda halnya dengan `KEEPFILTERS()`, fungsi ini hanya memodifikasi *filter context* yang ada pada `CALCULATE()` dan `CALCULATETABLE()`. Fungsi ini tidak mengembalikan sebuah tabel yang difilter sehingga lebih efisien secara penggunaan memori ketika proses perhitungan dan evaluasi dilakukan.

Penting untuk diperhatikan bahwa kita disarankan untuk menggunakan fungsi `KEEPFILTERS()` daripada `FILTER()` saat bekerja dengan `CALCULATE()` ataupun `CALCULATETABLE()` untuk efisiensi dan efektifitas penggunaan memori ketika proses perhitungan.

Namun, adakalanya kita tetap memerlukan fungsi `FILTER()` di dalam `CALCULATE()`. Misal, kita hanya ingin menghitung total penjualan yang nilainya lebih dari 500 ribu. Dalam kasus ini kita akan menggunakan *measure* `[Total Sales]` sebagai argumen filter.

```
Total Sales > 500000 (FILTER) =
CALCULATE(
    [Total Sales],
    FILTER(
        Prescript,
        [Total Sales] > 500000
    )
)
```

| REG_AS | Total Sales | Total Sales > 500000 (FILTER) |
|---|---|---|
| | 1,803,562,698.63 | |
| A | 4,947,474.40 | 1,979,948.00 |
| I | 853,423,500.24 | 307,392,313.64 |
| R | 5,858,917,513.48 | 3,081,127,820.43 |
| **Total** | **8,520,851,186.75** | **3,390,500,082.06** |

Kita menggunakan fungsi `FILTER()` untuk memfilter *measure* `[Total Sales]`. Namun, hal yang berbeda ketika mencoba menggunakan `KEEPFILTERS()` untuk memfilter *measure* seperti berikut:

```
1 Total Sales > 500000 (KEEPFILTERS) =
2 CALCULATE(
3     [Total Sales],
4     KEEPFILTERS(
5         [Total Sales] > 500000
6     )
7 )
```

A function 'PLACEHOLDER' has been used in a True/False expression that is used as a table filter expression. This is not allowed.

Kita justru akan mendapatkan *error* pada rumus DAX yang ditulis. Hal ini terjadi karena *measure* tidak bisa dipakai sebagai argumen filter pada fungsi `KEEPFILTERS()` sehingga pada kondisi khusu seperti ini kita perlu menggunakan `FILTER()` di dalam `CALCULATE()` sebagai pengganti `KEEPFILTERS()`.

Untuk kinerja terbaik, disarankan kita menggunakan `KEEPFILTERS()` sebagai argumen filter di dalam `CALCULATE()`, kapan pun memungkinkan. Oleh karena itu, fungsi `FILTER()` sebaiknya hanya digunakan ketika diperlukan. Kita dapat menggunakannya untuk melakukan perbandingan kolom yang kompleks dalam filter seperti melibatkan Measures, penggunaan fungsi DAX `OR()`, atau operator logika OR (||).

# Optimizing DAX with Variables for Readability & Efficiency

Dalam penulisan DAX, penggunaan variabel dapat secara signifikan meningkatkan efisiensi dan keterbacaan kode. Dengan memanfaatkan variabel, kita dapat menyimpan hasil perhitungan sementara dan menggunakannya kembali dalam ekspresi yang sama, sehingga mengurangi pengulangan kalkulasi yang kompleks, tidak hanya meningkatkan kinerja model data di Power BI, tetapi juga membuat logika perhitungan lebih mudah dipahami dan dikelola.

`VAR` adalah ekspresi DAX yang digunakan untuk membuat sebuah Variabel yang dapat dipakai lebih dari satu kali di dalam DAX. Penggunaan `VAR` dapat meningkatkan *readability* dan performa kueri/ekspresi DAX yang kita buat.

Format Penulisan:

```
VAR name = expression
```

Contoh:

```
Rasio Profit Penjualan =
VAR total_jual =
    SUMX(
        Sales,
        Sales[QTY] * Sales[HJ]
    )

VAR total_modal =
    SUMX(
        Sales,
        Sales[QTY] * Sales[HNA]
    )

RETURN
    DIVIDE(
        total_jual - total_modal,
        total_jual
    )
```

DAX di atas membuat sebuah *measure* bernama `[Rasio Profit Penjualan]` untuk menghitung besar keuntungan dari penjualan. Variabel `total_jual` dan `total_modal` dibuat untuk memudahkan proses penghitungan profit. Membuat sebuah variabel seperti contoh di atas dapat menyederhanakan ekspresi yang kita buat. Cara ini juga dapat meningkatkan performa DAX karena proses menghitung atau evaluasi nilai `[total_jual]` hanya dilakukan satu kali meskipun nilai ini dipanggil sebanyak dua kali dalam ekspresi akhir DAX kita. `RETURN` digunakan

sebagai pernyataan untuk mengevaluasi penghitungan dan menggunakan variabel yang sudah kita definisikan.

Berikut ini beberapa hal yang perlu diperhatikan dalam memberikan nama pada sebuah variabel:

- Tidak boleh diawali dengan angka
- Tidak boleh terdapat spasi
- Tidak boleh mengandung karakter spesial (kecuali *underscore* "_")
- Tidak boleh menggunakan nama tabel yang sudah ada pada *data model*
- Tidak boleh menimpa nama variabel pada ekspresi yang sama

Variabel di DAX bersifat lokal, artinya variabel tersebut hanya dapat diakses di dalam kueri/ekspresi DAX tempat variabel tersebut didefinisikan. Sebuah variabel juga dapat mengakses variabel lain yang sudah didefinisikan sebelumnya seperti pada contoh DAX berikut:

```
Rasio Profit Penjualan =
VAR total_jual =
    SUMX(
        Sales,
        Sales[QTY] * Sales[HJ]
    )

VAR total_modal =
    SUMX(
        Sales,
        Sales[QTY] * Sales[HNA]
    )

VAR profit_jual =
    total_jual - total_modal

RETURN
    DIVIDE(
        profit_jual,
        total_jual
    )
```

Variabel akan "dibekukan" begitu didefinisikan sehingga tidak dapat diubah *context*-nya bahkan dengan menggunakan fungsi `CALCULATE()` sekalipun. DAX di bawah ini merupakan contoh ekspresi DAX yang berusaha mengubah *context* sebuah variabel dengan fungsi `CALCULATE()`. Variabel `var1` didefinisikan agar menghitung `[Total Transactions]` untuk BPJS ("`A`"). Kemudian, filternya diubah menggunakan `CALCULATE()` pada *statement* `RETURN` untuk Insurance ("`I`").

```
Total Transactions (Var Freezed Demo) =
VAR var1 =
    CALCULATE(
        [Total Transactions],
        Prescript[REG_AS] = "A"
    )

RETURN
    CALCULATE(
        total_transactions_bpjs,
        Prescript[REG_AS] = "I"
    )
```

Hasil akhir dapat dilihat pada gambar di bawah ini dimana nilai `[Total Transactions]` tetap untuk BPJS (“A”) sebagaimana didefinisikan di dalam variabel `var1`. Tidak ada yang dapat memodifikasi sebuah variabel bahkan `CALCULATE()` sekalipun kecuali mengganti langsung variabel tersebut.

| REG_AS | Total Transactions | Total Transactions (Var Freezed Demo) |
|---|---|---|
| A | 90 | 90 |
| I | 24116 | 90 |
| R | 102858 | 90 |
| **Total** | **127064** | **90** |

## Advanced Scenario for `CALCULATE()` with Filter Modifier

Dalam analisis data, Pareto Chart sering digunakan untuk mengidentifikasi faktor-faktor utama yang berkontribusi terhadap sebagian besar hasil, sesuai dengan prinsip Pareto 80/20. Membuat Pareto Chart di Power BI melibatkan teknik tingkat lanjut, termasuk penggunaan fungsi `CALCULATE()` yang dikombinasikan dengan filter modifier untuk menghitung persentase kumulatif dari total kontribusi. Pendekatan ini memungkinkan kita memprioritaskan faktor-faktor penting, seperti produk atau kategori, yang berperan signifikan dalam pencapaian target bisnis.

Misal, kita memiliki nama-nama produk dengan total penjualan untuk setiap produknya yang sudah diurutkan dari `[Total Sales]` terbesar seperti berikut:

| KD_OBAT | Total Sales |
|---|---|
| AI-0618 | 1,740,193,497.00 |
| AI-0165 | 695,495,765.09 |
| AI-1325 | 336,258,282.12 |
| AI-1311 | 192,637,826.66 |
| AI-0164 | 189,073,422.79 |
| AI-0634 | 184,524,540.65 |
| AI-0031 | 144,131,189.00 |
| AI-0821 | 118,677,025.50 |
| AI-1338 | 112,252,051.92 |
| R-0701 | 98,752,576.00 |
| AI-0982 | 97,927,053.21 |
| AI-1324 | 96,973,197.72 |
| AI-0903 | 96,193,809.83 |
| AI-1187 | 81,693,908.52 |
| AI-0812 | 80,730,292.75 |
| R-2340 | 79,664,842.29 |
| **Total** | **8,520,851,186.75** |

Selanjutnya, kita akan membuat *measure* yang menghitung total kumulatif (*running total*) namun berdasarkan urutan `[Total Sales]` dari yang terbesar menggunakan DAX seperti berikut:

```
Total Sales Cumulative by Value =
SUMX(
    FILTER(
        SUMMARIZE(
            ALL('Sales'),
            Sales[KD_OBAT],
            "Total Sales per Product",
            SUMX(
                Sales,
                Sales[QTY] * Sales[HJ]
            )
        ),
        [Total Sales per Product] >=
        SUMX(
            Sales,
            Sales[QTY] * Sales[HJ]
        )
    ),
    [Total Sales per Product]
)
```

Kemudian, kita masukkan `[Total Sales Cumulative by Value]` ke dalam Tabel Visual untuk memverifikasi hasil DAX yang dibuat:

| KD_OBAT | Total Sales | Total Sales Cumulative by Value |
|---|---|---|
| AI-0618 | 1,740,193,497.00 | 1,740,193,497.00 |
| AI-0165 | 695,495,765.09 | 2,435,689,262.09 |
| AI-1325 | 336,258,282.12 | 2,771,947,544.21 |
| AI-1311 | 192,637,826.66 | 2,964,585,370.87 |
| AI-0164 | 189,073,422.79 | 3,153,658,793.66 |
| AI-0634 | 184,524,540.65 | 3,338,183,334.31 |
| AI-0031 | 144,131,189.00 | 3,482,314,523.31 |
| AI-0821 | 118,677,025.50 | 3,600,991,548.81 |
| AI-1338 | 112,252,051.92 | 3,713,243,600.73 |
| R-0701 | 98,752,576.00 | 3,811,996,176.73 |
| AI-0982 | 97,927,053.21 | 3,909,923,229.93 |
| AI-1324 | 96,973,197.72 | 4,006,896,427.65 |
| AI-0903 | 96,193,809.83 | 4,103,090,237.48 |
| AI-1187 | 81,693,908.52 | 4,184,784,146.00 |
| AI-0812 | 80,730,292.75 | 4,265,514,438.75 |
| R-2340 | 79,664,842.29 | 4,345,179,281.04 |
| **Total** | **8,520,851,186.75** | |

Kemudian, kita buat *measure* untuk menghitung persentase `[Total Sales Cumulative by Value]` terhadap keseluruhan `[Total Sales]` menggunakan DAX berikut:

```
Total Sales Cumulative % =
VAR all_sales =
    CALCULATE(
        [Total Sales],
        ALL(Sales)
    )

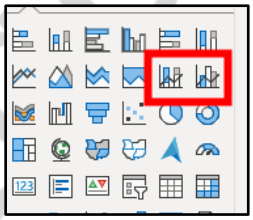
RETURN
    DIVIDE([Total Sales Cumulative by Value], All_sales)
```

Masukkan `[Total Sales Cumulative %]` ke dalam Tabel Visual untuk memverifikasi hasil DAX yang dibuat:

| KD_OBAT | Total Sales | Total Sales Cumulative by Value | Total Sales Cumulative % |
|---|---|---|---|
| AI-0618 | 1,740,193,497.00 | 1,740,193,497.00 | 20.42% |
| AI-0165 | 695,495,765.09 | 2,435,689,262.09 | 28.59% |
| AI-1325 | 336,258,282.12 | 2,771,947,544.21 | 32.53% |
| AI-1311 | 192,637,826.66 | 2,964,585,370.87 | 34.79% |
| AI-0164 | 189,073,422.79 | 3,153,658,793.66 | 37.01% |
| AI-0634 | 184,524,540.65 | 3,338,183,334.31 | 39.18% |
| AI-0031 | 144,131,189.00 | 3,482,314,523.31 | 40.87% |
| AI-0821 | 118,677,025.50 | 3,600,991,548.81 | 42.26% |
| AI-1338 | 112,252,051.92 | 3,713,243,600.73 | 43.58% |
| R-0701 | 98,752,576.00 | 3,811,996,176.73 | 44.74% |
| AI-0982 | 97,927,053.21 | 3,909,923,229.93 | 45.89% |
| AI-1324 | 96,973,197.72 | 4,006,896,427.65 | 47.02% |
| AI-0903 | 96,193,809.83 | 4,103,090,237.48 | 48.15% |
| AI-1187 | 81,693,908.52 | 4,184,784,146.00 | 49.11% |
| AI-0812 | 80,730,292.75 | 4,265,514,438.75 | 50.06% |
| R-2340 | 79,664,842.29 | 4,345,179,281.04 | 50.99% |
| **Total** | **8,520,851,186.75** | | |

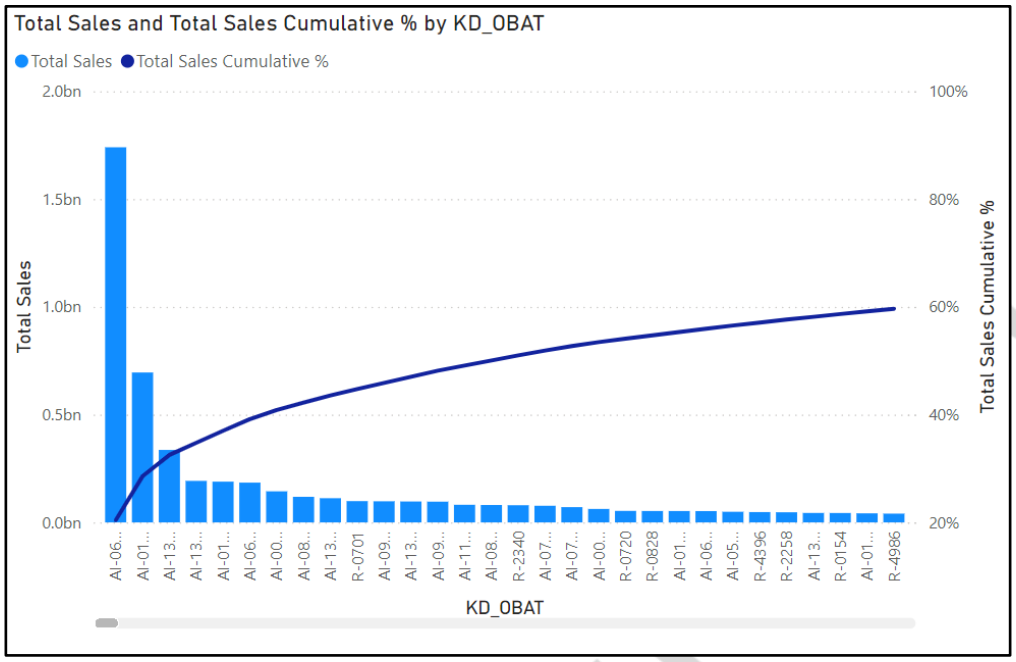
Sampai di sini kita sudah berhasil mempersiapkan metrik yang dibutuhkan untuk membuat Pareto Chart. Langkah terakhir adalah memasukkan metrik tersebut ke dalam bentuk visualisasi Pareto Chart.

Pilih salah satu dari Combo Chart (***Line and Stacked Column Chart*** atau ***Line and Clustered Column Chart***)



Masukkan:

- Kolom “KD_OBAT” dari tabel Sales ke X-Axis
- `[Total Sales]` ke Column Y-Axis
- `[Total Sales Cumulative %]` ke Line Y-Axis




# Time Intelligence Calculations in DAX

## The Importance of Time Intelligence in DAX

Time Intelligence di DAX mengacu pada sekelompok fungsi yang memungkinkan kita melakukan analisis dan perhitungan berdasarkan dimensi waktu, seperti bulan, kuartal, tahun, dan periode khusus lainnya. Fungsi-fungsi ini sangat penting dalam analisis data bisnis karena memberikan kemampuan untuk menghitung metrik berdasarkan waktu, seperti penjualan Year-to-Date (YTD), perbandingan dengan periode sebelumnya, atau tren dari bulan ke bulan.

Dengan Time Intelligence, kita bisa menjawab pertanyaan seperti:

- Bagaimana kinerja bisnis pada bulan ini dibandingkan bulan yang sama tahun lalu?
- Berapa total pendapatan hingga kuartal ini (Quarter-to-Date)?
- Bagaimana tren rata-rata penjualan selama 6 bulan terakhir?

Time Intelligence menjadi krusial dalam berbagai skenario bisnis untuk menganalisis perubahan kinerja dari waktu ke waktu, melihat pola musiman, dan mendapatkan wawasan mengenai tren historis. Dalam analisis bisnis, data tidak hanya dilihat dalam konteks statis tetapi juga berdasarkan waktu.

Beberapa contoh kasus bisnis yang menggunakan Time Intelligence adalah:

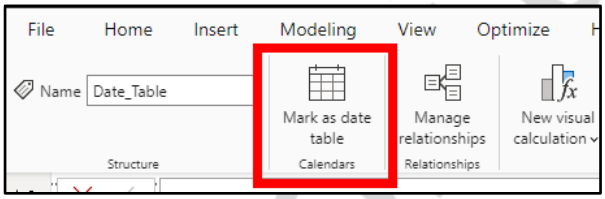
- Laporan keuangan bulanan dan tahunan.
- Analisis tren penjualan untuk menentukan pola musiman.
- Evaluasi kinerja marketing dari periode kampanye sebelumnya.

Agar fungsi Time Intelligence dapat bekerja dengan benar di DAX, kita perlu memiliki Date Table yang valid di model data kita. Date Table adalah tabel yang mencakup setiap hari dalam rentang tanggal yang relevan untuk analisis, dan harus ditandai secara khusus sebagai Date Table di Power BI.

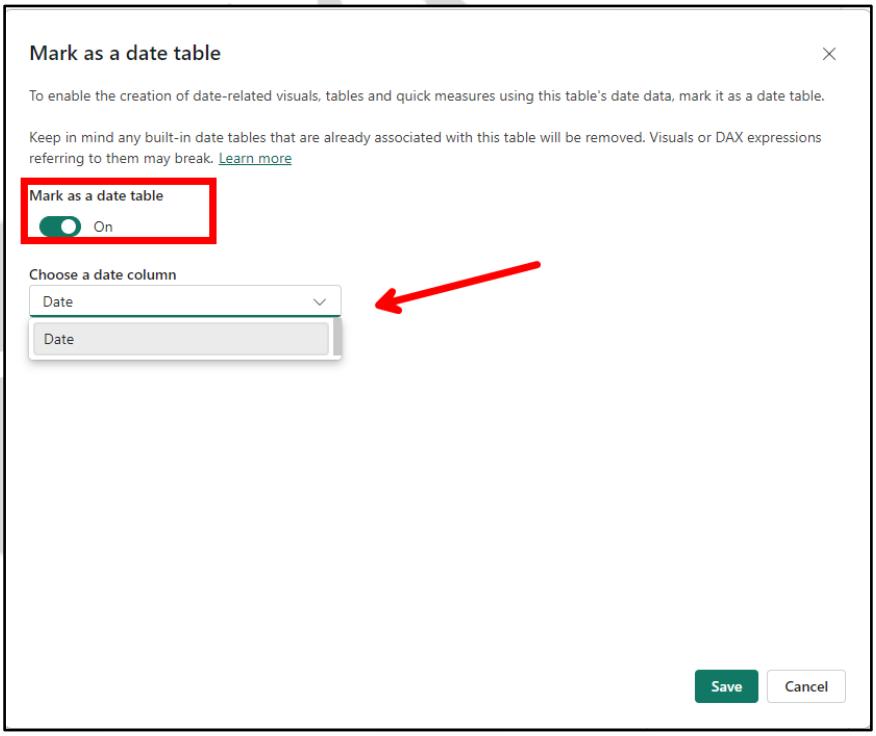
Di tahap awal, kita sudah membuat Date Table untuk model data yang kita gunakan yang dilengkapi dengan kolom Tahun, Bula, hingga Kuartal. Selanjutnya, yang perlu kita lakukan adalah menandai tabel tersebut sebagai Date Table di Power BI. Hal ini akan memudahkan kita dalam menggunakan rumus Time Intelligence di DAX dan juga termasuk ketika nantinya kita ingin menggunakan Quick Measure.

Untuk menandai Date_Table yang kita buat sebagai Date Table di Power BI, kita perlu lakukan:

1. Pastikan tabel Date_Table sudah dipilih sehingga akan muncul Ribbon **Table Tools**
2. Pergi ke Ribbon **Table Tools**
3. Pilih menu **Mark as Date Table**




4. Aktifkan “Mark as a date table”.
5. Lalu pilih kolom yang ditandai sebagai kolom tanggal.




Beberapa fungsi penting dalam Time Intelligence:

- `TOTALYTD()`, `TOTALQTD()`, `TOTALMTD()` – Menghitung akumulasi (*cumulative total*) berdasarkan periode waktu seperti Year-to-Date (YTD), Quarter-to-Date (QTD), dan Month-to-Date (MTD).
- `PREVIOUSMONTH()`, `PREVIOUSYEAR()`, `PREVIOUSQUARTER()` – Mengambil data dari periode sebelumnya.
- `SAMEPERIODLASTYEAR()` – Membandingkan periode waktu saat ini dengan periode yang sama di tahun sebelumnya.
- `DATEADD()` – Menggeser tanggal dalam model berdasarkan periode tertentu (misalnya, menambahkan atau mengurangi hari, bulan, atau tahun).
- `PARALLELPERIOD()` – Mengambil data dari periode waktu paralel (misal, perbandingan antara satu tahun yang lalu dan tahun ini).

## Calculating Previous Period in DAX Time Intelligence

Dalam analisis bisnis, seringkali kita perlu membandingkan kinerja dari waktu ke waktu. Salah satu cara umum adalah dengan melihat nilai dari periode waktu sebelumnya, seperti bulan, kuartal, atau tahun sebelumnya. DAX menyediakan beberapa fungsi Time Intelligence yang dirancang khusus untuk tujuan ini:

- `PREVIOUSMONTH()`: Mengambil data dari bulan sebelumnya.
- `PREVIOUSQUARTER()`: Mengambil data dari kuartal sebelumnya.
- `PREVIOUSYEAR()`: Mengambil data dari tahun sebelumnya.

Fungsi-fungsi ini memungkinkan kita untuk dengan mudah menghitung nilai dari periode waktu tertentu sebelum periode yang sedang kita analisis. Perbandingan antara periode saat ini dan periode sebelumnya adalah metrik yang penting dalam banyak skenario bisnis, seperti:

- Melacak perubahan pendapatan atau penjualan bulanan.
- Membandingkan pertumbuhan penjualan dari kuartal ini dengan kuartal sebelumnya.
- Menganalisis kinerja tahunan dalam laporan keuangan.

Dengan fungsi-fungsi `PREVIOUSMONTH()`, `PREVIOUSQUARTER()`, dan `PREVIOUSYEAR()`, kita bisa dengan cepat membuat perhitungan yang membandingkan metrik dari waktu ke waktu untuk mendapatkan wawasan tren dan pola musiman. Ketiganya memiliki struktur fungsi yang sama yaitu:

**PREVIOUSMONTH(*<dates>*)**
**PREVIOUSQUARTER(*<dates>*)**
**PREVIOUSYEAR(*<dates>*)**

Misal, kita ingin membuat laporan yang membandingkan banyaknya transaksi bulan ini dengan bulan sebelumnya. Misalkan kita sudah memiliki *measure* untuk menghitung total transaksi menggunakan DAX berikut:

```
Total Transactions = DISTINCTCOUNT(Prescript[NO_RESEP])
```

| Bulan | Total Transactions |
|---|---|
| JAN | 7329 |
| FEB | 9592 |
| MAR | 10283 |
| APR | 10746 |
| MAY | 9858 |
| JUN | 11515 |
| JUL | 9666 |
| AUG | 11206 |
| SEP | 11797 |
| OCT | 11964 |
| NOV | 11259 |
| DEC | 11849 |
| **Total** | **127064** |

Untuk menghitung penjualan di bulan sebelumnya, kita bisa menggunakan `PREVIOUSMONTH()` dalam kombinasi dengan `CALCULATE()` untuk merubah *filter context* pada baris yang dievaluasi menjadi tanggal yang sama di bulan sebelumnya.

```
Total Transactions PREVIOUSMONTH =
CALCULATE(
    [Total Transactions],
    PREVIOUSMONTH(Date_Table[Date])
)
```

| Bulan | Total Transactions | Total Transactions PREVIOUSMONTH |
|---|---|---|
| JAN | 7329 | |
| FEB | 9592 | 7329 |
| MAR | 10283 | 9592 |
| APR | 10746 | 10283 |
| MAY | 9858 | 10746 |
| JUN | 11515 | 9858 |
| JUL | 9666 | 11515 |
| AUG | 11206 | 9666 |
| SEP | 11797 | 11206 |
| OCT | 11964 | 11797 |
| NOV | 11259 | 11964 |
| **Total** | **127064** | |

Dalam contoh ini, fungsi `PREVIOUSMONTH()` mengambil tanggal dari bulan sebelumnya, dan `CALCULATE()` digunakan untuk menghitung total penjualan selama bulan tersebut. Proses ini serupa untuk `PREVIOUSQUARTER()` dan `PREVIOUSYEAR()`, yang bekerja dengan prinsip yang sama, tetapi dengan cakupan waktu yang berbeda.

## Year-to-Date, Quarter-to-Date, and Month-to-Date

Perhitungan akumulatif seperti Year-to-Date (YTD), Quarter-to-Date (QTD), dan Month-to-Date (MTD) adalah komponen penting dari analisis Time Intelligence di DAX. Fungsi-fungsi ini membantu kita menghitung total kinerja kumulatif hingga titik waktu tertentu dalam tahun, kuartal, atau bulan yang sedang berjalan.

Perhitungan ini sangat umum dalam dunia bisnis, terutama dalam pelaporan keuangan dan analisis performa penjualan. Metrik ini memungkinkan kita melacak akumulasi pendapatan, penjualan, atau metrik lainnya dari awal tahun, kuartal, atau bulan hingga tanggal tertentu.

DAX menyediakan fungsi-fungsi yang secara khusus dirancang untuk menghitung akumulasi waktu:

- `TOTALYTD()`: Menghitung total kumulatif dari awal tahun hingga tanggal tertentu.
- `TOTALQTD()`: Menghitung total kumulatif dari awal kuartal hingga tanggal tertentu.
- `TOTALMTD()`: Menghitung total kumulatif dari awal bulan hingga tanggal tertentu.

Ketiga fungsi ini bekerja dengan prinsip yang sama, tetapi mereka berfokus pada periode waktu yang berbeda: tahun, kuartal, dan bulan. Struktur fungsinya adalah:

**TOTALMTD(*<expression>,<dates>[,<filter>]*)**
**TOTALMTD(*<expression>,<dates>[,<filter>]*)**
**TOTALMTD(*<expression>,<dates>[,<filter>]*)**

Misal, kita ingin menghitung total transaksi Year-to-Date untuk bisnis kita. Untuk menghitung penjualan Year-to-Date, kita bisa menggunakan fungsi `TOTALYTD()` sebagai berikut:

```
Total Transactions YTD =
TOTALYTD(
    [Total Transactions],
    Date_Table[Date]
)
```

| Date | Total Transactions | Total Transactions YTD |
|---|---|---|
| 12/20/2015 | 157 | 123259 |
| 12/21/2015 | 135 | 123394 |
| 12/22/2015 | 461 | 123855 |
| 12/23/2015 | 442 | 124297 |
| 12/24/2015 | 464 | 124761 |
| 12/25/2015 | 167 | 124928 |
| 12/26/2015 | 273 | 125201 |
| 12/27/2015 | 196 | 125397 |
| 12/28/2015 | 165 | 125562 |
| 12/29/2015 | 599 | 126161 |
| 12/30/2015 | 473 | 126634 |
| 12/31/2015 | 430 | 127064 |
| **Total** | **127064** | **127064** |

Fungsi ini akan menghitung total penjualan dari awal tahun hingga tanggal terakhir di dalam tahun yang sama pada model data kita.

Demikian juga, kita bisa menghitung penjualan untuk kuartal-to-date. Berikut adalah contohnya menggunakan `TOTALQTD()`:

```
Total Transactions YTD =
TOTALYTD(
    [Total Transactions],
    Date_Table[Date]
)
```

| Bulan | Total Transactions | Total Transactions QTD |
|---|---|---|
| JAN | 7329 | 7329 |
| FEB | 9592 | 16921 |
| MAR | 10283 | 27204 |
| APR | 10746 | 10746 |
| MAY | 9858 | 20604 |
| JUN | 11515 | 32119 |
| JUL | 9666 | 9666 |
| AUG | 11206 | 20872 |
| SEP | 11797 | 32669 |
| OCT | 11964 | 11964 |
| NOV | 11259 | 23223 |
| DEC | 11849 | 35072 |
| **Total** | **127064** | **35072** |

Kita akan mendapatkan total penjualan kumulatif dari awal kuartal hingga tanggal terakhir di dalam kuartal yang sama pada model data. Fungsi YTD, QTD, dan MTD sangat bermanfaat dalam berbagai konteks bisnis, seperti:

- Pelaporan Keuangan: Pelaporan Year-to-Date sering digunakan dalam laporan keuangan untuk menunjukkan akumulasi pendapatan, laba, atau biaya sejak awal tahun fiskal.
- Analisis Kuartalan: Dengan `TOTALQTD()`, kita dapat melacak kinerja kumulatif dalam sebuah kuartal, yang berguna untuk analisis performa bisnis dalam kerangka kuartal.
- Analisis Bulanan: Perhitungan MTD membantu memahami akumulasi kinerja dalam bulan tertentu, sering kali digunakan untuk melacak target penjualan atau performa pemasaran bulanan.

Dengan fungsi `TOTALYTD()`, `TOTALQTD()`, dan `TOTALMTD()`, kita dapat membuat visualisasi dalam Power BI untuk memantau kinerja bisnis secara kumulatif dari waktu ke waktu. Contoh penggunaan praktisnya termasuk:

- Membandingkan penjualan Year-to-Date dengan target tahunan.
- Melacak kinerja kuartal dengan total QTD untuk melihat apakah bisnis berada di jalur yang benar untuk mencapai target kuartalan.

- Memonitor progres bulanan melalui perhitungan Month-to-Date yang memberikan gambaran tentang perkembangan penjualan atau pemasukan pada bulan yang sedang berjalan.

Perhitungan akumulatif memungkinkan bisnis untuk melihat performa kinerja secara progresif, bukan hanya snapshot statis di satu titik waktu. Dengan perhitungan YTD, QTD, dan MTD, kita dapat memahami bagaimana kinerja bisnis berjalan sejak awal periode waktu tertentu sehingga memudahkan analisis tren jangka panjang dan pengambilan keputusan berbasis data.

## Dynamic Shifting Date Calculations

Pada kasus-kasus sebelumnya, kita menggunakan Time Intelligence untuk mundur atau maju dalam interval waktu tetap seperti bulanan, kuartalan, dan tahunan. Selain itu, kita juga menghitung total kumulatif pada rentang yang tetap seperti bulanan, kuartalan, ataupun tahunan. Namun, adakalanya kita perlu untuk menggeser tanggal pada rentang waktu yang lebih dinamis dan tidak bergantung pada cakupan periode tertentu.

Misal, kita ingin melihat dan membandingkan banyaknya transaksi antara hari ini dengan banyaknya transaksi tepat di 7 hari sebelumnya. DAX menyediakan fungsi `DATEADD()` untuk keperluan tersebut dimana kita dapat menggeser *filter context* dari tanggal secara lebih dinamis dan mengikuti kebutuhan bisnis.

Struktur fungsi dari `DATEADD()` adalah sebagai berikut:
**`DATEADD(`*`<dates>,<number_of_intervals>,<interval>`*`)`**

Sehingga untuk menghitung banyaknya transaksi pada 7 hari sebelumnya kita bisa membuat *measure* menggunakan DAX berikut:

```
Total Transactions Prev-30-Days =
CALCULATE(
    [Total Transactions],
    DATEADD(
        Date_Table[Date],
        -7,
        DAY
    )
)
```

| Date | Total Transactions | Total Transactions Prev-30-Days |
|---|---|---|
| 01/01/2015 | 127 | |
| 01/02/2015 | 274 | |
| 01/03/2015 | 237 | |
| 01/04/2015 | 156 | |
| 01/05/2015 | 105 | |
| 01/06/2015 | 278 | |
| 01/07/2015 | 289 | |
| 01/08/2015 | 283 | 127 |
| 01/09/2015 | 268 | 274 |
| 01/10/2015 | 246 | 237 |
| 01/11/2015 | 120 | 156 |
| 01/12/2015 | 141 | 105 |
| 01/13/2015 | 258 | 278 |
| **Total** | **127064** | **124761** |

Fungsi DATEADD() yang kita gunakan akan mengganti *filter context* dari tanggal yang ada menjadi mundur 7 hari sebelumnya dimana ***<interval>*** yang digunakan adalah DAY sehingga proses penggeseran akan mengikuti tingkat detail harian.

Hal serupa bisa juga kita lakukan untuk melihat dan membandingkan banyaknya transaksi di tanggal yang sama pada bulan sebelumnya. Untuk keperluan tersebut kita bisa membuat *measure* menggunakan DAX berikut:

```
Total Transactions Prev-30-Days =
CALCULATE(
    [Total Transactions],
    DATEADD(
        Date_Table[Date],
        -1,
        MONTH
    )
)
```

| Date | Total Transactions | Total Transactions Same Date prev Month |
|---|---|---|
| 01/28/2015 | 298 | |
| 01/29/2015 | 328 | |
| 01/30/2015 | 330 | |
| 01/31/2015 | 176 | |
| 02/01/2015 | 179 | 127 |
| 02/02/2015 | 202 | 274 |
| 02/03/2015 | 500 | 237 |
| 02/04/2015 | 456 | 156 |
| 02/05/2015 | 441 | 105 |
| 02/06/2015 | 440 | 278 |
| 02/07/2015 | 392 | 289 |
| 02/08/2015 | 157 | 283 |
| 02/09/2015 | 167 | 268 |
| **Total** | **127064** | **115215** |

DAX akan menghitung nilai `[Total Transactions]` untuk tanggal yang sama 1 bulan sebelumnya. Penting untuk diketahui bahwa total hari dalam satu bulan itu berbeda-beda sehingga jika kita ingin membandingkan nilai pada tanggal yang sama di bulan sebelumnya, cara yang paling tepat adalah menggunakan `MONTH` bukan `DAY` untuk parameter ***<interval>***.

Kemudian, bagaimana dengan tanggal yang tidak ada di bulan sebelumnya? Contohnya, adalah tanggal 31 Maret, bulan sebelumnya yaitu Februari hanya memiliki tanggal hingga 28 Februari. Pada kasus seperti ini, DAX akan mengambil nilai dari tanggal maksimum di bulan sebelumnya.

| Date | Total Transactions | Total Transactions Same Date prev Month |
|---|---|---|
| 03/24/2015 | 486 | 421 |
| 03/25/2015 | 398 | 335 |
| 03/26/2015 | 382 | 422 |
| 03/27/2015 | 468 | 399 |
| 03/28/2015 | 475 | 460 |
| 03/29/2015 | 129 | 460 |
| 03/30/2015 | 150 | 460 |
| 03/31/2015 | 149 | 460 |
| 04/01/2015 | 310 | 132 |
| 04/02/2015 | 433 | 159 |
| 04/03/2015 | 462 | 432 |

Dapat dilihat dari gambar di atas bahwa untuk tanggal 28 hingga 31 Maret, DAX akan mengembalikan nilai total transaksi yang sama, yaitu untuk tanggal 28 Februari karena tanggal 28 adalah tanggal terakhir yang di bulan Februari.

Kemudian, perlu untuk kita perhatikan juga bahwa fungsi `DATEADD()` untuk mencari nilai pada bulan sebelumnya akan memberikan hasil yang berbeda jika dibandingkan dengan penggunaan fungsi `PREVIOUSMONTH()`.

| Date | Total Transactions | Total Transactions Same Date prev Month | Total Transactions PREVIOUSMONTH |
|---|---|---|---|
| 02/02/2015 | 202 | 274 | 7329 |
| 02/03/2015 | 500 | 237 | 7329 |
| 02/04/2015 | 456 | 156 | 7329 |
| 02/05/2015 | 441 | 105 | 7329 |
| 02/06/2015 | 440 | 278 | 7329 |
| 02/07/2015 | 392 | 289 | 7329 |
| 02/08/2015 | 157 | 283 | 7329 |
| 02/09/2015 | 167 | 268 | 7329 |
| 02/10/2015 | 455 | 246 | 7329 |
| 02/11/2015 | 384 | 120 | 7329 |
| 02/12/2015 | 376 | 141 | 7329 |
| 02/13/2015 | 412 | 258 | 7329 |
| 02/14/2015 | 317 | 139 | 7329 |
| **Total** | **127064** | **115215** | |

Fungsi `PREVIOUSMONTH()` akan mengembalikan nilai total dari suatu *measure* pada bulan sebelumnya. Sementara itu, fungsi `DATEADD()` akan mengembalikan nilai tunggal dari suatu *measure* di tanggal yang sama pada bulan sebelumnya, dengan catatan ia akan mengembalikan nilai dari tanggal terakhir pada bulan sebelumnya apabila tanggal yang dievaluasi tidak ada pada bulan sebelumnya.

Kasus selanjutnya yang sangat mungkin kita temui adalah ketika kita ingin menghitung nilai kumulatif dari tanggal yang dipilih sebanyak periode waktu tertentu. Misal, kita ingin menghitung total transaksi selama 15 hari terakhir berdasarkan tanggal yang kita pilih di Slicer. Pada kasus ini, kita memerlukan fungsi yang dapat merubah *filter context* seakan-akan tanggal yang dipilih adalah kumpulan 15 tanggal terakhir sehingga nilai *measure* yang digunakan akan melihat ke 15 tanggal tersebut.

DAX menyediakan fungsi `DATESINPERIOD()` dimana fungsi ini akan mengembalikan tabel yang berisi tanggal dari awal tanggal yang ditetapkan hingga banyaknya tanggal sesuai interval yang kita tentukan.

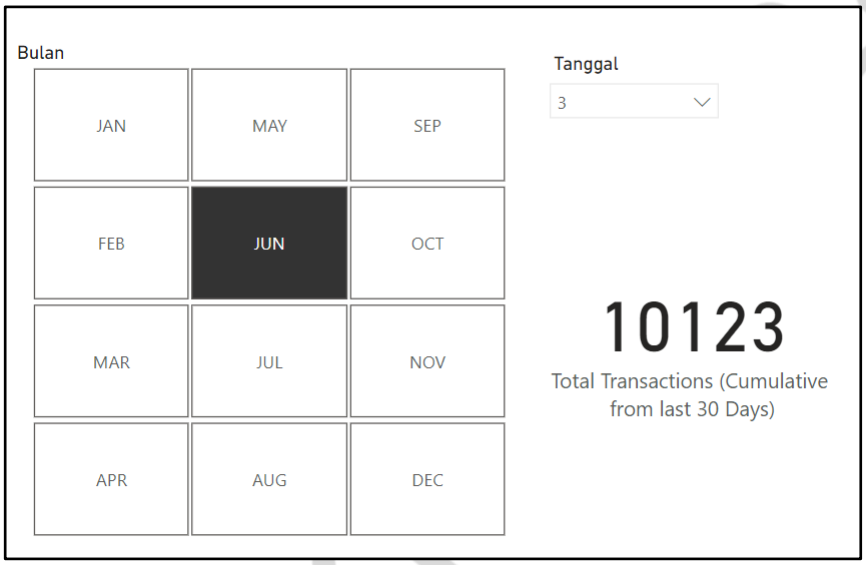
Struktur dari fungsi `DATESINPERIOD()` adalah sebagai berikut:
**`DATESINPERIOD(`*`<dates>, <start_date>, <number_of_intervals>, <interval>`*`)`**

Di sini kita akan menggunakan visual Card untuk *measure* yang dibuat dengan DAX berikut:

```
Total Transactions (Cumulative from last 30 Days) =
CALCULATE(
    [Total Transactions],
    DATESINPERIOD(
        Date_Table[Date],
        SELECTEDVALUE(Date_Table[Date]),
        -30, DAY
    )
```

```
)
```

Fungsi `DATESINPERIOD()` akan merubah *filter context* dari tanggal yang dipilih yang didefinisikan oleh fungsi `SELECTEDVALUE()`. Fungsi `SELECTEDVALUE()` akan mengembalikan nilai Tanggal yang dipilih dari Slicer yang kita gunakan. Kemudian, fungsi `DATESINPERIOD()` akan mengembalikan daftar tanggal yang merentang dari tanggal yang kita pilih hingga ke 30 hari sebelumnya. `CALCULATE()` akan mengevaluasi `[Total Transactions]` untuk 30 tanggal yang dihasilkan dari fungsi `DATESINPERIOD()`.



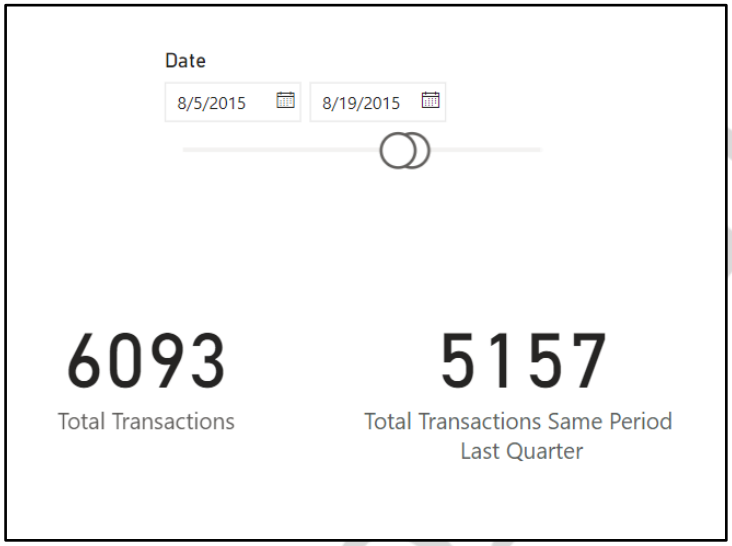

Pada contoh di atas, kita memilih tanggal 3 Juni 2015 maka `[Total Transactions (Cumulative from last 30 Days)]` akan mencari total transaksi dari tanggal 5 Mei 2015 hingga tanggal tersebut.

Kasus selanjutnya yang akan sering kita temui adalah membandingkan total nilai dari rentang tanggal saat ini terhadap rentang tanggal yang sama di bulan, kuartal, atau tahun sebelumnya. Cara ini sering digunakan untuk membandingkan performa rentang saat ini terhadap rentang yang sama di periode sebelumnya.

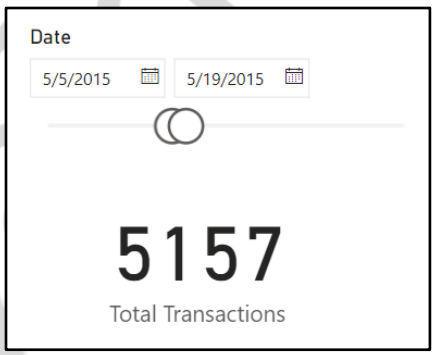
Misal, kita memilih rentang tanggal pada Slicer dari 5 Agustus 2015 hingga 19 Agustus 2015. Kemudian, kita ingin membandingkan total transaksi pada rentang tanggal yang sama di kuartal sebelumnya. Artinya, kita ingin membandingkannya dengan total transaksi pada rentang tanggal 5 Mei 2015 hingga 19 Mei 2015. Untuk mendapatkan hasil tersebut kita bisa membuat *measure* dengan DAX seperti berikut:

```
Total Transactions Same Period Last Quarter =
CALCULATE(
    [Total Transactions],
    DATEADD(
```

```
        Date_Table[Date],
        -1,
        QUARTER
    )
)
```




Kita dapat validasi hasil perhitungannya dengan melihat nilai total transaksi pada tanggal 5 Mei 2015 hingga 19 Mei 2015:




Namun, adakalanya juga kita memerlukan rentang tanggal yang spesifik seperti ketika perusahaan mengadakan suatu *campaign* atau *event* tertentu. `DATESBETWEEN()` adalah fungsi DAX yang digunakan untuk mengembalikan rentang tanggal antara dua tanggal yang spesifik. Fungsi ini memungkinkan pengguna untuk secara eksplisit menentukan tanggal awal dan akhir dalam analisis data.

Struktur dari fungsi ini adalah:
**`DATESBETWEEN(`*`<Dates>, <StartDate>, <EndDate>`*`)`**

Misal, kita menjalankan *campaign* besar selama periode 25 November hingga 2 Desember untuk memanfaatkan momentum Black Friday dan Cyber Monday. Kemudian, kita juga memiliki *event*

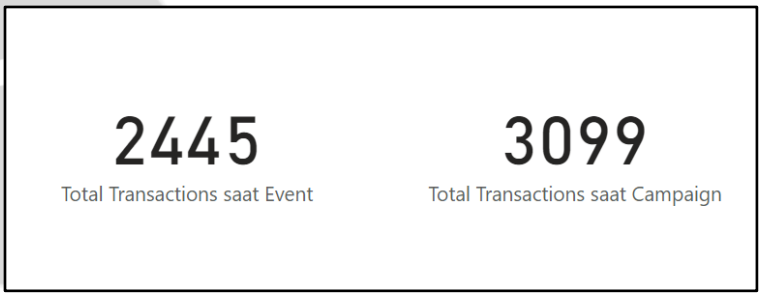
besar spesial ulang tahun perusahaan yang berlangsung dari tanggal 20 Juli 2015 hingga 27 Juli 2015. Kita ingin membandingkan performa banyaknya transaksi yang terjadi antara *event* dan *campaign*.

Pertama-tama kita bisa membuat *measure* untuk menghitung total transaksi pada saat *event* dengan DAX berikut:

```
Total Transactions saat Event =
CALCULATE(
    [Total Transactions],
    DATESBETWEEN(
        Date_Table[Date],
        DATE(2015, 7, 20),
        DATE(2015, 7, 27)
    )
)
```

Lalu membuat *measure* untuk menghitung total transaksi pada saat *campaign* berlangsung dengan DAX berikut:

```
Total Transactions saat Campaign =
CALCULATE(
    [Total Transactions],
    DATESBETWEEN(
        Date_Table[Date],
        DATE(2015, 11, 25),
        DATE(2015, 12, 2)
    )
)
```




Sampai di sini kita sudah berhasil melihat perbandingan antara total transaksi saat *event* dan saat *campaign*. Selanjutnya, kedua *measure* ini bisa kita manfaatkan untuk membuat *measure* lain misalkan menghitung selisihnya, atau rasio perbedaannya.

# Quick Measure and Data Analysis

## Overview of Quick Measure

Quick Measure adalah salah satu fitur di Power BI yang memungkinkan pengguna untuk membuat kalkulasi atau metrik (*measure*) secara otomatis tanpa perlu menulis rumus DAX secara manual. Fitur ini sangat bermanfaat bagi pengguna yang tidak memiliki pengetahuan mendalam tentang DAX namun tetap ingin melakukan analisis data yang lebih kompleks.

Quick Measure adalah alat bantu otomatis yang mempermudah pengguna Power BI dalam membuat perhitungan yang umum digunakan, seperti penjumlahan, rata-rata tertimbang, rasio, hingga year-to-date (YTD), tanpa harus menulis kode DAX sendiri. Ini dilakukan dengan memilih beberapa opsi melalui antarmuka Power BI, dan Power BI akan secara otomatis menghasilkan DAX yang diperlukan.

Manfaat menggunakan Quick Measure**:**

- **Kemudahan Penggunaan**: Quick Measure menyediakan antarmuka yang ramah pengguna, memungkinkan pengguna untuk membuat perhitungan kompleks tanpa perlu memahami sintaks DAX yang mendetail.
- **Efisiensi Waktu**: Karena Power BI secara otomatis menghasilkan rumus DAX, pengguna dapat menghemat waktu saat membuat perhitungan dan analisis.
- **Belajar DAX**: Quick Measure juga berguna bagi mereka yang ingin belajar DAX. Dengan melihat hasil rumus yang dihasilkan oleh Quick Measure, pengguna bisa mulai memahami bagaimana sintaks DAX bekerja.

Meskipun Quick Measure memudahkan pembuatan perhitungan, fleksibilitasnya lebih terbatas dibandingkan menulis DAX secara manual. Perhitungan yang lebih kompleks dan kustom mungkin memerlukan penulisan rumus DAX manual. Selain itu, Quick Measure tidak mencakup semua skenario yang mungkin ditemui dalam analisis data. Jika analisis memerlukan logika yang lebih rumit, penulisan DAX secara manual tetap dibutuhkan.

Quick Measure adalah fitur yang sangat bermanfaat di Power BI, terutama bagi mereka yang baru mulai menggunakan DAX atau membutuhkan perhitungan cepat. Dengan berbagai template perhitungan yang tersedia, Quick Measure dapat membantu mempercepat proses analisis data dan membuat DAX lebih mudah diakses bagi semua pengguna.

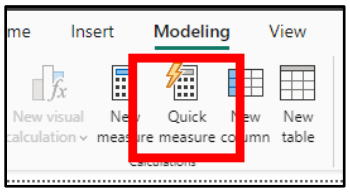
## Using Quick Measure

Misal, kita ingin memantau kinerja transaksi secara kumulatif dari waktu ke waktu. Kita perlu menghitung *running total* atau total kumulatif total transaksi harian, mingguan, atau bulanan untuk memahami bagaimana penjualan berkembang selama periode tertentu. Informasi ini dapat digunakan untuk mengevaluasi tren penjualan, mengidentifikasi puncak performa, atau

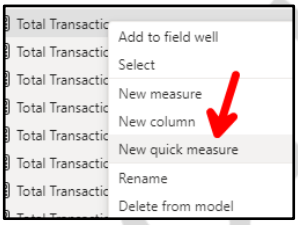
memproyeksikan pencapaian target penjualan ke depan. Dengan Quick Measure di Power BI, kita dapat dengan cepat menghitung *running total* penjualan tanpa harus menulis rumus DAX-nya.

Untuk menggunakan Quick Measure:

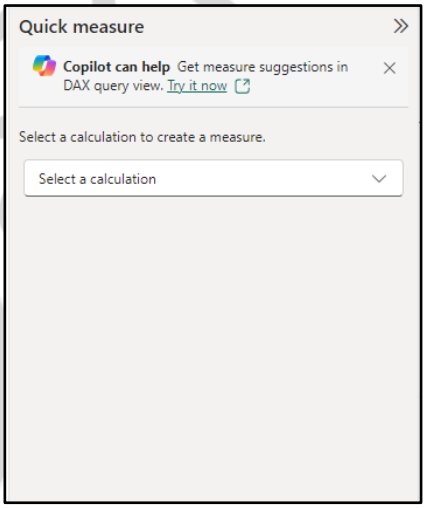
1. Pergi ke ribbon **Modeling**
2. Pilih menu **Quick Measure**



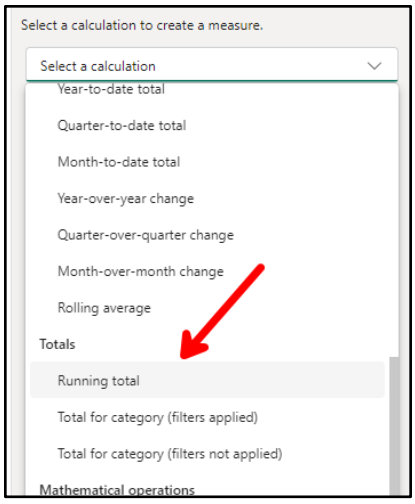

Atau, kita juga bisa langsung klik kanan di kolom atau *measure* yang ingin dibuat kalkulasi. Kemudian, pilih "**New Quick Measure**"



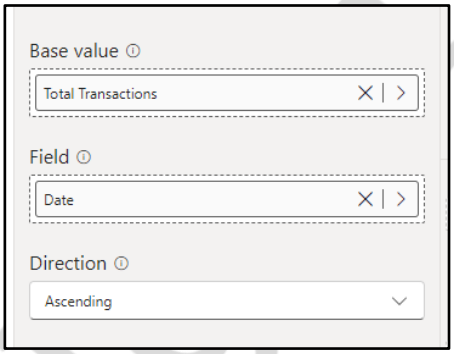

Sebuah panel Quick Measure akan muncul.



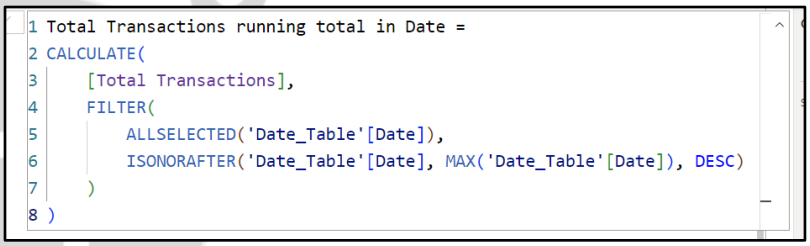

3. Pilih tipe kalkulasi yang kita butuhkan. Berbagai kebutuhan kalkulasi yang umum sudah disediakan oleh Power BI. Misal kita ingin mencari running total.




4. Konfigurasikan kalkulasinya. Pilih *measure* [Total Transactions] untuk *Base value* dan kolom tanggal “Date” dari tabel Date_Table untuk *Field*. Jika sudah maka klik “Add”.

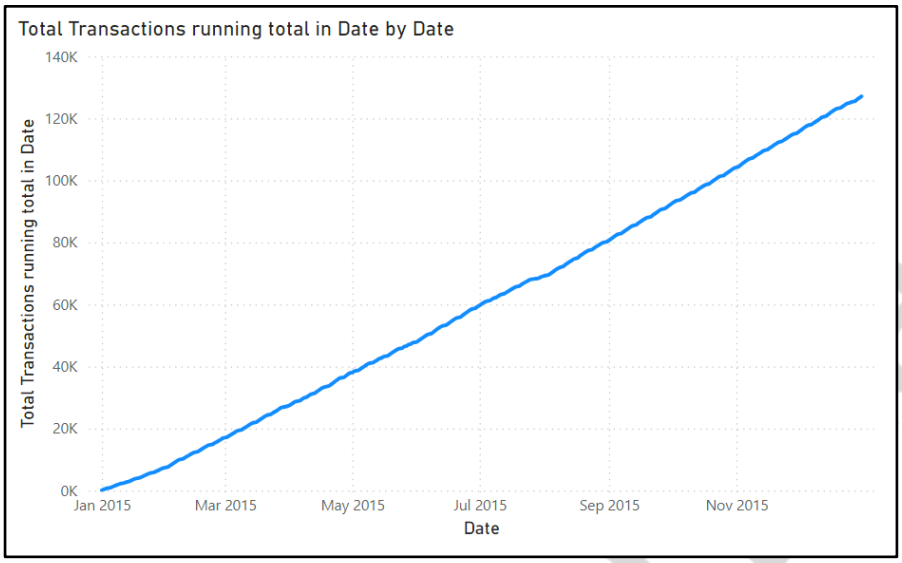



5. Setelah klik “Add”, Power BI akan membuatkan secara otomatis formula DAX yang dibutuhkan untuk melakukan kalkulasi tersebut. Kita bisa lihat formulanya di bagian atas kanvas.



```
1 Total Transactions running total in Date =
2 CALCULATE(
3     [Total Transactions],
4     FILTER(
5         ALLSELECTED('Date_Table'[Date]),
6         ISONORAFTER('Date_Table'[Date], MAX('Date_Table'[Date]), DESC)
7     )
8 )
```

Kita berhasil membuat sebuah kalkulasi running total. Kalkulasi ini dapat kita gunakan pada visualisasi seperti berikut:




# What-If Analytics in Power BI

## Introduction to Parameterization for What-If Analysis

What-If Analysis adalah salah satu fitur analitik penting dalam Power BI yang memungkinkan pengguna untuk mensimulasikan berbagai skenario bisnis berdasarkan variabel yang dapat diubah. Dengan What-If Analysis, kita dapat memprediksi bagaimana perubahan dalam parameter tertentu akan memengaruhi hasil akhir sehingga membantu dalam pengambilan keputusan yang lebih baik.

Salah satu komponen kunci dalam What-If Analysis di Power BI adalah parameter. Parameter adalah variabel yang dapat diubah-ubah untuk melihat dampaknya terhadap data dan visualisasi. Misalnya, parameter dapat digunakan untuk mengubah tingkat diskon, harga, atau volume penjualan, dan melihat bagaimana perubahan ini memengaruhi total pendapatan atau keuntungan.

Parameterisasi adalah proses mendefinisikan variabel atau parameter yang dapat disesuaikan oleh pengguna dalam What-If Analysis. Dengan parameter, kita dapat membuat skenario dinamis yang memengaruhi hasil analisis yang kemudian divisualisasikan dalam laporan atau dashboard Power BI.

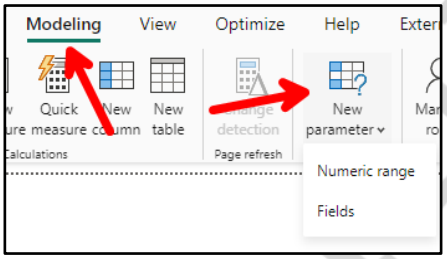
Dalam Power BI, parameter dapat diatur untuk memiliki nilai minimum, maksimum, dan langkah kenaikan. Dengan menggunakan slider atau kotak input, pengguna dapat dengan mudah menyesuaikan nilai parameter dan melihat perubahan secara langsung pada visualisasi data.

What-If Analysis memberikan berbagai manfaat bagi bisnis dan pengambilan keputusan, di antaranya:

- Pengambilan Keputusan yang Lebih Luas: Dengan mensimulasikan berbagai skenario, pengguna dapat memahami dampak potensial dari keputusan tertentu sebelum mengambil langkah strategis.
- Eksplorasi Skenario: Parameterisasi memungkinkan eksplorasi berbagai kemungkinan hasil dengan cepat, misalnya jika biaya produksi naik atau volume penjualan turun.
- Analisis Risiko: What-If Analysis juga dapat digunakan untuk memahami risiko yang terkait dengan keputusan tertentu dan bagaimana mitigasi dapat dilakukan.

## Creating and Managing Parameters

Parameter dapat dibuat dan dikonfigurasi melalui Ribbon **Modelling**. Kita akan menemukan menu **New Parameter**.

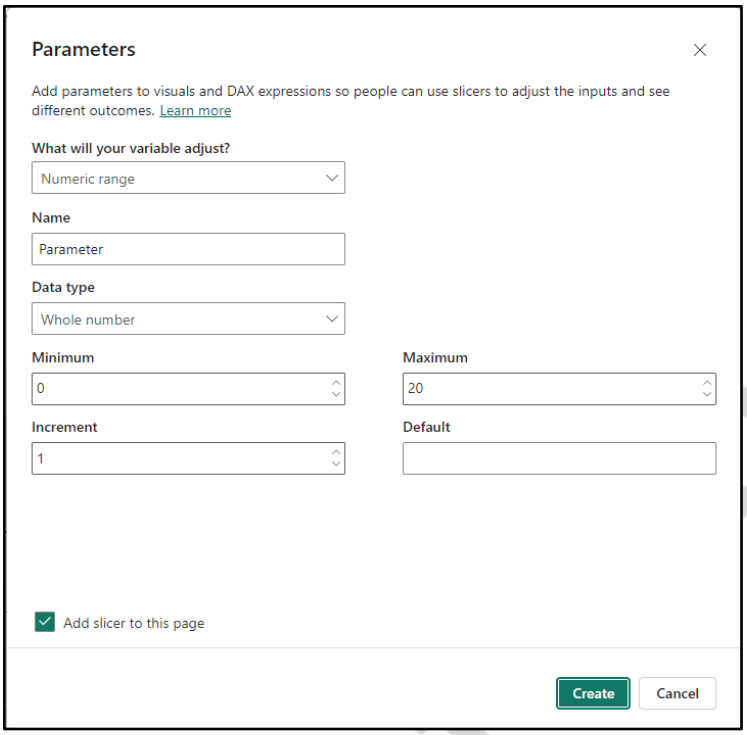



Ada dua jenis Parameter yang dibuat di Power BI, yaitu Numeric Range dan Fields.

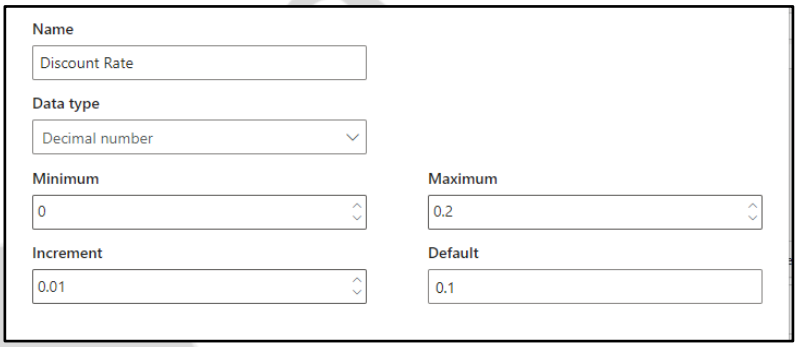
**Parameter Numeric Range**
Parameter ini adalah jenis parameter yang digunakan untuk membuat nilai numerik yang dapat diatur dalam suatu rentang tertentu. Numeric Range sering digunakan dalam skenario di mana kita ingin mengubah nilai angka tertentu untuk melihat dampaknya terhadap hasil analisis, seperti tingkat diskon, jumlah unit, atau target pendapatan.

Fitur-Fitur Utama dari Numeric Range:

- **Nilai Minimum dan Maksimum**: Pengguna dapat menentukan batas minimum dan maksimum dari parameter. Misalnya, jika kita ingin mensimulasikan perubahan harga, kita bisa menetapkan harga minimum sebagai Rp 100 dan harga maksimum sebagai Rp 500.
- **Increment (Kenaikan)**: Langkah atau kenaikan nilai parameter dapat diatur sesuai kebutuhan. Misalnya, kita bisa menetapkan kenaikan sebesar Rp 10, sehingga setiap kali parameter diubah, nilai akan meningkat atau menurun sebesar Rp 10.
- **Visualisasi Slicer**: Power BI secara otomatis menyediakan Slicer (penggeser) yang memungkinkan pengguna untuk mengatur nilai parameter dengan mudah dalam visualisasi mereka.




Sebagai contoh, kita ingin menganalisis dampak perubahan tingkat diskon terhadap total penjualan. Kita dapat membuat parameter Numeric Range yang disebut Discount Rate dengan rentang 0% hingga 20%, dan menetapkan kenaikan sebesar 1% seperti gambar berikut ini:




Numeric Range sangat berguna untuk simulasi skenario yang melibatkan variabel-variabel angka, seperti mengubah harga produk, mengatur tingkat diskon atau markup, dan memodelkan skenario peningkatan atau penurunan volume produksi.

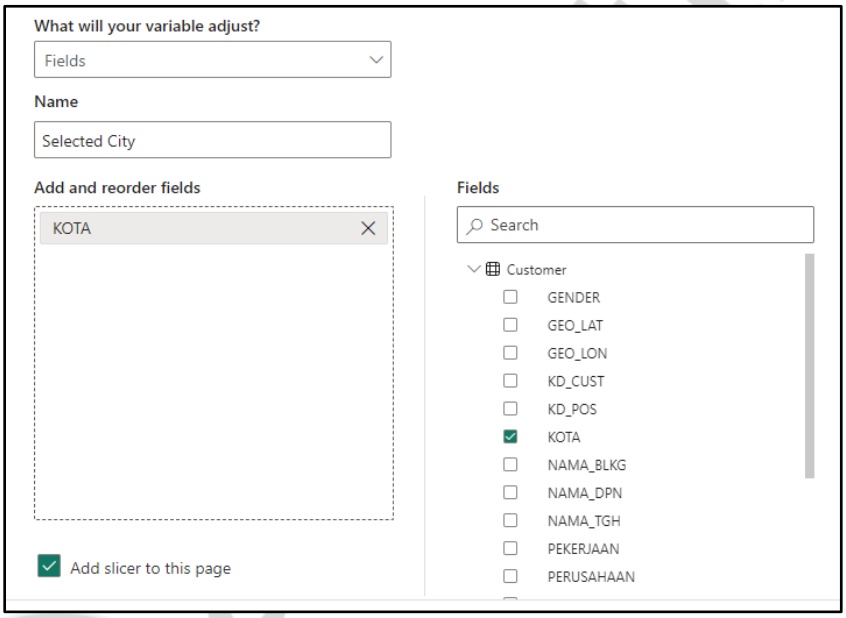
**Parameter Fields**

Selain Numeric Range, Power BI juga mendukung parameter berbasis Fields di mana kita dapat membuat parameter yang memungkinkan pengguna memilih antara berbagai nilai atau kategori yang tersedia dalam model data. Fields Parameter digunakan untuk skenario di mana kita ingin mengubah kategori atau bidang yang dipilih dalam analisis.

Fitur-Fitur Utama dari Fields Parameter:

- **Pilihan Kategori atau Nilai**: Pengguna dapat memilih antara berbagai kategori yang ada di dalam data. Misalnya, kita bisa membuat parameter yang memungkinkan pengguna untuk memilih antara berbagai wilayah penjualan, produk, atau karyawan.
- **Visualisasi Dropdown atau Slicer**: Fields Parameter biasanya muncul dalam bentuk dropdown atau Slicer, yang memungkinkan pengguna memilih kategori atau nilai yang diinginkan dalam visualisasi.
- **Interaksi dengan Visualisasi**: Ketika parameter diubah, visualisasi yang terkait akan memperbarui data berdasarkan nilai atau kategori yang dipilih.

Misalkan kita ingin memberikan pengguna kemampuan untuk memilih wilayah penjualan (misalnya, Wilayah Barat, Wilayah Timur, Wilayah Tengah) dan melihat bagaimana penjualan berubah sesuai wilayah yang dipilih. Dalam hal ini, kita bisa membuat Fields Parameter yang berisi berbagai wilayah penjualan seperti pada gambar di bawah ini:

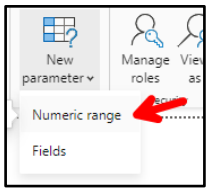



## Using Parameters for Analysis

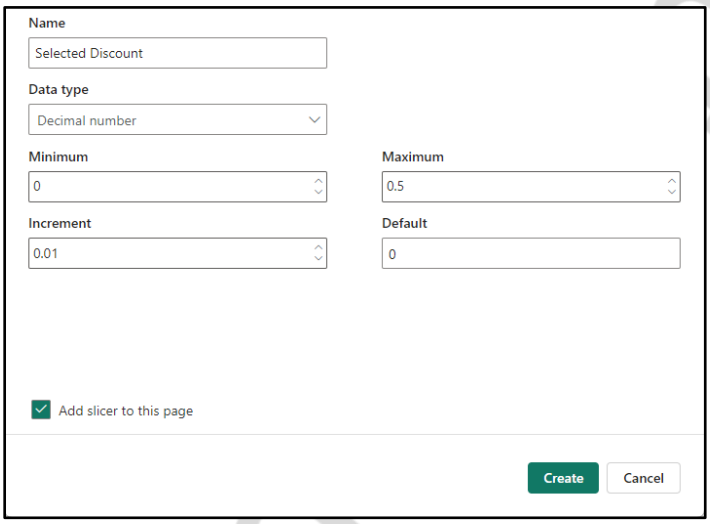
Misal, kita ingin membuat parameter di Power BI untuk mensimulasikan berbagai tingkat diskon. Kita akan menggunakan Numeric Range Parameter yang memungkinkan pengguna memilih tingkat diskon dari 0% hingga 50% dengan kenaikan 1%. Dengan parameter ini, manajemen dapat mensimulasikan bagaimana diskon yang berbeda akan berdampak pada total penjualan dan margin keuntungan.

Pertama-tama kita perlu membuat Parameter untuk skenario diskon tersebut:

1. Pergi ke **Ribbon Modeling** lalu pilih **New Parameter** lalu pilih **Numeric Range**




2. Pada *dialog box* yang muncul kita isikan informasi yang kita butuhkan, yaitu nama parameter “Selected Discount”. Nilai Minimum yaitu 0 dan Maximum yaitu 0.5 dan Increment yaitu 0.01. Kita juga menetapkan nilai default-nya yaitu 0 untuk mengisi nilai Parameter dalam posisi default.




3. Selanjutnya, klik “Create” maka sebuah Slicer berbentuk *slider* akan otomatis terbentuk masuk ke dalam Canvas.




4. Selanjutnya, kita perlu membuat *measure* yang mensimulasikan total penjualan andaikan kita menerapkan diskon tersebut:

```
Total Sales after Discount =
SUMX(
    Sales,
    [Total Sales] * (1 - 'Selected Discount'[Selected Discount Value])
)
```

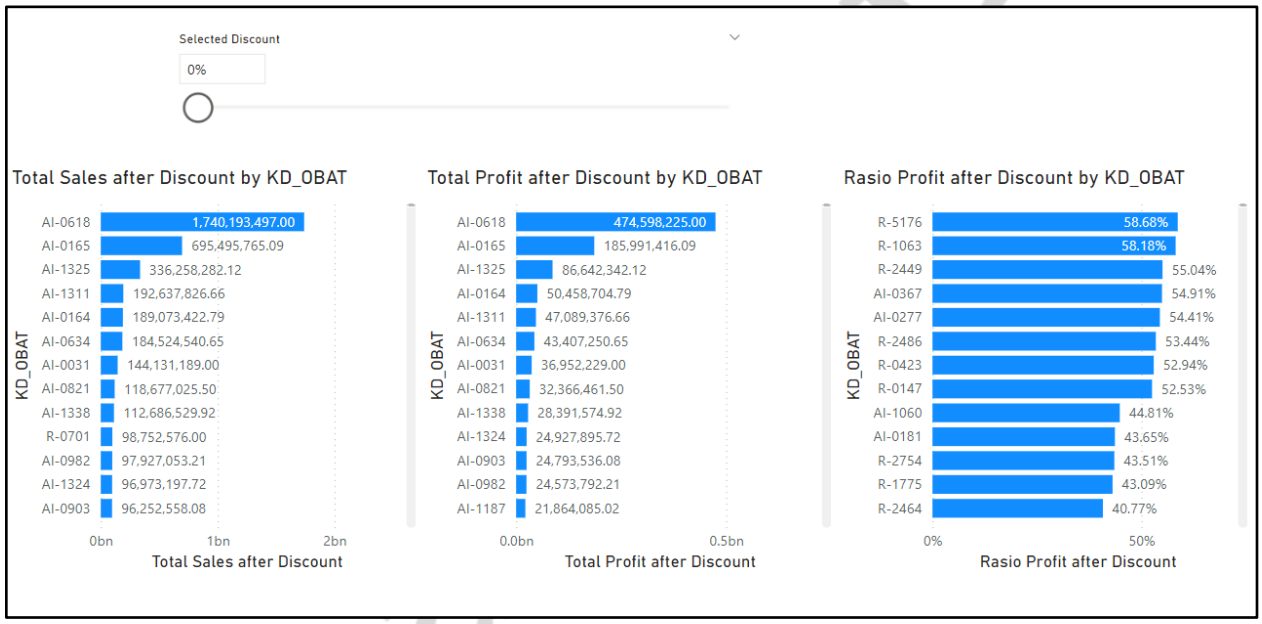
5. Kemudian, kita juga membuat *measure* yang mensimulasikan total keuntungan berdasarkan diskon yang diterapkan:

```
Total Profit after Discount =
[Total Sales after Discount] - [Total Cost]
```

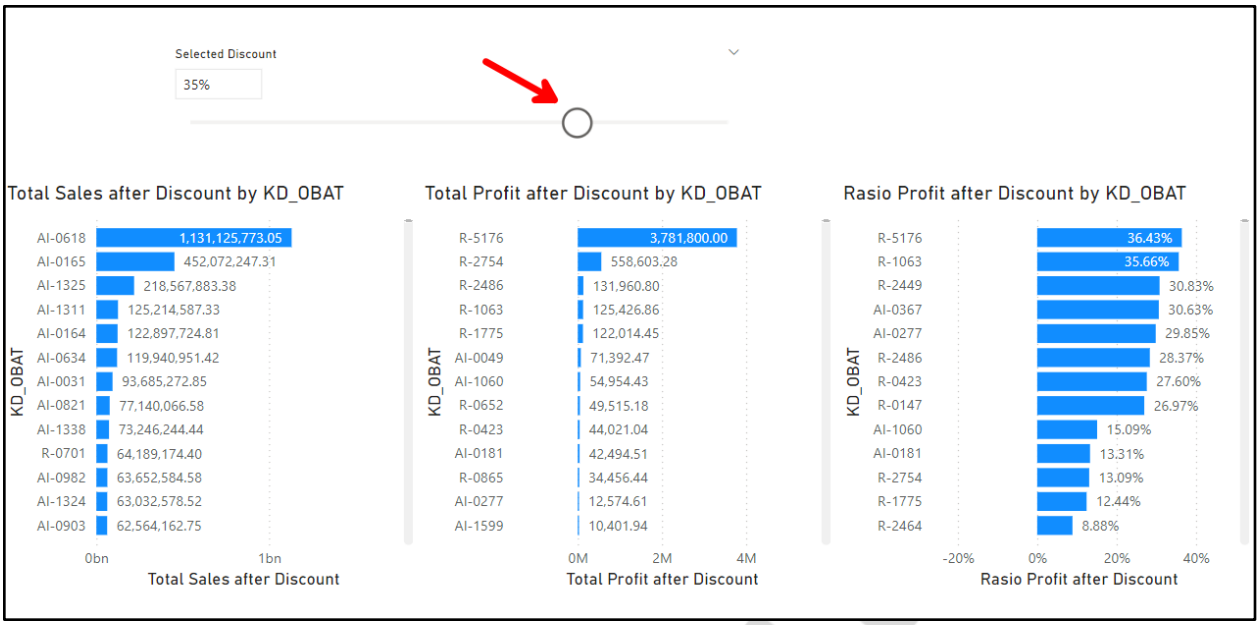
6. Kemudian, kita juga membuat *measure* yang mensimulasikan rasio keuntungan berdasarkan diskon yang diterapkan:

```
Rasio Profit after Discount =
[Total Profit after Discount] / [Total Sales after Discount]
```

7. Terakhir, kita buatkan visualisasi menggunakan kolom “KD_OBAT” dari tabel Sales menggunakan masing-masing *measure* yang sudah kita buat tersebut.

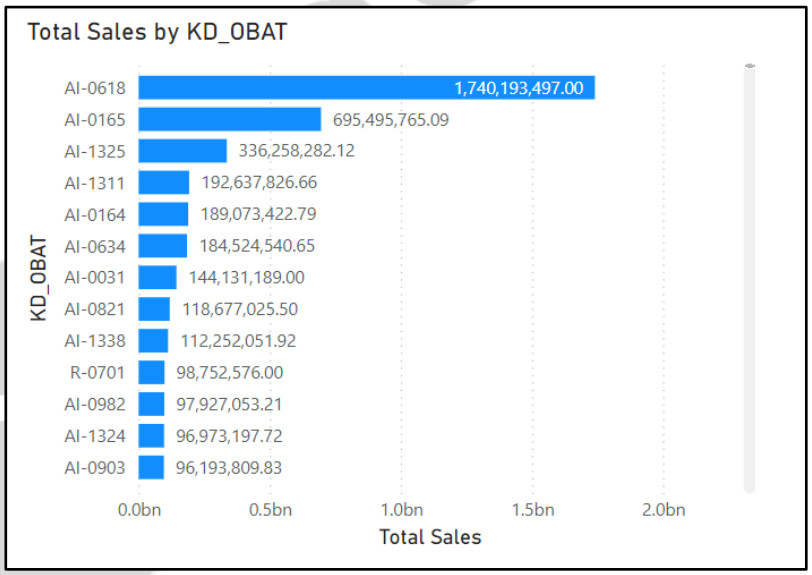



Selanjutnya, apabila kita geser Slicer untuk discount tersebut, maka kita dapat melihat berbagai simulasi total penjualan dan total keuntungan dari berbagai skenario. Dapat dilihat ketika kita menerapkan nilai diskon hingga 35% maka ada beberapa produk yang total profitnya besar begitupun dengan rasio profitnya walaupun total penjualannya tidak sebesar produk lainnya.




## Visualizing Different Scenarios and Outcomes

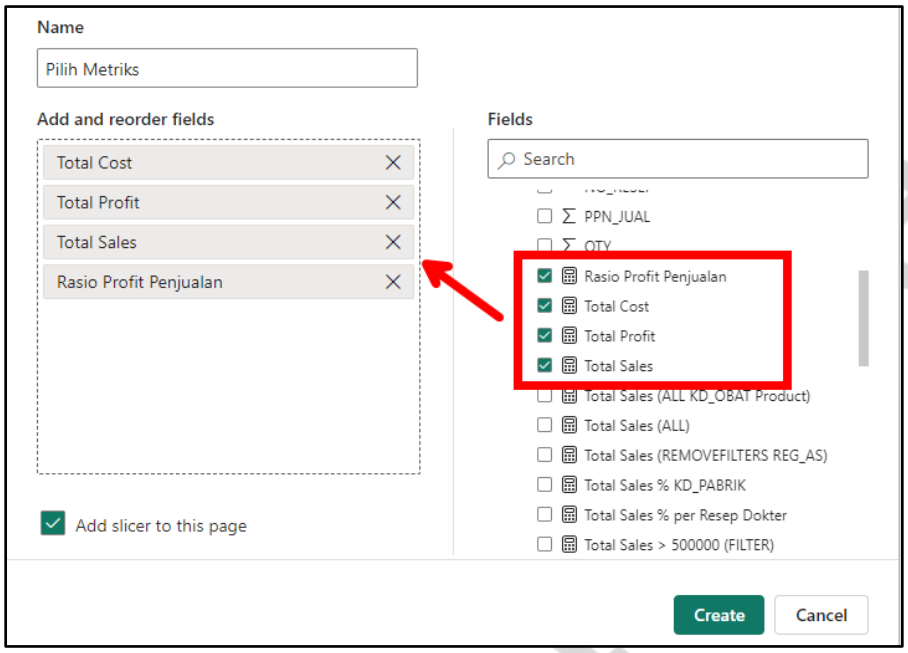
Salah satu contoh pemanfaatan parameter yang sering dibutuhkan di kasus nyata adalah metrik dinamis. Misal, kita memiliki satu buah Bar Chart yang menampilkan kolom “KD_OBAT” dari tabel Sales dengan *measure* `[Total Sales]` sebagai metriknya.



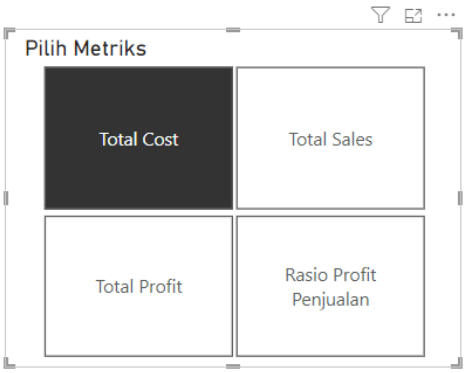

Selanjutnya, kita ingin agar pengguna dashboard dapat memilih dan mengganti metrik yang ingin ditampilkan di Bar Chart tersebut sehingga informasi yang ditampilkan menjadi dinamis dan sesuai kebutuhan. Misal, kita ingin Metrik-metrik yang nantinya bisa dipilih oleh pengguna adalah total penjualan, total biaya modal, total keuntungan, dan rasio keuntungan. Untuk mencapai tujuan tersebut kita bisa memanfaatkan Parameter seperti berikut ini:

1. Pergi ke Ribbon **Modeling** lalu pilih **New parameter** dan pilih **Fields**

2. Akan muncul dialog box. Isi nama parameternya. Kemudian tarik *measure* yang akan dijadikan metrik dinamis ke area kosong di sebelah kiri.




3. Klik “Create”, sebuah slicer akan muncul di Canvas. Atur slicer-nya menjadi “Single select”




Sampai di sini kita sudah berhasil membuat parameter yang akan menjadi tombol pemilih metrik

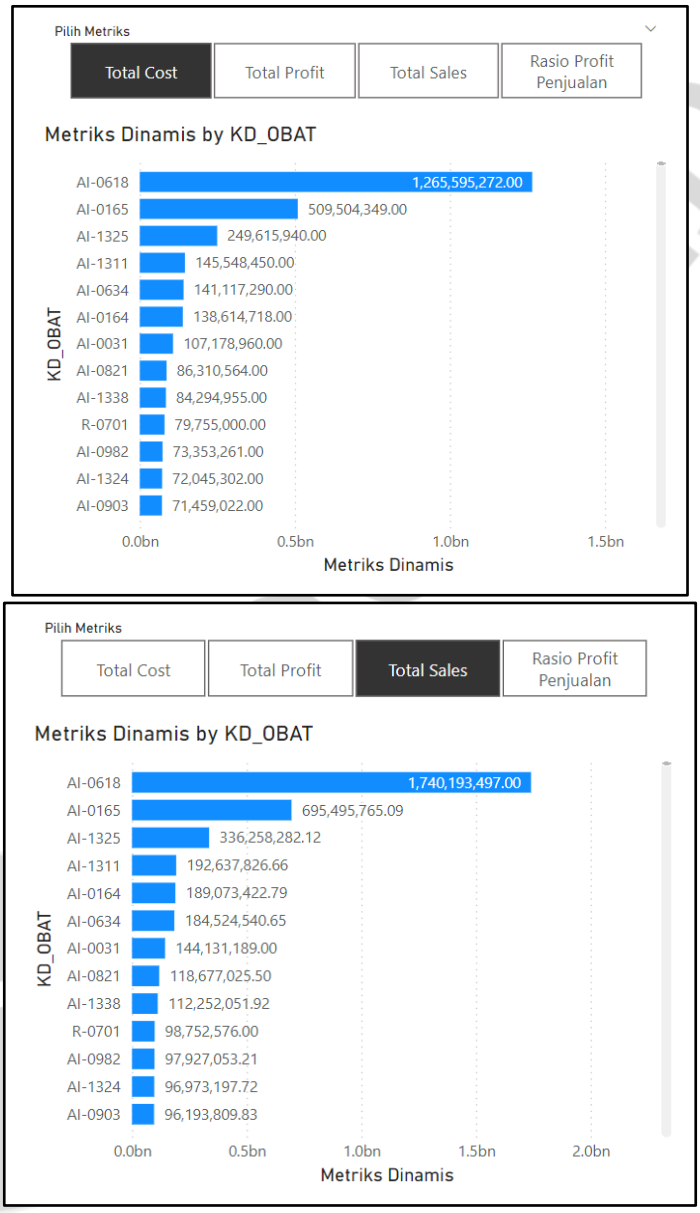
4. Selanjutnya, buat *measure* menggunakan DAX berikut:

```
Metriks Dinamis =
SWITCH(
    VALUES('Pilih Metriks'[Pilih Metriks]),
    "Total Cost", [Total Cost],
    "Total Profit", [Total Profit],
    "Total Sales", [Total Sales],
```

```
    "Rasio Profit Penjualan", [Rasio Profit Penjualan]
)
```

DAX tersebut akan memilih *measure* berdasarkan parameter metrik yang dipilih.

5. Buat bar chart menggunakan “KD_OBAT” dari tabel Sales dan *measure* `[Metriks Dinamis]` yang barusan kita buat




Sekarang nilai yang tampil di Bar Chart menjadi dinamis tergantung metrik yang dipilih oleh pengguna dashboard melalui Slicer yang disediakan.

# Predictive Analytics in Power BI

Power BI menyediakan integrasi ke bahasa pemrograman seperti R dan Python untuk membantu memperluas kemampuan analitik dashboard hingga ke tingkat prediktif. Namun, selain integrasi ke bahasa pemrograman, Power BI juga menyediakan *built-in* fitur visualisasi untuk *predictive analytic* yaitu *Key Influencer*. Kapabilitas *predictive analytic* dari Power BI dapat mempermudah melakukan kebutuhan prediksi untuk pengambilan keputusan berdasarkan proyeksi data di masa depan.

## Finding Influence Factor with Key Influencers Visualization

Visualisasi **Key Influencers** di Power BI adalah alat yang dirancang untuk membantu pengguna memahami faktor atau variabel utama yang paling memengaruhi hasil tertentu. Visualisasi ini menggunakan algoritma *machine learning* untuk menganalisis pola dalam data dan menyoroti faktor-faktor utama yang memiliki dampak signifikan terhadap metrik atau hasil yang dipilih. Dengan menampilkan variabel-variabel berpengaruh ini, pengguna bisa mendapatkan wawasan tentang faktor-faktor pendorong di balik kinerja atau perilaku, yang memungkinkan mereka membuat keputusan berbasis data yang lebih baik.

Visualisasi **Key Influencers** membuat analisis canggih dapat diakses bahkan oleh pengguna tanpa keahlian teknis yang mendalam. memberikan cara yang jelas dan intuitif untuk melihat bagaimana berbagai faktor berkontribusi terhadap hasil, seperti kepuasan pelanggan, kinerja penjualan, atau keberhasilan produk.

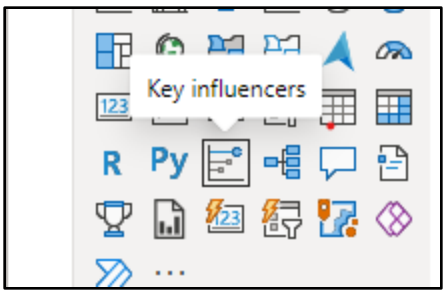
Visualisasi Key Influencer digunakan untuk:

- **Mengidentifikasi Faktor Kunci**: Menunjukkan faktor atau variabel (misalnya, usia, lokasi, jenis produk) yang memiliki pengaruh paling besar terhadap hasil tertentu, seperti kinerja penjualan, *churn* pelanggan, atau skor kepuasan.
- **Menganalisis Pola Data**: Membantu mendeteksi pola dalam data yang sulit dilihat dengan.
- **Memandu Keputusan Bisnis**: Dengan memahami faktor-faktor apa yang paling mempengaruhi hasil, perusahaan dapat menyesuaikan strategi mereka, baik itu untuk mengoptimalkan kampanye pemasaran, meningkatkan pengalaman pelanggan, atau membuat perubahan operasional.

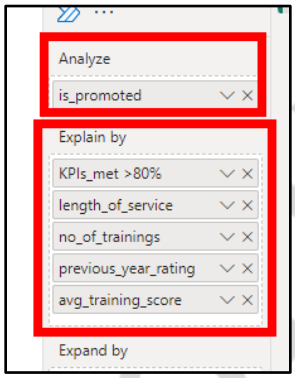
Misal, sebuah perusahaan ingin mengetahui faktor apa yang dapat menyebabkan karyawan dipromosikan sehingga harapannya manajemen bisa memutuskan kapan karyawan layak untuk dipromosikan. Data yang akan digunakan untuk modul ini adalah data *promotion_clean.csv* yang dapat diunduh di sini.

Untuk mulai menggunakan Key Influencers kita bisa lakukan:

1. Pilih Key Influencers

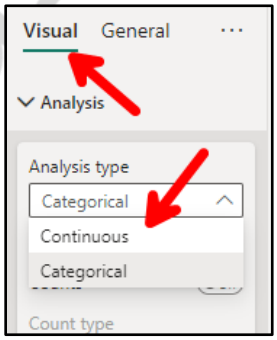



2. Masukkan kolom yang ingin diprediksi ke ***Analyze***. Sementara itu, kolom yang diduga sebagai faktor ke dalam ***Explain by***. Jangan lupa untuk mengganti semua agregasi kolomnya menjadi "***Don't Summarize***"



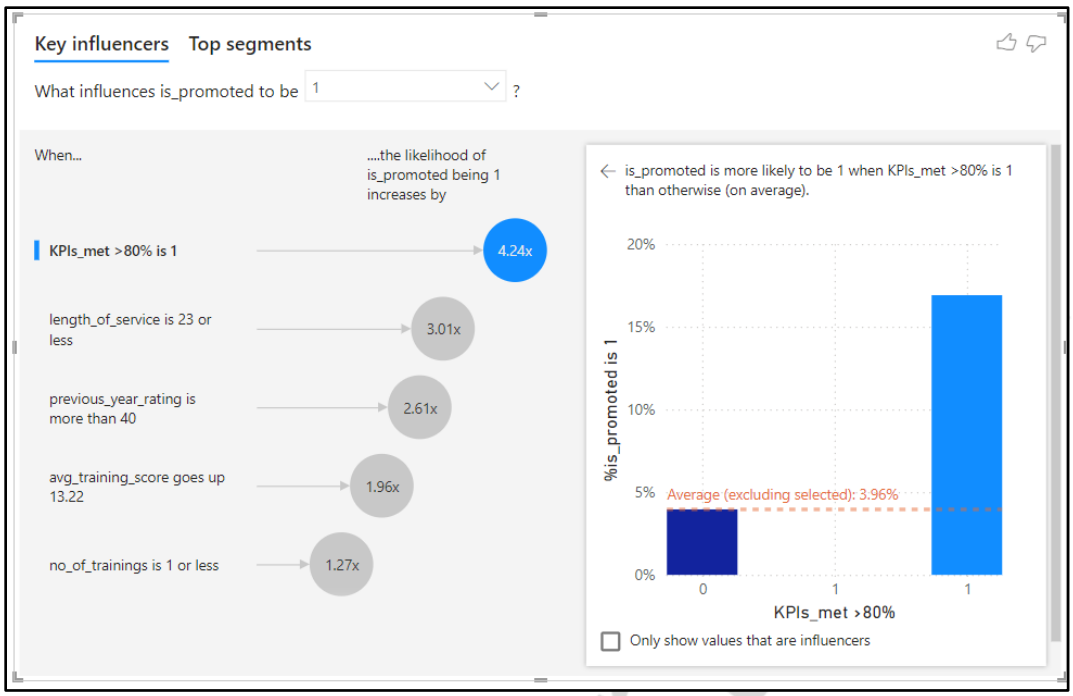

Dalam kasus ini, kita akan mencari faktor-faktor yang menjadi penyebab seseorang dipromosikan. Oleh karena itu, kolom *is_promoted* dimasukkan ke dalam ***Analyze***. Sementara itu, ***Explain by*** kita isi dengan kolom sisanya yang kita ingin lihat bagaimana pengaruhnya terhadap *is_promoted*.

3. Pergi ke **Format Visual** lalu buka tab **Analysis**. Kemudian, buka menu **Analysis Type** dan ganti dari *Continuous* menjadi *Categorical*.




Kita perlu menggantinya menjadi *Categorical* karena kolom *is_promoted* berisi angka 0 dan 1. Sementara itu, angka 0 dan 1 di kolom ini bermakna kategori, 0 untuk *not promoted* dan 1 untuk *promoted*. Hal ini memengaruhi algoritma prediksi yang akan digunakan oleh Power BI untuk menganalisis data kita.

Hasilnya adalah sebagai berikut:



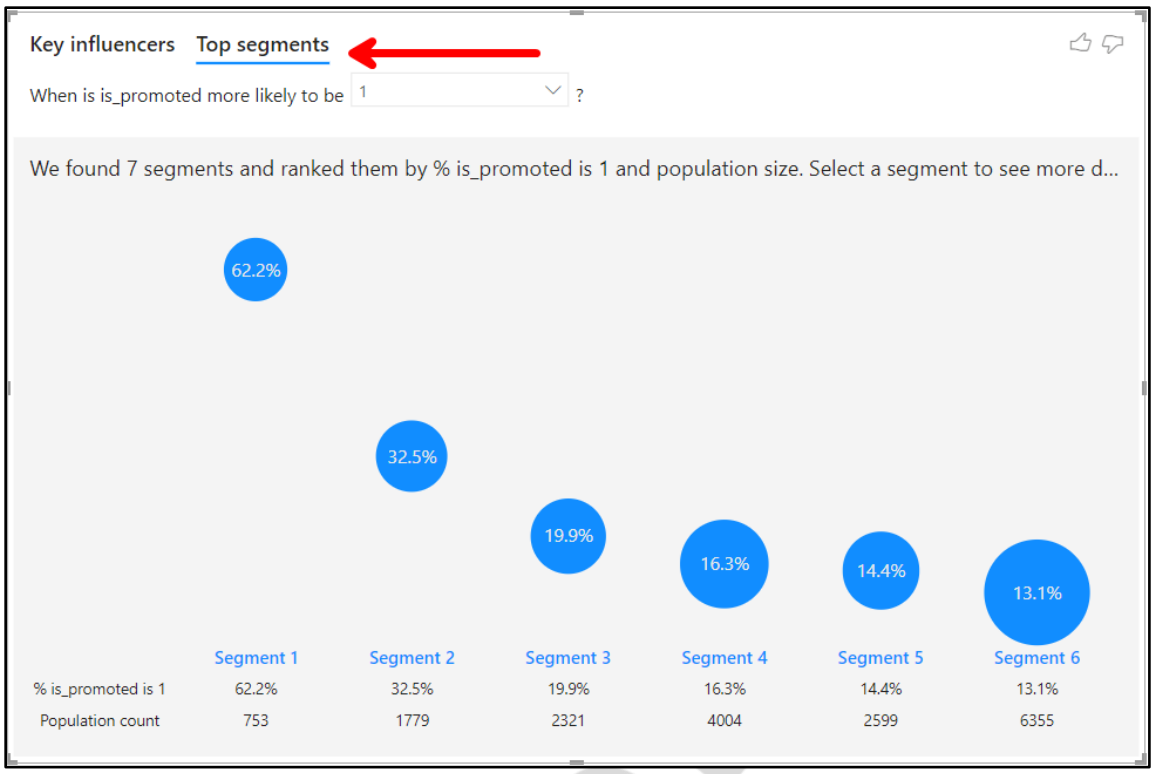

**Key Influencers** akan menghasilkan dua tab analisis yaitu *Key Influencers* dan *Top Segments*.

*Key Influencers***:**
Berisi informasi mengenai bagaimana setiap kolom-kolom di **E*xplain by*** memengaruhi kolom target di ***Analyze***. Dalam kasus ini, kita dapat ambil beberapa poin seperti berikut:

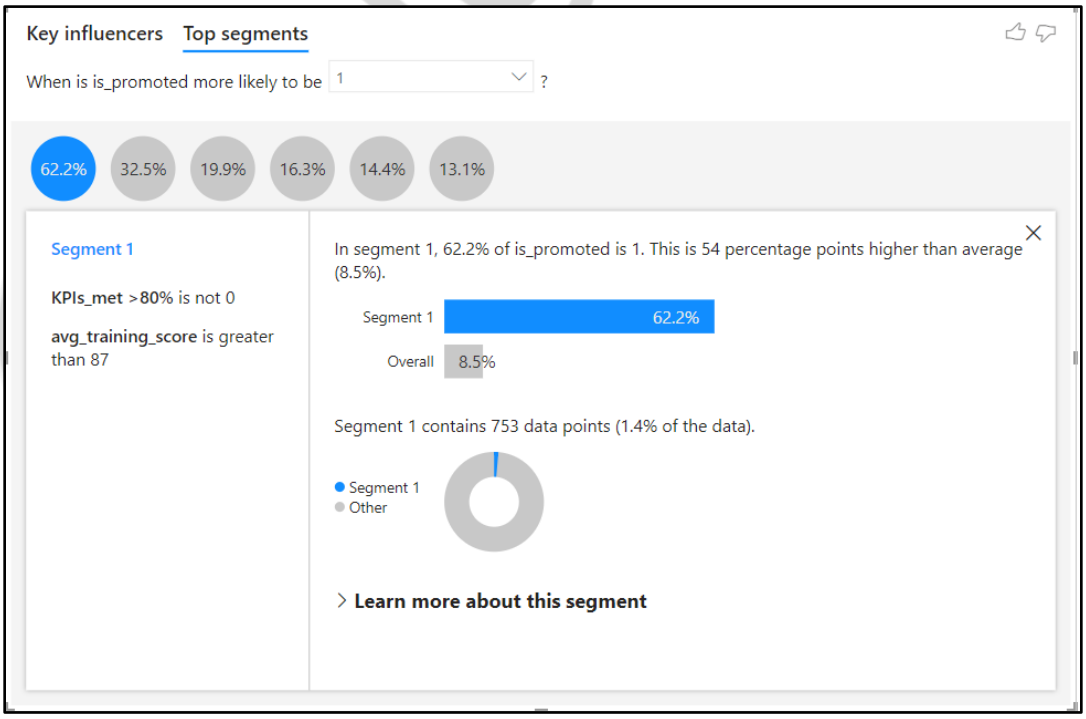
- Ketika seorang karyawan berhasil memenuhi KPI (*KPIs_met>80%*) maka kemungkinan dia untuk dipromosikan akan naik hingga 4.24 kali.
- Ketika *previous_year_rating* seseorang lebih besar dari 40 maka kemungkinan dia untuk dipromosikan akan naik hingga 2.61 kali.
- Ketika *avg_training_score* bertambah 13.22 poin maka kemungkinan untuk dipromosikan juga meningkat hingga 1.96 kali.

Dari sini kita dapat menetapkan standar perusahaan untuk kelayakan seorang karyawan dipromosikan. Hal ini juga dapat membantu kita mengevaluasi apakah ada kejanggalan dalam proses mempromosikan karyawan.

*Top Segments*:
*Top Segments* membagi data ke dalam beberapa segmen berdasarkan kombinasi faktor yang paling berpengaruh, sehingga kita dapat menjelajahi karakteristik setiap segmen dan kaitannya dengan hasil yang dianalisis.




Pada hasil analisis di atas, terbentuk 7 segmen dimana Segmen 1 diisi hingga 62% karyawan yang dipromosikan. Segmen 6 memiliki jumlah karyawan paling banyak (ukuran lingkaran paling besar) namun hanya 13.1% saja karyawan yang dipromosikan di segmen tersebut. Kita bisa klik lingkaran di Segmen 1 untuk melihat karakteristik seperti apa yang dimiliki oleh para karwayan di Segmen 1.



Manfaat menggunakan *Top Segments*:

- **Fokus pada Kelompok Berdampak Tinggi**: kita dapat memprioritaskan upaya pada segmen yang berkontribusi paling signifikan terhadap hasil yang dianalisis, mengoptimalkan sumber daya, dan memaksimalkan dampak.
- **Penargetan dan Personalisasi yang Lebih Baik**: Dengan memahami karakteristik spesifik setiap segmen, bisnis dapat menyesuaikan pemasaran, penawaran produk, atau layanan pelanggan untuk memenuhi kebutuhan berbagai kelompok dengan lebih baik.
- **Penyempurnaan Strategi Berbasis Data**: Wawasan yang diperoleh dari analisis segmen memungkinkan penyempurnaan strategi berkelanjutan berdasarkan segmen mana yang mendorong hasil.
- **Mudah Dipahami dan Ditindaklanjuti**: Fitur *Top Segments* menyediakan visualisasi langsung yang memudahkan pengguna non-teknis untuk memahami dan memanfaatkan wawasan data yang kompleks untuk pengambilan keputusan.

Kelebihan dari menggunakan visualisasi **Key Influencer** adalah:

- **Intuitif dan Mudah Digunakan**: Mudah digunakan, bahkan bagi pengguna non-teknis, karena tidak memerlukan pengetahuan mendalam tentang *data science* atau *machine learning* untuk mendapatkan wawasan.
- **Wawasan Otomatis**: Secara otomatis menampilkan faktor kunci dari dataset yang besar dan kompleks, menghemat waktu dan mengurangi kebutuhan analisis manual.
- **Data yang Dapat Ditindaklanjuti**: Wawasan yang diberikan oleh visualisasi ini langsung dapat diterapkan pada pengambilan keputusan, membantu bisnis membuat perbaikan yang tepat sasaran dan efektif.

Visualisasi Key Influencer di Power BI adalah alat yang sangat berguna untuk analitik prediktif, memungkinkan pengguna untuk menemukan faktor-faktor yang paling memengaruhi hasil bisnis. Baik dalam menganalisis kinerja penjualan, perilaku pelanggan, efisiensi operasional, atau metrik penting lainnya, visualisasi ini memungkinkan pengambilan keputusan berbasis data dengan mudah.

## Machine Learning with Python in Power BI

Power BI menyediakan visual dengan bahasa pemrograman salah satunya adalah Python. Kita bisa menggunakan fitur ini untuk menerapkan model *machine learning* sehingga bisa melakukan prediksi terhadap data kita. Kali ini, kita akan membuat sebuah dashboard analitik yang berfungsi untuk memprediksi kemungkinan seorang karyawan untuk dipromosikan. Harapannya dasbor ini bisa digunakan oleh perusahaan sebagai pertimbangan untuk mempromosikan seorang karyawan.

Kita akan menggunakan kolom *is_promoted* sebagai variabel target, yaitu variabel yang akan diprediksi. Sementara itu, variabel prediktornya (variabel yang menjadi faktor) adalah kolom:

- *avg_training_score*
- *length_of_service*
- *age*

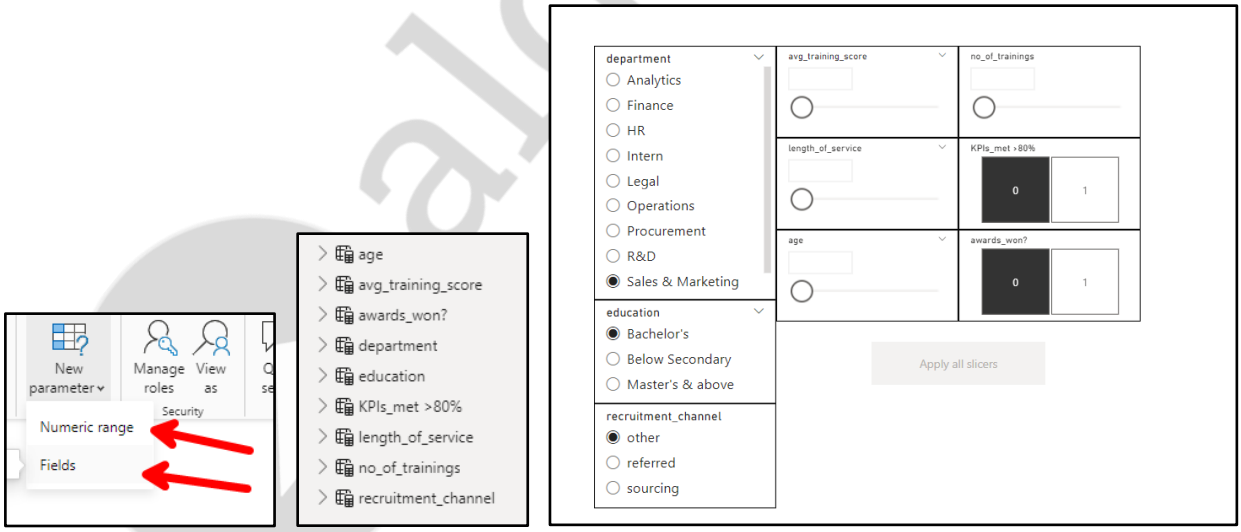
- *no_of_training*
- *KPIs_met >80%*
- *awards_won?*
- *education*
- *department*
- *recruitment_channel*.

Download model *machine learning* yang akan kita gunakan **DI SINI**. Nama filenya adalah "*promotion_model.pkl*". Simpan file model tersebut di lokasi yang mudah ditemui.

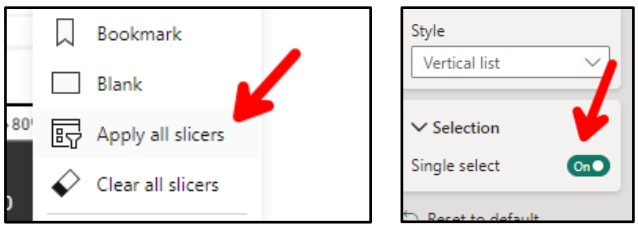
Sebelum itu, kita perlu untuk membuat parameter sebagai nilai input yang akan diprediksi. Kita gunakan parameter *Numeric Range* untuk kolom angka dan parameter *Fields* untuk kolom teks. Parameter ini nantinya bisa diatur sesuai kebutuhan *end-user* untuk memprediksi promosi karyawan. Karena ada 9 kolom yang digunakan sebagai variabel prediktor maka kita perlu membuat 9 parameter:

- 3 Parameter *Fields* masing-masing untuk kolom *department*, *education*, dan *recruitment_channel*
- 6 Parameter *Numeric Range* masing-masing untuk kolom sisanya.

Berikan masing-masing nama Parameter yang sama persis dengan nama kolomnya. Hal ini perlu dilakukan karena model *machine learning* yang akan digunakan sudah dilatih untuk memprediksi menggunakan nama-nama kolom tersebut. Jika sudah maka tampilannya akan seperti gambar paling kanan berikut:

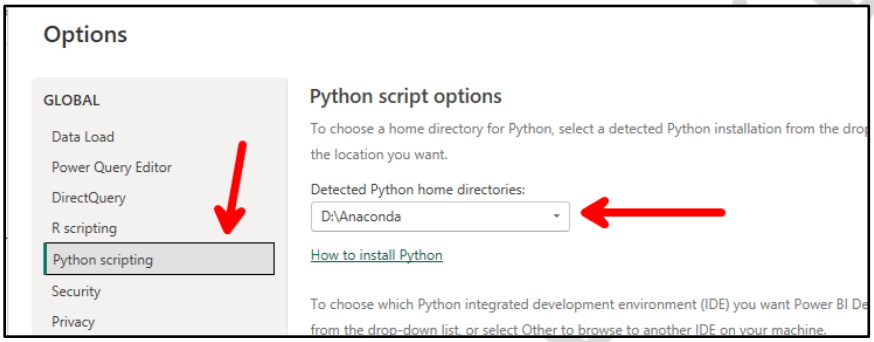



Setiap parameter ditampilkan dalam bentuk *slicer* untuk digunakan oleh *end-user* sebagai input. Tidak lupa juga kita masukkan tombol *Apply All Slicers*. Tombol ini berfungsi untuk menahan model *machine learning* agar tidak langsung memprediksi setiap kali nilai pada *slicer* diganti. Selain itu, pastikan juga agar pengaturan *slicer* sudah "***Single Select***"



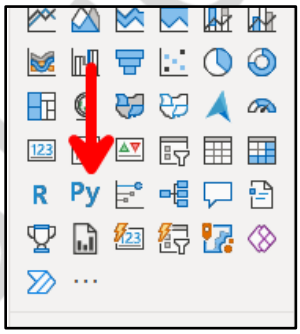

Setelah *input*-nya sudah diatur, baru kita bisa mulai menyiapkan Python untuk memprediksi berdasarkan nilai yang diberikan di setiap parameter:

1. Pergi ke ***File → Options and settings → Settings***
2. Pergi ke ***Global → Python Scripting***

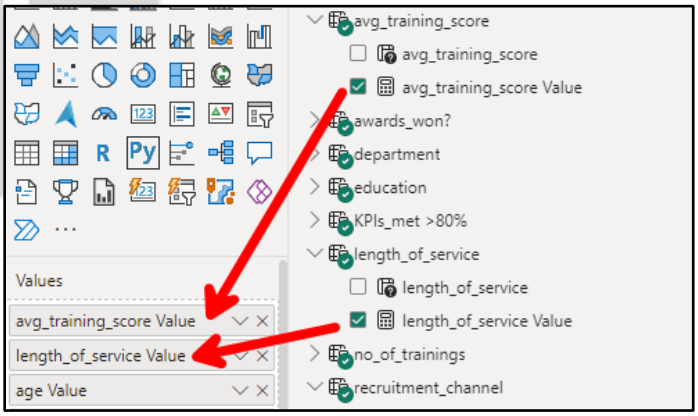



Pastikan Detected Python Home Directories sudah pada environment Python yang sesuai

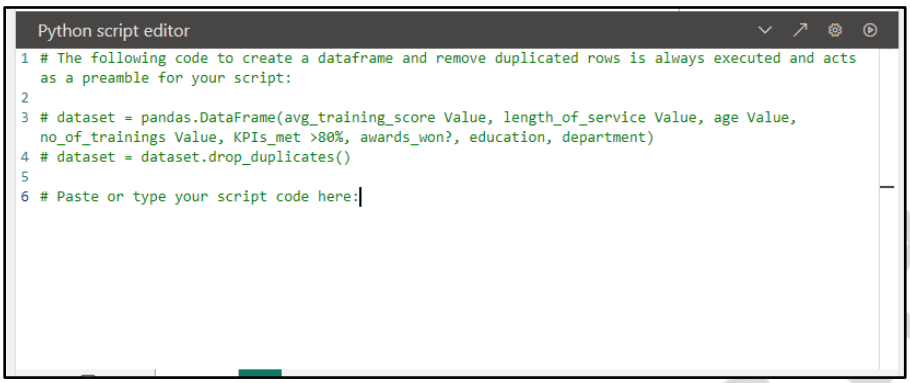
3. Masukkan visual **Python**



4. Masukkan 9 parameter ke ***Values***. Khusus untuk parameter *Numeric Range* yang dimasukkan adalah Value-nya



5. Akan muncul *dialog box* Python seperti berikut di sisi bawah




Di baris paling bawah tambahkanlah *script* berikut:

```
import joblib
import sklearn
import matplotlib.pyplot as plt

#mengganti nama parameter yang berakhiran Value
dataset = dataset.rename(columns={
    'avg_training_score Value': 'avg_training_score',
    'length_of_service Value': 'length_of_service',
    'age Value': 'age',
    'no_of_training Value': 'no_of_trainings',
    'KPIs_met >80% Value': 'KPIs_met >80%',
    'awards_won Value': 'awards_won?',
    })

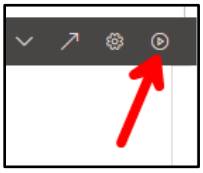
#mengatur ulang urutan kolom agar sesuai dengan yang sudah dipelajari model
dataset = dataset[['avg_training_score', 'length_of_service', 'age',
'no_of_trainings', 'KPIs_met >80%', 'awards_won?', 'education',
'department', 'recruitment_channel']]

#PENTING!!!! memanggil model machine learning
model = joblib.load("C://Users//Asani//Desktop//promotion_model.pkl")
#pastikan lokasinya sudah diganti dengan tempat kita menyimpan modelnya

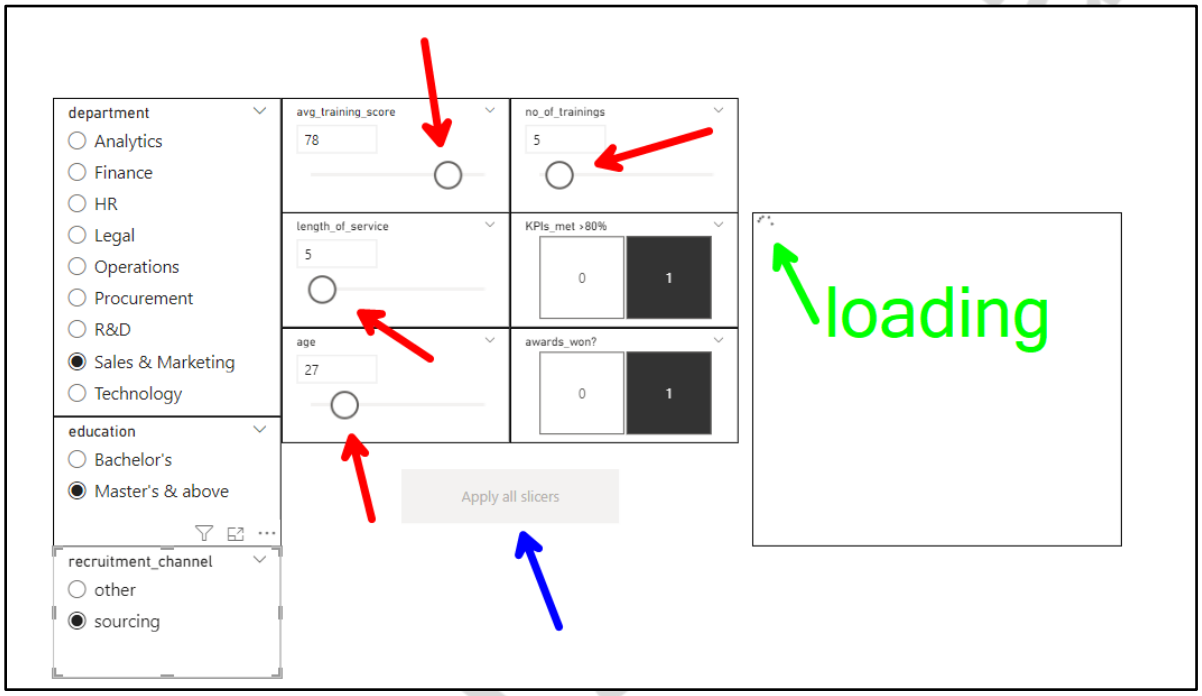
#model memprediksi promosi sesuai dengan nilai yang ada di parameter
result = model.predict(dataset)

#menampilkan hasilnya
fig = plt.figure()
ax = fig.add_subplot()
ax.axis([0,1,0,1])
plt.grid(False)
plt.axis('off')
ax.text(0.3, 0.5, 'Promoted' if str(result[0])=='1' else 'Not Promoted',
fontsize=30)
plt.show()
```

6. Klik tombol *run* di kanan atas



7. Jika kita mengganti-ganti nilai parameternya, jangan lupa untuk klik "***Apply All Slicers***"




Akan muncul lambang *loading* di kiri atas visual Python menandakan model sedang memprediksi.